MATHEMATICS FOR AI — READING EDITION

Math for AI

A printable companion: from foundations to deep-learning applications

---

Contents

Part I. Calculus for AI
Single-variable through multivariable, optimisation, and the loss landscape

Part II. Linear Algebra for AI
Vectors, matrices, eigenvalues, SVD — the data structures of deep learning

Part III. Probability & Statistics for AI
Distributions, Bayes, information theory, sampling, inference

PART I

Calculus for AI

Single-variable through multivariable, optimisation, and the loss landscape

Print-friendly reading edition

01 Functions & Graphs

Mapping inputs to outputs — what every neural network IS

🎛

Plain English: A function is a machine — one input in, one output out. f(3)=9 means input 3, output 9. The graph plots every (input, output) pair. A neural network IS a function: pixels go in, class probabilities come out. Activation functions determine what kind of function each layer can express.

AI Deep Dive — Functions & Graphs

🧠 The Concept

A function is a deterministic rule: every input produces exactly one output. Mathematically **f: X → Y** assigns each x ∈ X a single value f(x) ∈ Y. The graph is the set of all (x, f(x)) pairs visualised as a curve. Some functions are **linear** (straight lines, the only ones that obey f(a+b) = f(a)+f(b)); most useful ones are **non-linear** (curves, kinks, jumps). What separates ML's preferred non-linearities — ReLU, GELU, sigmoid, tanh — from arbitrary curves is that each was chosen because of what its *derivative* looks like, not its output.

💡 ***Intuition:*** *A neural network is a giant nested function $f(x) = f_n(...f_2(f_1(x)))$. Training picks the weights inside each layer's function. The output curve is just the visible shadow of millions of tiny linear+nonlinear pieces composed together.*

? Why AI needs this

Neural networks are functions. f(x;W) maps inputs to predictions. Training = finding W so f(x;W)≈y for all training data.

- Activation functions chosen for derivative properties, not just output shape
- ReLU derivative ∈{0,1} — enabled deep learning by preventing vanishing gradients
- Sigmoid max derivative 0.25 — caused 1990s deep learning failure in 10+ layer nets
- GELU smooth everywhere → stable gradient flow in 100-layer transformers

⚙ How AI uses it

Every layer applies: **output = activation(Wx+b)**

```
layer = nn.Sequential(
  nn.Linear(784, 256),
  nn.GELU(),        # GPT-2/3, BERT, T5
  nn.Linear(256, 10),
  nn.Softmax(dim=-1)
)
```

🧮 Worked Example

Compare what three popular activations actually output at x = 1. They all "fire" but the shape of their curves is what matters for training.

Evaluating activations at x = 1

GIVEN	x = 1, sigmoid $\sigma(z) = 1/(1+e^{-z})$, GELU(x) ≈ x · σ(1.702x)
RELU	ReLU(1) = max(0, 1) = 1.000 — passthrough, slope 1
SIGMOID	$\sigma(1) = 1/(1 + e^{-1}) = 1/(1 + 0.368) = 1/1.368 = 0.731$
GELU	$GELU(1) = 1 \cdot \sigma(1.702) = 1 \cdot 1/(1 + e^{-1.702}) = 1 \cdot 0.846 = 0.846$
RESULT	[ReLU, GELU, Sigmoid] → [1.000, 0.846, 0.731]. GELU is the smooth ReLU GPT/BERT prefer.

02 Limits

The foundation of every derivative — and therefore all of training

> 🗣
>
> **Plain English:** "The limit as x→2" asks: what does f(x) approach as x gets infinitely close to 2 from both sides? We care about the destination, not whether we arrive. Every derivative is defined as a limit. Without limits → no derivatives → no gradients → no training.

```
f′(x) = lim[h→0] (f(x+h)−f(x))/h
```

AI Deep Dive — Limits

📚 The Concept

A limit asks: as x gets arbitrarily close to some value a (without ever reaching it), what value does f(x) approach? The notation is $\lim_{x \to a} f(x) = L$. Crucially the limit cares only about the *destination*, not whether f is even defined at a. This is what lets us divide by "zero" safely: (f(x+h) − f(x)) / h is undefined at h=0, but its limit as h→0 is well-defined and gives us the derivative. Limits also formalise **continuity** (limit equals the value), **convergence** (training loss approaches its minimum), and **asymptotic behaviour** (what happens as x → ∞).

> 💡 ***Intuition:*** *A limit is a "what would happen if we got infinitely close" thought experiment. Calculus is built on the trick of taking ratios that look like 0/0, then computing what they approach instead of what they equal.*

? Why AI needs this

The derivative is defined as a limit. Without limits there is no gradient, no backprop, no training signal.

- lim[h→0](f(x+h)−f(x))/h — this limit IS the gradient
- Continuity (limit=value) required for gradient descent to work
- Convergence of training loss is formally a limit statement
- torch.autograd.gradcheck() uses the limit definition with small h

⚙ How AI uses it

Gradient checking verifies autograd using the limit definition directly:

```
h = 1e-5
# Numerical gradient = limit definition
grad_num = (loss(w+h) - loss(w-h)) / (2*h)
# Should match autograd:
loss.backward()
assert abs(grad_num - w.grad) < 1e-4
# torch.autograd.gradcheck() does this
```

🧮 Worked Example

The famous limit $\lim_{h \to 0} \sin(h)/h$. The function is undefined at h=0 (you'd be dividing 0/0), but the limit is exactly 1. Watch the values converge.

Numerically computing $\lim_{h \to 0} \sin(h)/h$

GIVEN	f(h) = sin(h) / h, undefined at h = 0
H = 0.5	sin(0.5)/0.5 = 0.4794/0.5 = 0.9589
H = 0.1	sin(0.1)/0.1 = 0.0998/0.1 = 0.9983
H = 0.01	sin(0.01)/0.01 = 0.00999983/0.01 = 0.999983
H = 0.001	sin(0.001)/0.001 = 0.99999983
LIMIT	$\lim_{h \to 0} \sin(h)/h = 1$, even though f(0) doesn't exist. This is exactly how gradient checking samples a derivative.

03 The Derivative

Instantaneous slope — the only learning signal in every neural network

📐

Plain English: The derivative f′(x) asks: "if x increases by a tiny ε, how much does f change?" That ratio Δf/Δx as ε→0 is the slope of the tangent line at x. In AI: ∂L/∂w tells training whether increasing weight w raises or lowers the loss. Positive → decrease w. Negative → increase w. This IS the learning signal.

```
f′(x) = lim[h→0] (f(x+h)−f(x))/h
Weight update: w ← w − α × ∂L/∂w
```

AI Deep Dive — The Derivative

📚 The Concept

The derivative **f′(x)** measures the instantaneous rate of change of f at the point x. Geometrically it's the slope of the tangent line at x. Algebraically it's defined by the limit **$f'(x) = \lim_{h\to 0} (f(x+h) - f(x)) / h$** — average slope over an interval, taken as the interval shrinks to a point. The sign of f′ tells direction (rising vs falling); the magnitude tells speed of change. A function is *differentiable* at x if this limit exists and is the same approaching from both sides — ReLU famously fails this at x=0 (left slope 0, right slope 1).

💡 ***Intuition:*** *If your loss is a hill and your weight is your position, the derivative is the steepness right under your feet. Walk downhill — that's gradient descent in one sentence.*

? Why AI needs this

The derivative is the only learning signal. ∂L/∂w tells every optimizer exactly how to change each weight.

- Positive ∂L/∂w → increasing w raises loss → decrease w
- Negative ∂L/∂w → increasing w lowers loss → increase w
- Zero ∂L/∂w → at an extremum → stop updating (converged)
- |∂L/∂w| controls step size — large gradient = aggressive update

⚙ How AI uses it

param.grad stores ∂L/∂param after loss.backward():

```
loss.backward()  # compute ∂L/∂w for every w
for p in model.parameters():
    p.data -= lr * p.grad   # w ← w − α(∂L/∂w)
# Clip to prevent exploding gradients:
torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
```

🧮 Worked Example

Quadratic loss $L(w) = (w - 3)^2$ with current weight w = 5. Compute the derivative two ways — numerical (limit definition) and analytical — then take one gradient descent step.

Derivative of $L(w) = (w - 3)^2$ at w = 5

GIVEN	$L(w) = (w - 3)^2$, w = 5, learning rate α = 0.1
STEP 1	Numerical (h = 0.001): $L(5.001) = (2.001)^2 = 4.004001$, $L(4.999) = (1.999)^2 = 3.996001$
STEP 2	(L(5.001) − L(4.999)) / (2 · 0.001) = 0.008 / 0.002 = 4.000
STEP 3	Analytical: L′(w) = 2(w − 3), so L′(5) = 2 · 2 = 4 ✓ (matches numerical)
STEP 4	Update: w ← w − α · L′(w) = 5 − 0.1 · 4 = 4.6
RESULT	New loss $L(4.6) = 1.6^2 = 2.56$ (down from 4.0). One step moved closer to the optimum at w = 3.

04 Common Derivatives

The 8 rules you use every single day in machine learning

Plain English: Memorize these like multiplication tables. Most critical: e^x is its own derivative — unique in all of mathematics. d/dx[ln x]=1/x appears in every cross-entropy backprop. ReLU's binary derivative {0,1} is why it enabled deep learning in 2012.

⚠ **Vanishing gradient in numbers:** sigmoid max derivative = 0.25. In a 10-layer network: 0.25 × 0.25 × … (10 times) = 0.25^{10} ≈ 0.0000001. First layers receive gradient ≈ 0 and cannot learn. ReLU's derivative=1 when active — no shrinkage — solved this in AlexNet (2012).

f(x)	f′(x)	AI use	Key fact
x^n	$n \cdot x^{n-1}$	All polynomial layers	Power rule — most used
e^x	e^x (itself!)	Softmax numerator	Self-derivative — mathematically unique
ln(x)	1/x	Cross-entropy backprop	$\partial/\partial\hat{y}[-\log \hat{y}] = -1/\hat{y}$
sin(x)	cos(x)	Positional encodings	Smooth oscillation
sigmoid(x)	$\sigma(1-\sigma) \leq 0.25$	Binary classification	MAX 0.25 → vanishing gradient!
tanh(x)	$1-\tanh^2(x)$	LSTM/GRU gates	Zero-centered, max derivative = 1
ReLU(x)	0 (x<0) or 1 (x>0)	All deep networks	Never shrinks gradient — enabled deep learning
GELU(x)	$\sigma(1.702x) + x \cdot \sigma'(\ldots)$	GPT, BERT hidden layers	Smooth everywhere unlike ReLU

AI Deep Dive — Common Derivatives

📚 The Concept

You almost never compute derivatives from the limit definition. Instead you memorise a small library of "atomic" derivatives plus four combination rules (sum, product, quotient, chain) and assemble the rest. The most important atomic ones for ML are the **power rule** ($x^n \to nx^{n-1}$), **exponential** ($e^x \to e^x$, the only function that is its own derivative), **logarithm** (ln x → 1/x, which appears in every cross-entropy backprop), and the activation derivatives — sigmoid, tanh, ReLU, GELU. The derivative of an activation at a point is what determines whether gradient flows through that neuron or gets killed.

💡 ***Intuition:*** *Activations are picked for the shape of their derivative, not their output. Sigmoid's max derivative 0.25 means 10 stacked layers shrink gradients by 4 million times. ReLU's derivative is 0 or 1 — never shrinks. That single property birthed deep learning.*

? Why AI needs this

Every backward pass multiplies derivatives via the chain rule. Whether each derivative vanishes, stays constant, or explodes determines whether deep networks can be trained at all.

- d/dx[e^x]=e^x → softmax backprop trivial — gradient of exp(z) is just exp(z)
- d/dx[ln x]=1/x → cross-entropy gradient $-y/\hat{y}$ grows large when model is very wrong
- ReLU′∈{0,1} → gradient never shrinks — 10 layers of ReLU still gets same gradient magnitude
- Sigmoid max 0.25 → $0.25^{10} \approx 10^{-6}$ at layer 1 → 1990s deep net failure

⚙ How AI uses it

Derivatives of activations appear in every backward pass:

```
# Sigmoid: d/dz = s*(1-s) ≤ 0.25
s = torch.sigmoid(x); ds = s * (1 - s)
# ReLU: d/dz = 0 or 1 (never shrinks)
dr = (x > 0).float()
# Cross-entropy gradient: d/dŷ[-log ŷ] = -1/ŷ
dL_dy = -target / prediction  # 1/x rule
```

🧮 Worked Example

Trace what happens to a gradient flowing through a 10-layer sigmoid network vs a 10-layer ReLU network. This is the calculation that explains why every modern deep network uses ReLU-family activations.

Vanishing gradient — sigmoid vs ReLU after 10 layers

GIVEN	Same incoming gradient = 1.0 entering a stack of 10 layers
SIGMOID	$\sigma'(z) = \sigma(z)(1 - \sigma(z))$. Maximum at z=0: 0.5 · 0.5 = 0.25

10 LAYERS	Best case: $0.25^{10} = 9.54 \times 10^{-7} \approx 0.00000095$
RELU	$\mathrm{ReLU}'(z) = 1$ if $z > 0$, else 0. Active neurons multiply by 1 — no shrinkage.
10 LAYERS	If all active: $1^{10} = 1.0$ — gradient passes through unchanged
RATIO	ReLU delivers ~1 million × more gradient to layer 1 than sigmoid. This is why deep CNNs / transformers became possible.

05 Higher-Order Derivatives

f″ measures curvature — whether you're in a bowl or on a hill

Plain English: f′ is speed; f″ is acceleration. f″>0 everywhere means the function is bowl-shaped — concave up. f″<0 means hill-shaped — concave down. Curvature tells gradient descent whether it's converging toward a minimum or diverging away from a maximum. It also determines the maximum safe learning rate.

```
f″>0 → concave up (bowl) → minimum candidate
f″<0 → concave down (hill) → maximum
Safe learning rate: α < 2/max|f″(x)|
```

AI Deep Dive — Higher-Order Derivatives

The Concept

The first derivative f′ tells you how fast f is changing. The second derivative **f″** = (f′)′ tells you how fast f′ is changing — the rate of change of the rate of change. Geometrically: f′ is slope, f″ is curvature. f″ > 0 means the slope is increasing (function curves upward, like a bowl ◡); f″ < 0 means slope decreasing (curves downward, like a hill ◠); f″ = 0 is an inflection point. You can keep going — f‴, $f^{(4)}$, etc. — but second derivatives are the workhorse in optimisation because they decide whether a critical point is a minimum, a maximum, or a saddle.

Intuition: *If f′ is velocity, f″ is acceleration. Gradient descent uses only velocity ("which way is downhill"). Newton's method and Adam use acceleration too ("how fast is the slope changing") and converge in far fewer steps near a minimum.*

? Why AI needs this

The second derivative measures curvature — how the slope itself is changing. It determines safe learning rates and whether a critical point is a minimum.

- f″>0 everywhere = convex loss = gradient descent always converges (logistic regression)
- Max eigenvalue of Hessian = max safe learning rate: α_max = 2/λ_max
- Adam uses diagonal Hessian approximation to set per-parameter step sizes
- Newton's method: x←x−f′/f″ — curvature-corrected, fewer iterations

How AI uses it

Adam approximates the diagonal of the Hessian via squared gradients:

```
# Adam: v ≈ diagonal of Hessian (f″ per parameter)
v = beta2*v + (1-beta2)*(grad**2)
# Divide by sqrt(curvature) → normalized step
param -= lr * m_hat / (torch.sqrt(v_hat) + eps)
# Safe lr from curvature: α < 2/max_eigenvalue
```

Worked Example

Use the second derivative to classify the two critical points of **$f(x) = x^3 - 6x^2 + 9x$**. The "second derivative test": f′(x*)=0 and f″(x*) > 0 → local min, f″(x*) < 0 → local max.

Classifying critical points via f″

GIVEN	$f(x) = x^3 - 6x^2 + 9x$
STEP 1	$f'(x) = 3x^2 - 12x + 9 = 3(x - 1)(x - 3)$ → critical points at x = 1, x = 3
STEP 2	$f''(x) = 6x - 12$
AT X=1	$f''(1) = 6 - 12 = -6 < 0$ → curves downward → **local maximum**
AT X=3	$f''(3) = 18 - 12 = +6 > 0$ → curves upward → **local minimum**
RESULT	f(1) = 4 (local max), f(3) = 0 (local min). f″ alone classifies which is which without plotting.

06 Partial Derivatives

Differentiate one variable at a time — all others frozen

🏔

Plain English: Imagine a mountain h(x,y). ∂h/∂x asks: "if I walk east (x increases, y frozen), how steep?" ∂h/∂y asks the same going north. Neural networks have millions of weights — training computes ∂L/∂w for every single one simultaneously in one backward pass.

```
∂f/∂x: differentiate w.r.t. x, treating y as a constant
Example: f(x,y)=x²+2xy → ∂f/∂x=2x+2y, ∂f/∂y=2x
```

AI Deep Dive — Partial Derivatives

🧠 The Concept

Most real functions take many inputs: f(x, y, z, …). A partial derivative **∂f/∂x** asks the same question as a normal derivative — "how does the output change when this one input wiggles?" — but holds all other inputs frozen. You compute it by treating every other variable as a constant and applying the usual rules. There are as many partial derivatives as there are inputs. Stacking them produces the gradient (next topic). The mixed second partials $\partial^2 f/\partial x \partial y$ measure how the slope along x changes when y moves — and Clairaut's theorem says these are equal to $\partial^2 f/\partial y \partial x$ for nice functions, the symmetry that gives the Hessian its symmetric structure.

💡 ***Intuition:*** *Standing on a curved mountain, the partial derivatives are like asking "what's the slope if I walk strictly east?" and "strictly north?" One per direction. The full picture comes from combining them as a vector — that's the gradient.*

? Why AI needs this

Networks have millions of weights. We need ∂L/∂w for every single one simultaneously — partial derivatives make this possible.

- param.grad = ∂loss/∂param for every parameter in the model
- GPT-3: 175B partial derivatives computed per backward pass in O(n) time
- Partial derivatives assign blame — how much did THIS weight contribute to the error?
- Mixed partials $\partial^2 L/\partial w_i \partial w_j$ = Hessian off-diagonal (weight interactions)

⚙ How AI uses it

loss.backward() computes all partial derivatives at once:

```
loss.backward()
for name, p in model.named_parameters():
    # p.grad = ∂loss/∂p (partial derivative)
    print(f"{name}: grad shape {p.grad.shape}")
```

🧮 Worked Example

Compute both partial derivatives of **$f(x, y) = x^2y + 3xy^2$** at the point (2, 1). Walk through the "treat other variables as constants" rule explicitly.

Partials of $f(x, y) = x^2y + 3xy^2$ at (2, 1)

GIVEN	$f(x, y) = x^2y + 3xy^2$, evaluate at (x, y) = (2, 1)
STEP 1	$\partial f/\partial x$: treat y as constant. $d/dx[x^2y] = 2xy$. $d/dx[3xy^2] = 3y^2$. So $\partial f/\partial x = 2xy + 3y^2$
STEP 2	$\partial f/\partial y$: treat x as constant. $d/dy[x^2y] = x^2$. $d/dy[3xy^2] = 6xy$. So $\partial f/\partial y = x^2 + 6xy$
STEP 3	At (2, 1): $\partial f/\partial x = 2\cdot2\cdot1 + 3\cdot1^2 = 4 + 3 = 7$
STEP 4	At (2, 1): $\partial f/\partial y = 2^2 + 6\cdot2\cdot1 = 4 + 12 = 16$
RESULT	$\nabla f(2, 1) = [7, 16]$ — the function rises faster in the y-direction here. Gradient descent would step proportionally further in y to reduce f.

07 The Gradient

All partial derivatives as one vector — always pointing uphill

🧭

Plain English: The gradient ∇f packs ALL partial derivatives into one vector that ALWAYS points toward steepest ascent. Like a compass pointing uphill. To minimize loss, gradient descent steps in the NEGATIVE gradient direction: w ← w − α∇L. That minus sign is the most important symbol in AI.

```
∇f = [∂f/∂x₁, ..., ∂f/∂xₙ] — always points uphill
Gradient descent: w ← w − α∇L
```

AI Deep Dive — The Gradient

📚 The Concept

The gradient **∇f** is the vector of all partial derivatives: $\nabla f = [\partial f/\partial x_1, \partial f/\partial x_2, \ldots, \partial f/\partial x_n]$. Two key properties make it the backbone of optimisation. **(1) Direction:** ∇f at a point always points in the direction of *steepest ascent* — the single direction in which f increases fastest. **(2) Magnitude:** ‖∇f‖ tells you how steep that ascent is. The negative gradient −∇f therefore points in the direction of steepest descent, which is exactly where you want to go to minimise a loss. The gradient is also *perpendicular to level curves* — it points across contour lines, not along them.

💡 ***Intuition:*** *Drop a marble on a curved surface. The gradient ∇f tells you the exact direction water would flow uphill. Water flows downhill (−∇f). That's gradient descent. The minus sign is the most consequential negative sign in machine learning.*

? Why AI needs this

∇L is the output of backpropagation. The update w←w−α∇L is the equation that trains every AI model.

- loss.backward() computes ∇L — gradient of loss w.r.t. every parameter
- GPT-3: 175B-dimensional gradient vector per backward pass
- |∇L| = gradient norm — indicates how far from convergence
- Gradient direction determines which weights to increase and which to decrease

⚙ How AI uses it

All optimizers start from the gradient ∇L:

```
loss.backward()  # computes ∇L
# Monitor gradient norm (convergence):
norm = sum(p.grad.norm()**2
           for p in model.parameters())**0.5
# norm ≈ 0 → converged
optimizer.step()  # applies w ← w − α∇L
```

🧮 Worked Example

Compute the gradient of an elliptical bowl loss **$L(x, y) = x^2 + 2y^2$** at the point (3, 1), then take one descent step.

Gradient and one descent step on $L = x^2 + 2y^2$

GIVEN	$L(x, y) = x^2 + 2y^2$, point (3, 1), learning rate $\alpha = 0.1$
STEP 1	$\partial L/\partial x = 2x$, $\partial L/\partial y = 4y$
STEP 2	$\nabla L(3, 1) = [2\cdot 3, 4\cdot 1] = [6, 4]$
STEP 3	$\|\nabla L\| = \sqrt{(36 + 16)} = \sqrt{52} \approx 7.21$ — steepness at this point
STEP 4	Descent step: $(x, y) \leftarrow (3, 1) - 0.1 \cdot [6, 4] = (3 - 0.6, 1 - 0.4) = (2.4, 0.6)$
RESULT	L dropped from 11 to 5.76 + 0.72 = 6.48 in one step. Note y moved more aggressively because of the $2y^2$ coefficient — that anisotropy is why Adam exists.

08 Chain Rule

The mathematical engine of backpropagation — layer by layer

⛓

Plain English: Gear A turns gear B which turns gear C. C's speed per A = (C/B) × (B/A). Neural networks are chains of functions. The chain rule computes the derivative of the entire chain by multiplying local derivatives at each layer. One backward pass, all gradients. This IS backpropagation.

```
d/dx[f(g(x))] = f′(g(x)) · g′(x)
Neural net: ∂L/∂W = ∂L/∂a × ∂a/∂z × ∂z/∂W
```

AI Deep Dive — Chain Rule

📚 The Concept

The chain rule is the rule for differentiating composed functions. If **y = f(g(x))**, then **dy/dx = f′(g(x)) · g′(x)** — the derivative of the outer function evaluated at the inner, times the derivative of the inner. With more layers it just keeps multiplying: y = f(g(h(x))) → dy/dx = f′(g(h)) · g′(h) · h′(x). That telescoping product is exactly what backpropagation is computing — one local derivative per layer, multiplied together right-to-left from the loss back to the input. The genius is that this cost is linear in the number of operations: one forward pass to record values, one backward pass to multiply.

💡 ***Intuition:*** *Three gears in a row. If gear A turns 2× faster than B, and B turns 3× faster than C, then A turns 6× faster than C. You multiplied two ratios. Backprop is doing this for billions of "gears" (operations) at once.*

? Why AI needs this

Backpropagation IS the chain rule on a computation graph. It computes all n gradients in O(n) cost — without it, you'd need n separate forward passes.

- One backward pass = one gradient for all n parameters (O(n))
- PyTorch/TensorFlow record every op and apply chain rule automatically
- Vanishing gradient = chain rule multiplying many values <1 (sigmoid: $0.25^{10} \approx 10^{-6}$)
- Exploding gradient = multiplying many values >1 (deep RNNs without clipping)

⚙ How AI uses it

PyTorch autograd applies the chain rule at every node automatically:

```
z = W @ x + b        # dz/dW = xᵀ
a = torch.relu(z)    # da/dz = 1(z>0)
loss = mse(a, y)     # dL/da = 2(a-y)/n
loss.backward()      # chain: dL/dW = dL/da·da/dz·dz/dW
```

🧮 Worked Example

One neuron: **y = σ(wx + b)**. Apply the chain rule to get dy/dw — the gradient that decides whether the weight grows or shrinks.

Chain rule on y = σ(wx + b)

GIVEN	w = 0.5, x = 2, b = 0, $\sigma(z) = 1/(1+e^{-z})$
STEP 1	Inner: z = wx + b = 0.5·2 + 0 = 1. Outer: y = σ(1) = 0.731
STEP 2	∂z/∂w = x = 2 (treat b, x as constants)
STEP 3	σ′(z) = σ(z)(1 − σ(z)) = 0.731 · 0.269 = 0.197
STEP 4	Chain: ∂y/∂w = σ′(z) · ∂z/∂w = 0.197 · 2 = 0.393
RESULT	∂y/∂w = 0.393. A 1-unit increase in w would raise y by ~0.39. Now imagine z = 5 instead: σ′(5) = 0.0066, gradient drops to 0.013 — vanishing gradient in action.

09 Jacobian & Hessian

Gradient for vector functions and the full curvature matrix

Plain English: The Jacobian is a matrix of all first-order partial derivatives for vector-valued functions (multiple outputs). The Hessian is a matrix of all second-order partial derivatives — it describes curvature in every direction simultaneously. Together they give optimizers the complete geometric picture of the loss surface.

```
Linear layer f(x)=Wx:
J = W (Jacobian = the weight matrix itself!)
Backward pass: ∂L/∂x = Wᵀ × ∂L/∂(Wx)
```

Eigenvalues	Point type	Training
All >0	Local minimum	Converged ✓
Mixed signs	Saddle point	May get stuck
All <0	Local maximum	Wrong direction

AI Deep Dive — Jacobian & Hessian

The Concept

The **Jacobian** generalises the gradient to vector-valued functions $f: \mathbb{R}^n \to \mathbb{R}^m$. It's an m×n matrix whose (i,j) entry is $\partial f_i/\partial x_j$ — every output's sensitivity to every input. For a scalar function $f: \mathbb{R}^n \to \mathbb{R}$, the Jacobian collapses to a single row, which is exactly the gradient. The **Hessian** is what you get when you take the Jacobian of the gradient: an n×n matrix of second partial derivatives $H_{ij} = \partial^2 f/\partial x_i \partial x_j$. It's symmetric (for nice functions), and its eigenvalues classify critical points: *all positive* = local minimum (positive definite), *all negative* = local maximum, *mixed signs* = saddle. The largest eigenvalue dictates the maximum stable learning rate: $\alpha_{max} = 2/\lambda_{max}$.

Intuition: *Jacobian = first-order story (slopes in every direction). Hessian = second-order story (how those slopes themselves bend). Knowing the Hessian is like knowing the shape of the bowl, not just which way is downhill.*

? Why AI needs this

The Jacobian of a linear layer equals W — so the backward pass is just multiplying by W^T. The Hessian determines the safe learning rate and is approximated by Adam.

- Jacobian of linear layer=W → backward = W^T multiplication (that's why backward is transpose)
- Max eigenvalue of Hessian → max safe learning rate: α < 2/λ_max
- Adam approximates diagonal of Hessian using squared gradients: v≈diag(H)
- K-FAC uses Kronecker-factored Hessian for more accurate second-order optimization

⚙ How AI uses it

Adam's v term is a diagonal Hessian approximation:

```
# Adam: v ≈ diagonal of Hessian
v = beta2*v + (1-beta2)*(grad**2)
# Optimal step ≈ -H⁻¹∇f ≈ -grad/sqrt(v)
param -= lr * m_hat / (sqrt(v_hat) + eps)
```

Worked Example

Use the Hessian eigenvalue test to classify the critical point at the origin for two different functions — one a bowl (convex), one a saddle.

Hessian eigenvalue test at the origin

BOWL	$f(x, y) = x^2 + y^2 \rightarrow \nabla f = (2x, 2y) = 0$ at origin
STEP 1	$\partial^2 f/\partial x^2 = 2$, $\partial^2 f/\partial y^2 = 2$, $\partial^2 f/\partial x \partial y = 0 \rightarrow H = [[2, 0], [0, 2]]$
STEP 2	Eigenvalues of H: $\lambda_1 = 2$, $\lambda_2 = 2$ — both positive → **local minimum** ✓
SADDLE	$g(x, y) = x^2 - y^2 \rightarrow \nabla g = (2x, -2y) = 0$ at origin
STEP 1	$\partial^2 g/\partial x^2 = 2$, $\partial^2 g/\partial y^2 = -2$, $\partial^2 g/\partial x \partial y = 0 \rightarrow H = [[2, 0], [0, -2]]$
STEP 2	Eigenvalues: $\lambda_1 = 2$, $\lambda_2 = -2$ — mixed signs → **saddle point**

RESULT Same gradient (zero), opposite geometry. The Hessian's eigenvalue signs are the verdict — and λ_max=2 also bounds the safe learning rate at $\alpha < 2/2 = 1$.

10 Antiderivatives

Accumulating infinitely small contributions over an interval

🏛

Plain English: If derivative=speed at each instant, the antiderivative=total distance. You're accumulating all the tiny contributions f(x)·dx. In ML: the true training objective is E[L] = ∫L(x)p(x)dx — the expected loss averaged over ALL possible inputs, weighted by their probability. This integral is the real goal; mini-batches approximate it.

```
∫xⁿ dx = xⁿ⁺¹/(n+1)+C
∫eˣ dx = eˣ+C
∫1/x dx = ln|x|+C
∫sin(x) dx = -cos(x)+C
```

```
True objective: E[L] = ∫L(f(x))p(x)dx
```

AI Deep Dive — Antiderivatives

📚 The Concept

An antiderivative of f is any function F whose derivative is f: $F'(x) = f(x)$. The notation **$\int f(x)\,dx = F(x) + C$** emphasises that there are infinitely many antiderivatives, all differing by a constant (since $(F+C)' = F'$). Where the derivative breaks change into instantaneous rates, the antiderivative reverses that — accumulating tiny contributions f(x)·dx back into a total. The *definite integral* $\int_a^b f(x)\,dx$ evaluates the antiderivative at the endpoints: $F(b) - F(a)$. Geometrically that's the signed area between the curve and the x-axis from a to b.

💡 ***Intuition:*** *Differentiation slices a quantity into instant-by-instant rates. Integration glues those rates back together to recover totals — speed integrated over time gives distance, density integrated over volume gives mass, log-likelihood integrated over data gives the training objective.*

? Why AI needs this

The true training objective is an integral: E[L]=∫L(x)p(x)dx. Every mini-batch gradient is a Monte Carlo approximation to this integral.

- True gradient = ∫(∂L/∂w)p(x)dx — integral over entire data distribution
- Mini-batch gradient = (1/n)Σ∂L/∂w ≈ Riemann sum approximation
- VAE ELBO = ∫q(z|x)log(p(x,z)/q(z|x))dz — integral over latent space
- Policy gradient RL: gradient = integral over trajectory distribution

⚙ How AI uses it

Mini-batch training is Monte Carlo integration of the true gradient:

```
# True gradient: ∫(∂L/∂w)p(x)dx  (integral)
# Approximated by random sampling:
for x, y in dataloader:  # sample from p(x)
    loss = model(x, y)
    loss.backward()   # gradient of batch avg
```

🧮 Worked Example

Compute the definite integral **$\int_0^4 (2x + 3)\,dx$** using the antiderivative — a calculation directly analogous to summing instantaneous contributions across a mini-batch.

Definite integral of 2x + 3 from 0 to 4

GIVEN $\int_0^4 (2x + 3)\,dx$

STEP 1 Find antiderivative term-by-term: $\int 2x\,dx = x^2$, $\int 3\,dx = 3x \rightarrow F(x) = x^2 + 3x$

CHECK	$F'(x) = 2x + 3$ ✓ matches integrand
STEP 2	Evaluate at endpoints: $F(4) = 16 + 12 = 28$, $F(0) = 0$
STEP 3	Definite integral $= F(4) - F(0) = 28 - 0 = 28$
RESULT	$\int_0^4 (2x + 3)\,dx = 28$ — exact area under the line. Approximating by sampling at points x=0,1,2,3,4 and averaging would give a Riemann-sum estimate ≈ 28; that's mini-batch training.

11 Riemann Sums

Rectangles converging to exact area — and the batch size analogy

Plain English: Approximate area under a curve with n rectangles, each of width $\Delta x = \text{range}/n$. $\text{Sum} = \Sigma f(x_i)\Delta x$. As $n \to \infty$, this converges to the exact integral. Mini-batch gradient descent IS this: each sample is one rectangle, batch_size=n controls approximation accuracy vs. computational cost.

AI Deep Dive — Riemann Sums

The Concept

A Riemann sum approximates the area under a curve by chopping the interval [a, b] into n equal slabs of width $\Delta x = (b-a)/n$, drawing a rectangle on each, and summing their areas. The choice of rectangle height — left endpoint, right endpoint, or midpoint of each slab — gives a slightly different sum, but as $n \to \infty$ they all converge to the same number: the definite integral. Formally $\int_a^b f(x)\,dx = \lim_{n\to\infty} \Sigma_i f(x_i)\,\Delta x$. The convergence is a quantitative statement: the midpoint rule has error that shrinks like $1/n^2$, so doubling rectangles cuts error by 4×.

Intuition: *A Riemann sum is averaging f at sample points and multiplying by interval length. A mini-batch gradient is averaging the per-example gradient at sampled examples and multiplying by 1. These are the same operation; batch size is just "how many rectangles."*

? Why AI needs this

Mini-batch training IS a Riemann sum approximation to the true gradient integral. Batch size = number of rectangles.

- True gradient $= \int(\partial L/\partial w)p(x)dx$ = integral over data distribution
- Mini-batch gradient $= (1/n)\Sigma\partial L/\partial w$ = Riemann sum with n sample points
- Larger n → more rectangles → more accurate → less noise → stabler training
- Small-batch noise can help escape saddle points — 'beneficial noise' of SGD

How AI uses it

Gradient accumulation simulates large batch with more 'rectangles' per update:

```
optimizer.zero_grad()
for i, (x, y) in enumerate(loader):
    loss = model(x, y) / accum_steps
    loss.backward()       # accumulate
    if (i+1) % accum_steps == 0:
        optimizer.step()  # update once
```

Worked Example

Approximate $\int_0^2 x^2\,dx$ (true value $8/3 \approx 2.667$) using the midpoint rule with n = 4 rectangles. Watch how the approximation tightens as n grows — exactly like enlarging the batch size.

Midpoint Riemann sum for $\int_0^2 x^2\,dx$, n = 4

GIVEN	$f(x) = x^2$, a = 0, b = 2, n = 4 → $\Delta x = 0.5$
STEP 1	Midpoints: x = 0.25, 0.75, 1.25, 1.75
STEP 2	Heights: f = 0.0625, 0.5625, 1.5625, 3.0625
STEP 3	Sum × Δx = (0.0625 + 0.5625 + 1.5625 + 3.0625) · 0.5 = 5.25 · 0.5 = 2.625
COMPARE	True value $8/3 \approx 2.6667$. Error ≈ 0.042 (1.6%)
RESULT	n=4 already within 2%; with n=32 (typical mini-batch) error ≈ 0.025%. This is exactly why batch_size=32 gives "good enough" gradients but n=1 (pure SGD) is wildly noisy.

12 Fundamental Theorem of Calculus

Differentiation and integration are inverses of each other

> ⚖️
>
> **Plain English:** FTC: ∫[a,b]f(x)dx = F(b)−F(a) where F′=f. You don't need millions of rectangles — just evaluate the antiderivative at the two endpoints. Also: differentiating an integral recovers the original function. In AI, Neural ODEs define continuous-depth networks using this theorem.

```
Part 1: d/dx[∫[a,x] f(t)dt] = f(x) — differentiating an integral recovers f
Part 2: ∫[a,b] f(x)dx = F(b)−F(a) — evaluate antiderivative at endpoints
```

AI Deep Dive — Fundamental Theorem

🧮 The Concept

The Fundamental Theorem of Calculus (FTC) is the bridge between the two pillars of calculus. **Part 1:** d/dx $[\int_a^x f(t)\,dt] = f(x)$. Differentiating an integral with a variable upper limit just gives back the integrand. **Part 2:** $\int_a^b f(x)\,dx = F(b) - F(a)$, where F is any antiderivative of f. The two parts together say: differentiation and integration are inverse operations. The reason this is so powerful is computational — without it, every integral would require taking a limit of Riemann sums; with it, you find any antiderivative and evaluate at two points. It's the same trick that lets compounding-then-discounting return the original quantity.

> 💡 ***Intuition:*** *If F is the "total" function (e.g., total distance travelled), then F′ is the "rate" function (e.g., speed). FTC says: integrating speed back over time recovers distance. You've spent your life using FTC informally — odometer = ∫ speed dt.*

? Why AI needs this

FTC underlies Neural ODEs — continuous-depth networks where depth is a continuous variable, not discrete layers.

- Neural ODEs: forward pass = integrate dh/dt=f(h,t,θ) from t=0 to t=1
- Adjoint method = FTC backward = O(1) memory gradient (vs O(depth) storing all activations)
- Continuous normalizing flows use FTC to compute log-determinant of Jacobian
- Diffusion model sampling integrates a reverse stochastic differential equation

⚙ How AI uses it

Neural ODEs use FTC to define infinite-depth continuous networks:

```
from torchdiffeq import odeint
def f(t, h): return net(h)  # the "layer"
# FTC: integrate from t=0 to t=1
h_out = odeint(f, h_init, torch.tensor([0., 1.]))
# Adjoint method: O(1) memory backprop via FTC
```

🧮 Worked Example

Verify both parts of FTC on a concrete pair: $f(x) = 3x^2$ and its antiderivative $F(x) = x^3$.

FTC verification on $f(x) = 3x^2$

GIVEN	$f(x) = 3x^2$, $F(x) = x^3$ (you can check $F' = 3x^2$)
PART 2	$\int_0^2 3x^2\,dx = F(2) - F(0) = 8 - 0 = 8$
CHECK	Riemann sum with n = 1000 midpoint rectangles also gives ≈ 8.000 ✓
PART 1	$G(x) = \int_0^x 3t^2\,dt = x^3$. Differentiate: $G'(x) = 3x^2$ — recovers the integrand exactly ✓
RESULT	Two operations, perfect inverses. This same identity is what lets a Neural ODE's adjoint method recover gradients of the input by integrating *backward* from t = 1 to t = 0 — saving O(depth) memory.

13 Critical Points

Where f′=0 — minimum, maximum, or saddle?

Plain English: At the top of a hill or bottom of a valley, the slope is exactly zero. That's a critical point. Gradient descent stops at any critical point — but zero gradient could mean minimum (good!), maximum (wrong direction!), or saddle point (gradient=0 but not a minimum). In deep networks with millions of parameters, most "flat spots" are saddle points.

```
f′(a)=0 AND f″(a)>0 → local minimum ✓
f′(a)=0 AND f″(a)<0 → local maximum ✗
f′(a)=0 AND f″(a)=0 → inconclusive
```

AI Deep Dive — Critical Points

The Concept

A critical point of f is any x* where **f′(x*) = 0** (or, in many variables, **∇f(x*) = 0**). At such a point f is locally flat — small steps in any direction barely change the value. Critical points come in three flavours: **local minimum** (curvature positive in every direction, f″ > 0 in 1D), **local maximum** (curvature negative everywhere), and **saddle** (positive in some directions, negative in others — only possible with ≥ 2 variables). The *second derivative test* uses curvature to classify the point. In high-dimensional ML loss surfaces, the proportions tilt heavily: actual local minima are rare, saddles are everywhere, and the global minimum is usually one of many near-equivalent low points.

Intuition: *Imagine a Pringle chip. The centre is a saddle: walking along one curve goes down, walking along the perpendicular curve goes up. Gradient is zero at the centre, but you're not at a minimum. That's why deep networks can have ∇L ≈ 0 yet still need to keep training.*

? Why AI needs this

Gradient descent converges to a critical point. In high-dimensional networks, most critical points are saddle points — not true minima.

- Gradient descent stops at any critical point — min, max, or saddle
- In n dimensions: true min requires f″>0 in ALL n directions simultaneously
- Most critical points in deep networks are saddles (mixed curvature eigenvalues)
- Good news: in high dimensions, most local minima ≈ global minimum in loss value

How AI uses it

Monitor gradient norm to detect convergence to a critical point:

```
norm = sum(p.grad.norm()**2
           for p in model.parameters())**0.5
if norm < 1e-6: print("At critical point!")
# SAM explicitly finds flat critical points:
# flat minima → better generalization
```

Worked Example

Find and classify all critical points of **f(x) = x³ − 3x**. This function has both a local max and a local min — a one-shot test of the procedure.

Critical points of f(x) = x³ − 3x

GIVEN	$f(x) = x^3 - 3x$
STEP 1	$f'(x) = 3x^2 - 3 = 3(x - 1)(x + 1)$
STEP 2	Solve $f' = 0$: $x = 1$ or $x = -1$ → two critical points
STEP 3	$f''(x) = 6x$
AT X=−1	$f''(-1) = -6 < 0$ → local maximum, value $f(-1) = -1 + 3 = 2$
AT X=1	$f''(1) = 6 > 0$ → local minimum, value $f(1) = 1 - 3 = -2$
RESULT	Two critical points: max at (−1, 2), min at (1, −2). A gradient descent run from x = 0.5 lands on the min; from x = −0.5 it climbs to the max — opposite of what we want. The starting point matters.

14 Gradient Descent

14 Gradient Descent

The algorithm that trains every neural network ever built

⛷

Plain English: Blindfolded on a mountain, trying to reach the lowest valley. You feel the slope (gradient), take one step downhill (subtract α×gradient), repeat. Learning rate α is the step size. Too large: overshoot the valley and diverge. Too small: take forever. This is the entire learning algorithm for every neural network.

```
w ← w − α × L′(w)
Safe convergence: α < 2/max|L″(x)|
```

AI Deep Dive — Gradient Descent

📚 The Concept

Gradient descent is the simplest optimisation algorithm that works at scale. Starting from any initial point, it repeats **$\mathbf{w \leftarrow w - \alpha \nabla L(w)}$**: compute the gradient, take a small step in the opposite direction, repeat until the gradient is tiny. The learning rate α controls step size. Convergence behaviour depends entirely on α and on the curvature: for a quadratic $L(w) = w^2$ with $\alpha < 1$, every step shrinks w by factor $(1 - 2\alpha)$ — geometric convergence. With $\alpha > 1$, every step *grows* w by factor $(2\alpha - 1)$ — divergence. The boundary $\alpha_{max} = 1/L_{smooth}$ (where L_{smooth} is the largest eigenvalue of the Hessian) is why curvature dictates safe learning rates.

💡 ***Intuition:*** *Gradient descent is a feedback loop: "where am I, which way is downhill, take a small step, repeat." The art of training neural networks is mostly about not screwing up that loop — picking α, scheduling it, smoothing it with momentum, escaping saddles, regularising the trajectory.*

? Why AI needs this

$w \leftarrow w - \alpha \nabla L$ trains everything from AlexNet to GPT-4. All optimizers (Adam, RMSprop, etc.) are gradient descent with modifications.

- Learning rate α is the most important hyperparameter — wrong value = no training
- Momentum: accumulate gradient direction → escapes shallow local minima faster
- Adam: per-parameter adaptive learning rates based on running gradient history
- Warmup: start with tiny α, prevents divergence at initialization

⚙ How AI uses it

All PyTorch optimizers are gradient descent variants:

```
optimizer = torch.optim.Adam(params, lr=1e-3)
# Warmup: prevents divergence (α too large initially)
scheduler = lr_scheduler.LinearLR(
    optimizer, start_factor=0.1, total_iters=1000)
# Cosine decay: shrink α as training progresses
scheduler = lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=100)
```

🧮 Worked Example

Run three steps of gradient descent on **$L(w) = w^2$** from $w_0 = 4$ with $\alpha = 0.1$, then show what happens if α is set too aggressively.

Three GD steps on $L(w) = w^2$, $\alpha = 0.1$ vs $\alpha = 1.1$

GIVEN	$L(w) = w^2$, $L'(w) = 2w$, $w_0 = 4 \rightarrow L_0 = 16$
STEP 1	$w_1 = 4 - 0.1 \cdot (2 \cdot 4) = 4 - 0.8 = 3.2 \rightarrow L = 10.24$ (down 36%)
STEP 2	$w_2 = 3.2 - 0.1 \cdot (2 \cdot 3.2) = 3.2 - 0.64 = 2.56 \rightarrow L = 6.55$
STEP 3	$w_3 = 2.56 - 0.1 \cdot (2 \cdot 2.56) = 2.56 - 0.512 = 2.048 \rightarrow L = 4.19$
PATTERN	Each step: $w_{t+1} = (1 - 2\alpha) \cdot w_t = 0.8 \cdot w_t \rightarrow$ geometric convergence to 0
A = 1.1	$w_1 = 4 - 1.1 \cdot 8 = 4 - 8.8 = -4.8$, $w_2 = -4.8 \cdot (1 - 2.2) = +5.76$, $\lvert w \rvert$ *grows*
RESULT	$\alpha = 0.1$ converges; $\alpha = 1.1$ diverges with oscillation. Safe range here is $\alpha < 1$, with optimal $\alpha^* = 0.5$ (one-shot convergence). Picking α is the difference between training and exploding.

15 Saddle Points

Gradient=0 but not a minimum — and why SGD's noise is a feature

🐎

Plain English: A horse saddle is lowest going left-right but highest going front-back. At a saddle point, gradient=0 but it's a minimum in some directions and maximum in others. In deep networks with millions of weights, most "critical points" are saddle points. Full-batch gradient descent gets stuck; SGD's mini-batch noise naturally perturbs past them.

AI Deep Dive — Saddle Points

📚 The Concept

A saddle point is a critical point ($\nabla f = 0$) that is neither a maximum nor a minimum. Geometrically: the surface curves up in some directions and down in others — like a horse saddle, or a Pringle. Saddle points only exist in two or more dimensions; in 1D, every critical point with $f'' \neq 0$ is a clean min or max. The Hessian's eigenvalues are the diagnostic: if the eigenvalues have *mixed signs*, the point is a saddle. The bigger surprise from modern ML theory: in very high dimensions, the probability that all n eigenvalues randomly happen to share the same sign is exponentially small. So in deep networks, almost every place where $\nabla L = 0$ is a saddle, not a true minimum. Vanishing gradient ≠ converged.

💡 ***Intuition:*** *A saddle is a "flat trap." Pure gradient descent reads $\nabla L = 0$ and stops. But the loss is still decreasing if you'd just look in the right direction. SGD's mini-batch noise is what actually nudges the optimiser off the saddle and back into productive descent.*

? Why AI needs this

Most critical points in deep networks are saddle points. SGD's mini-batch noise naturally helps escape them.

- In n dimensions: true minimum requires $f''>0$ in ALL n directions — very rare for large n
- Full-batch gradient descent gets stuck at saddles (gradient exactly 0)
- SGD noise from mini-batches breaks symmetry → natural saddle escape
- Flat minima generalize better than sharp minima (SAM, 2021)

⚙ How AI uses it

SAM explicitly seeks flat minima; SGD escapes saddles naturally via noise:

```
# SGD noise = mini-batch gradient ≠ exact 0
# This breaks the perfect saddle symmetry

# SAM: explicitly find flat minima
for x, y in loader:
    loss = model(x, y)
    loss.backward()
    optimizer.first_step(zero_grad=True)
    model(x, y).backward()
    optimizer.second_step(zero_grad=True)
```

🧮 Worked Example

The classic saddle $\mathbf{f(x, y) = x^2 - y^2}$. Verify the gradient is zero at the origin, the Hessian has mixed-sign eigenvalues, and trace what full-batch GD vs SGD do from a starting point right on the saddle.

$f(x, y) = x^2 - y^2$ at the origin

GIVEN	$f(x, y) = x^2 - y^2$, start at (0, 0.0001)
STEP 1	$\nabla f = (2x, -2y)$, at origin = (0, 0) → critical point
STEP 2	Hessian H = [[2, 0], [0, −2]] → eigenvalues +2, −2 → mixed signs → saddle
FULL-BATCH GD	$\nabla f(0, 0.0001) = (0, -0.0002)$, step away from saddle along y. After 100 steps with α=0.1: y goes from 0.0001 to ~12 (exponential blow-up *away* from origin) — escapes, but very slowly at first.
SGD	Mini-batch noise injects ±0.05 randomness in both x and y → gradient becomes (~0.05, ~−0.05), escape direction set in 1 step rather than 100.
RESULT	Saddle = ∇=0 + mixed-sign Hessian. SGD's noise is precisely what makes it a feature, not a bug, for deep training.

16 Convexity

Bowl-shaped = guaranteed convergence. Neural networks are NOT convex.

☕

Plain English: A convex function looks like a bowl — $f'' \geq 0$ everywhere, one minimum, always reachable. Logistic regression has a convex loss — it always converges to the global minimum. Neural networks are non-convex — many local minima — but in high dimensions, most local minima are just as good as the global one.

✅ **Why non-convex NNs still train:** In very high-dimensional parameter spaces, most local minima have similar loss values. You don't need the global minimum — any good local minimum generalizes well. (Choromanska et al., 2015)

Model	Loss convex?	Consequence
Linear regression	✓ Yes (MSE)	Always converges to global minimum
Logistic regression	✓ Yes (log-loss)	Unique global optimum guaranteed
SVM	✓ Yes (hinge)	Unique global solution
Neural network	✗ No	Many local minima & saddles
Deep learning	✗ No	Works anyway — local ≈ global in practice

AI Deep Dive — Convexity

📚 The Concept

A function f is **convex** if the line segment between any two points on its graph lies on or above the graph. Equivalently: $f(\lambda a + (1-\lambda)b) \leq \lambda f(a) + (1-\lambda)f(b)$ for all $\lambda \in [0,1]$ (Jensen's inequality). For twice-differentiable f, convexity is equivalent to $f'' \geq 0$ everywhere (1D), or the Hessian being positive semi-definite (multivariable). The defining magic property: any local minimum of a convex function is automatically the global minimum, and gradient descent is guaranteed to find it. Most classical ML losses (MSE for linear regression, log-loss for logistic regression, hinge loss for SVMs) are convex by design. Neural networks are emphatically *not* convex — but in practice they train anyway, because in very high dimensions the bad local minima are vanishingly rare.

💡 ***Intuition:*** *Convex = bowl-shaped, anywhere you stand, "downhill" leads to the unique bottom. Non-convex = mountainous landscape with many valleys, but in 100-million dimensions almost every valley is at the same elevation.*

? Why AI needs this

Convex loss + gradient descent = guaranteed global convergence. Non-convex (neural nets) = depends on initialization, but works in practice.

- Convex loss → gradient descent always finds global minimum (logistic regression, SVMs)
- Jensen's inequality ($f(E[X]) \leq E[f(X)]$ for convex f) underlies VAE's ELBO derivation
- Batch normalization smooths the loss landscape — reduces effective non-convexity
- He/Xavier init: start in a region of the loss landscape with good convergence properties

⚙ How AI uses it

Convex models always converge; deep nets use good initialization instead:

```
# Logistic regression: convex → always global min
model = nn.Linear(n_features, 1)
criterion = nn.BCEWithLogitsLoss()  # convex!
# Deep net: non-convex → init matters critically
# He init for ReLU (prevents gradient vanishing at init):
nn.init.kaiming_normal_(layer.weight)
# Xavier for sigmoid/tanh:
nn.init.xavier_uniform_(layer.weight)
```

🧮 Worked Example

Verify Jensen's inequality for $f(x) = x^2$ (convex) using two points $x = 1$ and $x = 3$ with mixing weight $\lambda = 0.5$.

Jensen's inequality on $f(x) = x^2$

GIVEN	$f(x) = x^2$, $a = 1$, $b = 3$, $\lambda = 0.5$
STEP 1	$f''(x) = 2 \geq 0$ everywhere → f is convex
LHS	$f(\lambda a + (1-\lambda)b) = f(0.5\cdot1 + 0.5\cdot3) = f(2) = 4$
RHS	$\lambda f(a) + (1-\lambda)f(b) = 0.5\cdot1 + 0.5\cdot9 = 0.5 + 4.5 = 5$
CHECK	$4 \leq 5$ ✓ Jensen holds — the chord lies above the curve at $x = 2$.
RESULT	For convex f, average-of-outputs ≥ output-of-average. This single inequality is what makes the VAE's evidence lower bound (ELBO) a valid lower bound on log-likelihood.

---

17 Exp & Log in ML

The two functions in every loss function and output layer

📐

Plain English: e^x is always positive (ideal for probabilities) and is its own derivative (makes gradients free). ln(x) converts products to sums — essential for multiplying millions of tiny probabilities without numerical underflow. Together they form the backbone of every classifier, language model, and generative AI system.

```
d/dx[eˣ] = eˣ — self-derivative, unique in mathematics
Softmax: σᵢ = exp(xᵢ)/Σexp(xⱼ)
Cross-entropy: L = −Σy·log(ŷ)
```

AI Deep Dive — Exp & Log in ML

📚 The Concept

The exponential $\mathbf{e^x}$ and natural log **ln(x)** are inverses: $\ln(e^x) = x$ and $e^{\ln x} = x$ for $x > 0$. Three properties make them omnipresent in ML. **(1) $e^x > 0$ always** — perfect for parameterising probabilities and variances that must stay positive. **(2) e^x is its own derivative**, the only nonzero function with that property — softmax gradients become trivially clean. **(3) ln turns products into sums:** $\ln(ab) = \ln a + \ln b$. Probabilities of independent events multiply, often producing numbers like 10^{-300} that no float can hold. Taking the log converts that catastrophe into a manageable sum. This is why every loss function involving probabilities (cross-entropy, log-likelihood, KL divergence, ELBO) lives in log-space.

💡 ***Intuition:*** *exp lifts numbers into the "positive multiplicative" world; log brings them down to the "additive" world. ML needs both: probabilities live in the multiplicative world, but optimizers do additive arithmetic.*

? Why AI needs this

exp and log appear in every loss function and output layer. exp ensures probabilities are positive; log prevents numerical underflow when multiplying many tiny probabilities.

- Softmax: $\exp(x_i)/\Sigma\exp(x_j)$ — exp ensures all outputs positive and sum to 1
- Cross-entropy: $-\Sigma y\cdot\log(\hat{y})$ — log makes penalty huge when $\hat{y}\approx0$ (very wrong prediction)
- Log-likelihood: $\log P = \Sigma\log P(x_i|\theta)$ — product of probs → sum (no underflow)
- KL divergence: $\int p\cdot\log(p/q)dx$ — used in VAEs, RLHF reward modeling

⚙ How AI uses it

Use log-softmax for numerical stability (avoids exp overflow):

```
# UNSTABLE: exp can overflow for large logits
probs = torch.exp(logits) / torch.exp(logits).sum()
# STABLE: log-softmax (subtracts max internally)
log_p = torch.log_softmax(logits, dim=-1)
# Most common (internally uses log-softmax):
loss = F.cross_entropy(logits, targets)
```

🧮 Worked Example

Take logits [2, 1, 0], turn them into a softmax probability distribution, then compute cross-entropy loss against the true class y = 0. This is the calculation every classifier ends with.

Softmax + cross-entropy on logits [2, 1, 0]

GIVEN	logits = [2, 1, 0], true class y = 0 (one-hot [1, 0, 0])
STEP 1	exp(logits) = [e^2, e^1, e^0] = [7.389, 2.718, 1.000]
STEP 2	Sum = 11.107 → softmax = [0.665, 0.245, 0.090] (sums to 1)
STEP 3	Cross-entropy: $L = -\Sigma\, y_i \cdot \log(\hat{y}_i) = -1 \cdot \log(0.665) = 0.408$
COMPARE	If predicted 0.99 for true class: $L = -\log(0.99) = 0.010$ (low loss). If predicted 0.01: $L = -\log(0.01) = 4.605$ (huge loss).
RESULT	Loss = 0.408 — model is confident-ish but not perfect. The −log curve punishes confident wrong predictions exponentially harder than diffuse wrong ones — the gradient signal that drives a classifier toward the correct answer.

---

18 Taylor Series

Any smooth function approximated by polynomials — Adam is 2nd order

🔬

Plain English: Any smooth function can be approximated near a point using only its derivatives there: $f(a)+f'(a)(x-a)+f''(a)(x-a)^2/2+\ldots$ More terms = better approximation, wider range. SGD uses 1st-order Taylor (linear). Adam uses 2nd-order (quadratic) — correcting for curvature to take better steps.

```
f(x) = Σ f⁽ⁿ⁾(a)/n! × (x−a)ⁿ
eˣ = 1+x+x²/2!+x³/3!+...
2nd-order: f(w+Δ) ≈ f(w)+∇f·Δ+½ΔᵀHΔ
```

AI Deep Dive — Taylor Series

📚 The Concept

The Taylor series of a function f around a point a is its representation as an infinite polynomial built entirely from f's derivatives at a: $\mathbf{f(x) = f(a) + f'(a)(x-a) + f''(a)(x-a)^2/2! + f'''(a)(x-a)^3/3! + \ldots}$. Truncating after k+1 terms gives a degree-k polynomial that matches f and its first k derivatives exactly at x = a, and approximates f well for x near a. This is how calculators evaluate sin, exp, and log internally. In optimisation it gives a clean way to ask "what's the best step from where I am?": expand the loss to second order around the current weights, minimise that quadratic approximation, take the result as the update. SGD uses only the first-order term; Newton's method uses the second-order; Adam uses an efficient diagonal approximation to second-order.

💡 ***Intuition:*** *Taylor series says: if I know everything about f's slope, curvature, and higher bends at one point, I can rebuild f near that point without ever looking at it again. Optimisers exploit this to plan their next step using only local information.*

? Why AI needs this

Adam is a 2nd-order Taylor approximation of the optimal weight update. SGD assumes 1st-order (linear). The difference is curvature correction.

- SGD: $f(w+\Delta) \approx f(w)+\nabla f \cdot \Delta$ (linear, 1st order) — ignores curvature
- Adam: $f(w+\Delta) \approx f(w)+\nabla f \cdot \Delta+\frac{1}{2}\Delta^T H \Delta$ (quadratic, 2nd order) — uses curvature
- Optimal Δ for 2nd order: $\Delta^* = -H^{-1}\nabla f$ (Newton step) — Adam approximates this
- Neural Tangent Kernel: infinite-width networks are linear in 1st-order Taylor sense

⚙ How AI uses it

Adam's update rule IS the 2nd-order Taylor optimal step with diagonal H approx:

```
m = beta1*m + (1-beta1)*grad    # ≈ ∇f (1st order)
v = beta2*v + (1-beta2)*grad**2  # ≈ diag(H) (2nd order)
# Optimal step = -H⁻¹∇f ≈ -m/sqrt(v):
param -= lr * m_hat / (torch.sqrt(v_hat) + eps)
```

🧮 Worked Example

Approximate **e^0.1** using the Taylor series of e^x around 0, term by term, and watch the error shrink — the same convergence ML uses when truncating optimiser approximations.

Taylor series of e^x at x = 0.1

GIVEN	$e^x = 1 + x + x^2/2! + x^3/3! + \ldots$ (every derivative of e^x at 0 is 1)
1 TERM	1 = 1.0000 (error 0.105 — 9.5%)
2 TERMS	1 + 0.1 = 1.1000 (error 0.0052 — 0.47%)
3 TERMS	1 + 0.1 + 0.01/2 = 1.1050 (error 0.00017 — 0.016%)
4 TERMS	+ 0.001/6 = 1.10516 (error 4.2×10^{-6} — 0.00038%)
TRUE VALUE	e^0.1 = 1.10517091...
RESULT	Each term cuts error roughly by another order of magnitude. SGD uses 2 terms (linear); Adam uses 3 (quadratic). Each "extra term" is exactly what the second-moment v in Adam is approximating.

---

19 Backpropagation

Chain rule on computation graphs — all gradients in one backward pass

Plain English: After a wrong prediction, backprop asks every weight: "how responsible were YOU for this error?" It walks backward from the loss, applies the chain rule at each layer, and accumulates each weight's gradient. One backward pass simultaneously computes the gradient for ALL parameters — same cost as the forward pass. This O(n) efficiency is what makes training feasible.

$$\partial L/\partial \hat{y} = 2(\hat{y}-y) \quad \partial L/\partial z = \partial L/\partial \hat{y} \times \sigma(z)(1-\sigma(z)) \quad \partial L/\partial w = \partial L/\partial z \times x$$

AI Deep Dive — Backpropagation

📚 The Concept

Backpropagation is the chain rule applied systematically to a computation graph. Forward pass: compute and cache the output of every node from inputs to loss. Backward pass: starting from $\partial L/\partial L = 1$ at the output, walk the graph in reverse and at each node multiply the upstream gradient by the local Jacobian (the node's own derivative w.r.t. its inputs). Because gradients flowing into a node are *summed* across all downstream paths, this single backward sweep produces $\partial L/\partial \theta$ for every parameter θ in the graph at the same total cost as one forward pass — O(n). Without this trick, computing one gradient per parameter would be $O(n^2)$ and training a billion-parameter model would be impossible. Reverse-mode automatic differentiation is the formal generalisation of backprop to arbitrary computation graphs and is what powers PyTorch's `autograd`, JAX's `grad`, and TensorFlow's `GradientTape`.

💡 ***Intuition:*** *Forward pass: light flows from inputs to loss. Backward pass: blame flows from loss back to every parameter. Each node passes the blame upstream after multiplying it by how much that node's output depends on its input. Backprop is just disciplined blame assignment.*

? Why AI needs this

One backward pass computes ALL n gradients at O(n) cost. Without the chain rule, you'd need n separate forward passes — $O(n^2)$ — completely infeasible for modern networks.

- Forward pass stores intermediate values; backward uses them for chain rule
- PyTorch autograd records every operation and differentiates automatically
- Gradient checkpointing: recompute activations instead of storing — O(1) memory
- Reverse-mode autodiff = backprop generalized to arbitrary computation graphs

⚙ How AI uses it

PyTorch applies the chain rule automatically at every node:

```
z = W @ x + b; a = torch.relu(z)
loss = F.mse_loss(a, y)
loss.backward()   # chain rule at every node
# W.grad = ∂loss/∂W computed automatically!
# Memory-efficient (gradient checkpointing):
from torch.utils.checkpoint import checkpoint
a = checkpoint(layer, x)  # recompute, don't store
```

🧮 Worked Example

Backprop end-to-end on the smallest possible network: $\hat{y} = w \cdot x$ with loss $L = (\hat{y} - y)^2$. Walk through forward, backward, and update — every step a number you can verify on paper.

Backprop on $\hat{y} = wx$, MSE loss

GIVEN	$x = 2$, $y = 5$ (target), $w = 3$ (current weight), $\alpha = 0.1$
FORWARD	$\hat{y} = w \cdot x = 3 \cdot 2 = 6 \rightarrow L = (\hat{y} - y)^2 = (6 - 5)^2 = 1$
BACKWARD 1	$\partial L/\partial \hat{y} = 2(\hat{y} - y) = 2 \cdot 1 = 2$
BACKWARD 2	$\partial \hat{y}/\partial w = x = 2$ (since $\hat{y} = w \cdot x$, holding x fixed)
CHAIN	$\partial L/\partial w = \partial L/\partial \hat{y} \cdot \partial \hat{y}/\partial w = 2 \cdot 2 = 4$
UPDATE	$w \leftarrow w - \alpha \cdot \partial L/\partial w = 3 - 0.1 \cdot 4 = 2.6$
RESULT	New $\hat{y} = 2.6 \cdot 2 = 5.2$, new $L = (5.2 - 5)^2 = 0.04$. Loss dropped 25× in one step. Scale this exact procedure to billions of parameters and you have GPT.

---

20 Loss Landscapes

The terrain gradient descent must navigate — shape determines everything

🏔

Plain English: Imagine height=loss value and the 2D plane represents two of the model's weights. Training is a ball rolling downhill. Convex landscape=easy, one valley. Narrow valley=zigzagging. Multiple valleys=different runs land at different minima. The shape of this landscape determines convergence speed, generalization, and what techniques you need.

AI Deep Dive — Loss Landscapes

📜 The Concept

The loss landscape is the high-dimensional graph of L plotted against every parameter of the model. For a network with *n* parameters it lives in n+1 dimensions, so we always view 2D or 3D *slices* through it. The **shape** of this surface — how curved, how anisotropic, how many basins — controls everything that matters in training: convergence speed, the safe learning rate, generalisation behaviour, and which optimiser to use. A *convex* landscape (single basin) is trivial; a *narrow valley* causes zigzagging because gradient descent steps perpendicular to the long axis; *saddles* cause stalling; *multi-modal* landscapes mean different initialisations find different minima. The Hessian's eigenvalues at any point characterise the local shape — and the *condition number* ($\lambda_{max}/\lambda_{min}$) predicts how aggressively the trajectory will zigzag.

💡 ***Intuition:*** *You're navigating a Himalayan-scale terrain by feel, one footstep at a time. The optimiser's job is to read local steepness and curvature and infer where to go next. The landscape's geometry, not the algorithm's cleverness, sets the speed limit.*

? Why AI needs this

The landscape shape determines convergence speed, learning rate safety, and generalization. Flat minima generalize better than sharp minima.

- Batch normalization smooths landscape — reduces max curvature → allows larger learning rates

- Residual connections prevent loss of smoothness in very deep networks
- Sharp minima overfit; flat minima generalize well (SAM, 2021)
- Loss landscape of transformers has many flat directions = attention head permutations

⚙ How AI uses it

Visualize and seek flat minima for better generalization:

```
# SAM: find flat minima (better generalization)
# SWA: average weights along trajectory → flat basin
swa_model = torch.optim.swa_utils.AveragedModel(model)
# Filter normalization visualization (Li et al. 2018):
dir1 = random_unit_perturbation(model)
# Plot loss over 2D slice of weight space
```

🧮 Worked Example

Quantify how landscape **anisotropy** (different curvature in different directions) destroys plain gradient descent. Compare a round bowl to a narrow valley — the same algorithm, the same α, wildly different convergence.

Round bowl vs narrow valley — same α, very different speed

ROUND	$L_1(x, y) = x^2 + y^2$, gradient descent with $\alpha = 0.1$, start (4, 4)
STEP RULE	Each axis: $w \leftarrow (1 - 2\alpha)\, w = 0.8\, w$ → identical decay in x and y
TRAJECTORY	L: 32 → 20.48 → 13.11 → 8.39 → … halves every ~3.1 steps. Path is straight to origin.
NARROW	$L_2(x, y) = 0.1x^2 + 5y^2$, same start (4, 4), same $\alpha = 0.1$
Y-AXIS	$y \leftarrow (1 - 2{\cdot}0.1{\cdot}5)\, y = 0 \cdot y$ → y collapses to 0 in **one step** (instantly converged on y)
X-AXIS	$x \leftarrow (1 - 2{\cdot}0.1{\cdot}0.1)\, x = 0.98\, x$ → x decays only 2% per step. ~35 steps to halve. ~230 steps to reach 0.01.
RESULT	Same α, same start: round bowl converges in ~10 steps; narrow valley needs ~230. The condition number $\kappa = 50$ is the slowdown factor. This is exactly why Adam's per-parameter learning rates exist — they normalise away anisotropy.

---

21 Cheat Sheet

Every formula from all 21 topics — one bookmark, complete reference

📐 Differentiation Rules

Derivative (definition)
`f′(x) = lim[h→0] (f(x+h)−f(x))/h`
Slope at a single point

Power rule
$d/dx[x^n] = n \cdot x^{n-1}$
Most used rule in ML

Chain rule
`d/dx[f(g(x))] = f′(g(x))·g′(x)`
Core of backpropagation

e^x derivative
$d/dx[e^x] = e^x$ `(self-derivative!)`
Softmax gradient free

ln(x) derivative
`d/dx[ln x] = 1/x`
Cross-entropy backprop

Sigmoid derivative
`σ′(x) = σ(x)(1−σ(x)) ≤ 0.25`
Max 0.25 → vanishing gradient

ReLU derivative
`0 if x<0, 1 if x>0`
Never shrinks → enabled deep learning

Product rule
`(fg)′ = f′g + fg′`
Attention backprop

🏔 Multivariable & Optimization

Gradient
$\nabla f = [\partial f/\partial x_1, \ldots, \partial f/\partial x_n]$
Steepest ascent vector

Gradient descent
$w \leftarrow w - \alpha \cdot \nabla L$
The training update rule

Safe learning rate
$\alpha < 2 / \max|L''(x)|$
Guarantees convergence

Critical point test
`f′=0 AND f″>0 → min ✓`
f″<0 → max ✗

Hessian
$H[i,j] = \partial^2 f/\partial x_i \partial x_j$
Curvature — Adam approx.

Taylor 2nd order
$f(w+\Delta) \approx f(w) + \nabla f \cdot \Delta + \frac{1}{2}\Delta^T H \Delta$
Basis of Adam optimizer

Newton's method
$x \leftarrow x - f'(x)/f''(x)$
2nd-order, fewer steps

Jacobian
$J[i,j] = \partial f_i/\partial x_j$
Linear layer backward = W^T

🤖 AI Applications

Cross-entropy loss
$L = -\Sigma\ y_i\ \log(\hat{y}_i)$
All classification tasks

Softmax
$\sigma_i = \exp(x_i)/\Sigma\exp(x_j)$
exp → positive probabilities

Log-likelihood
$\log P = \Sigma\ \log P(x_i|\theta)$
Product → sum, no underflow

Adam update
$w \leftarrow w - \alpha\cdot\hat{m}/(\sqrt{\hat{v}}+\varepsilon)$
Per-parameter adaptive lr

Backprop rule
$\partial L/\partial W = (\partial L/\partial z) \cdot x^T$
Chain rule at linear layer

Vanishing gradient
$0.25^{10} \approx 10^{-6}$ (sigmoid×10)
Fixed by ReLU (deriv=1)

Expected loss
$E[L] = \int L(x)p(x)dx$
Approximated by mini-batch

MSE loss
$L = (1/n)\Sigma(y_i-\hat{y}_i)^2$
MLE under Gaussian noise

PART II

Linear Algebra for AI

Vectors, matrices, eigenvalues, SVD — the data structures of deep learning

Print-friendly reading edition

01 Scalars

What is a scalar?

A scalar is a single real number. Denoted by lowercase letters: *a, b, λ, α*. It represents magnitude alone — no direction.

a = 3.14
b = -2.7
c = a × b = -8.478

Scalars in AI — everywhere

Loss value 0.342 — how wrong is the model?
Learning rate α 0.001 — step size in training
Temperature 0.8 — randomness in LLM output
Dropout 0.2 — fraction of neurons off
Attention score 3.7 — how much to focus here

1The Concept

A scalar is a **single real number** with magnitude only — no direction, no list, no structure. Mathematically, scalars live in ℝ (the real number line). They are the simplest object in linear algebra and serve as the "raw material" everything else is built from: a vector is a list of scalars, a matrix is a grid of scalars, a tensor is a higher-dimensional cube of scalars.

Scalars do two jobs in AI: they *measure* things (loss, accuracy, probability) and they *control* things (learning rate, temperature, dropout). Anywhere you see a single number governing behavior, you're looking at a scalar.

2Why It Matters in AI

Optimization needs a **single number to minimize**. You can't optimize "make the model better" — you have to compress the entire idea of "better" into one scalar (the loss). Backpropagation computes the gradient of this scalar with respect to every weight, and gradient descent uses the scalar learning rate α to scale each update step. Even a 175-billion-parameter model is ultimately driven by one scalar at the top.

3How It's Used — Step by Step

1. **Pick a loss function** that maps predictions to a scalar — e.g., MSE: `L = (1/n) Σ (ŷᵢ − yᵢ)²`.
2. **Run a forward pass** on a mini-batch. The model outputs predictions ŷ, and the loss collapses everything into one number.
3. **Compute the gradient** of that scalar with respect to every weight: `∇L = ∂L/∂w`.
4. **Update each weight** using the scalar learning rate: `w ← w − α × ∇L`.
5. **Repeat** for thousands of mini-batches. The scalar loss should trend downward over time.

4Worked Example

Suppose a model predicts house prices. For 3 houses the true prices are *y = [200, 300, 250]* (thousands $) and the model predicts *ŷ = [210, 290, 260]*.

Computing MSE Loss (a single scalar)

GIVEN	y = [200, 300, 250], ŷ = [210, 290, 260]
STEP 1	errors = ŷ − y = [10, −10, 10]
STEP 2	squared = [100, 100, 100]
STEP 3	sum = 300; mean = 300 / 3 = 100
RESULT	L = 100 # one scalar drives the entire training step
UPDATE	w ← w − 0.01 × ∇L # α=0.01 is also a scalar

02 Vectors

What is a vector?

An ordered list of n numbers. In 2D: [x, y]. In 768D: 768 numbers stacked together. Each number is a component.

v = [3, -1, 2]
$|v| = \sqrt{(3^2+1^2+2^2)} = \sqrt{14} \approx 3.74$

Real data as vectors

Data	Vector form
RGB pixel	[255, 128, 0]
User profile	[age, spend, clicks]
Word "cat"	[0.2, -0.5, ...] (768D)
28×28 image	784 pixel values

1The Concept

A vector is an **ordered list of n scalars**, written as $v = [v_1, v_2, \ldots, v_n]$. It lives in n-dimensional space $\mathbb{R}^n$. Two interpretations matter: (a) a vector is a **point** at coordinates $(v_1, \ldots, v_n)$, or (b) a vector is an **arrow** from the origin to that point with magnitude $|v| = \sqrt{(v_1^2+\ldots+v_n^2)}$ and direction. AI uses both interpretations interchangeably depending on context.

"Order matters." [3, 2] and [2, 3] are different vectors pointing in different directions. The position of each number encodes which feature it represents — slot 1 is always age, slot 2 is always income, slot 3 is always credit score, and so on.

2Why It Matters in AI

Computers work with numbers, not concepts. To process a sentence, photo, or user, AI must first **encode that thing as a vector**. Every input is converted into a fixed-length list of numbers — a 224×224 image becomes a 150,528-D pixel vector, a sentence becomes a sequence of 768-D embedding vectors, a user's profile becomes a feature vector. Once everything lives in vector form, the same mathematical machinery (dot products, distances, matrix multiplies) handles all data types uniformly.

3How It's Used — Step by Step

1. **Tokenize / featurize** the raw input. For text: split into words. For images: lay out pixel intensities. For users: pick measurable attributes (age, clicks, spend).
2. **Map to numbers.** Each token/feature becomes a scalar — either the raw value (pixel intensity 0–255) or a learned embedding (word "cat" → [0.21, −0.45, 0.13, ...]).
3. **Stack into a fixed-length vector.** Every example must end up in $\mathbb{R}^n$ for the same n, so the model's input layer has a known shape.
4. **Feed into the model.** The first layer multiplies this vector by a weight matrix: `z = Wx + b` — every neural network starts with vector inputs.

4Worked Example

Encode a user as a vector for a churn-prediction model. Features: *age, monthly spend ($), days since last login*.

Building & Comparing Two User Vectors

USER A	age=34, spend=$120, days_inactive=2 → u_A = [34, 120, 2]
USER B	age=29, spend=$15, days_inactive=45 → u_B = [29, 15, 45]
STEP 1	$\lvert u_A\rvert = \sqrt{(34^2 + 120^2 + 2^2)} = \sqrt{(1156+14400+4)} = \sqrt{15560} \approx 124.74$
STEP 2	u_A − u_B = [5, 105, −43] # difference vector
STEP 3	$\lvert u_A - u_B\rvert = \sqrt{(25+11025+1849)} \approx 113.6$ # distance
RESULT	Large distance → very different users → likely different churn risk

03 Vector Operations

Addition

$a + b = [a_1+b_1, a_2+b_2]$

$w \leftarrow w + (-\alpha\nabla L)$

Scalar multiply

$c \cdot v = [c \cdot v_1, c \cdot v_2]$

Normalize

$\hat{v} = v / |v|$

1The Concept

Three operations cover almost everything you do to vectors: **addition** (combine two vectors element-wise), **scalar multiplication** (stretch or shrink while keeping direction), and **normalization** (rescale to unit length). Together these define *linear combinations*: $c_1 \cdot v_1 + c_2 \cdot v_2 + \ldots + c_n \cdot v_n$. Any new vector you can possibly produce by mixing existing ones in this way lives in their **span**.

Linear combinations are the only "moves" allowed in linear algebra. Add. Scale. Combine. Everything else — projections, transformations, even neural layers — is built from these three primitives applied at scale.

2Why It Matters in AI

Every weight update is a vector addition: `w_new = w_old + Δw`. Every neuron output is a scaled sum of inputs: $z = w_1x_1 + w_2x_2 + \ldots + w_nx_n$ — that's a linear combination of the input components weighted by w. Residual connections in Transformers and ResNet are literal vector additions: `output = layer(x) + x`. Without that simple "+x", deep networks couldn't be trained because gradients would vanish through 100+ layers.

3How It's Used — Step by Step

1. **Compute the gradient** ∇L — itself a vector pointing uphill on the loss surface.
2. **Scale it** by the learning rate: `Δw = −α · ∇L` (negative because we want to go downhill).
3. **Add to current weights:** `w ← w + Δw`. This is pure vector addition.
4. **For normalization** (e.g. embeddings): compute `|v|`, then divide each component: `v̂ = v / |v|`. Now |v̂| = 1, direction preserved.

4Worked Example

The classic Word2Vec analogy: *king − man + woman ≈ queen*. Imagine the (toy) embeddings:

Word Arithmetic in Embedding Space

GIVEN	king = [0.7, 0.9, 0.2], man = [0.6, 0.1, 0.3]
GIVEN	woman = [0.5, 0.1, 0.7], queen = [0.6, 0.9, 0.6]
STEP 1	king − man = [0.7−0.6, 0.9−0.1, 0.2−0.3] = [0.1, 0.8, −0.1] # the "royalty" direction
STEP 2	+ woman = [0.1+0.5, 0.8+0.1, −0.1+0.7] = [0.6, 0.9, 0.6]
RESULT	[0.6, 0.9, 0.6] = queen ✓ # concept arithmetic, just by adding/subtracting vectors

04 Dot Product & Cosine Similarity

Formula

$a \cdot b = a_1b_1 + a_2b_2 + \ldots + a_nb_n$

$a \cdot b = |a| \times |b| \times \cos(\theta)$

$\text{sim}(a,b) = (a \cdot b) / (|a| \times |b|)$

What the angle means

θ angle	Meaning
0° (same dir)	Identical / maximum similarity
90° (perpendicular)	Unrelated / no similarity
180° (opposite)	Completely opposite

1The Concept

The dot product takes two vectors of the same length and returns **one scalar**: $a \cdot b = a_1b_1 + a_2b_2 + \ldots + a_nb_n$. Geometrically, this equals $|a| \cdot |b| \cdot \cos(\theta)$ where θ is the angle between them. So a dot product encodes *two* things at once — how big the vectors are, and how aligned they are. Divide by the magnitudes and you isolate alignment alone: that's **cosine similarity**, ranging from −1 (opposite) through 0 (perpendicular / unrelated) to +1 (identical direction).

If vectors encode meaning, then "similar meaning" = "small angle between vectors" = "high dot product." This single insight powers search engines, recommendation systems, and every Transformer ever built.

2Why It Matters in AI

The dot product is the universal **similarity measure**. A neuron's output is one giant dot product (input · weights) — it asks "how much does this input look like the pattern I've learned?" Transformer attention computes one dot product for every pair of tokens (Q · K) — it asks "which words should this word pay attention to?" Vector-database search returns the documents whose embedding has the highest dot product with your query. Modern GPUs are essentially specialized hardware for doing trillions of dot products per second.

3How It's Used — Step by Step

1. **Encode both items as vectors** in the same space and dimension (query and document, two words, two users).
2. **Multiply element-wise** and sum: $a \cdot b = \Sigma\ a_ib_i$.
3. **Normalize (optional but common):** divide by $|a| \cdot |b|$ to get cosine similarity. This removes magnitude effects so length doesn't bias results.
4. **Interpret:** the larger the score, the more aligned. For ranking, sort candidates by dot product descending and take the top-k.

4Worked Example

Semantic search: which document best matches the query "machine learning"?

Cosine Similarity for Semantic Search

QUERY	$q = [0.5, 0.4, 0.7]$ # "machine learning" embedding
DOC 1	$d_1 = [0.6, 0.3, 0.8]$ # "deep neural networks" article
DOC 2	$d_2 = [0.1, 0.9, 0.2]$ # "Italian recipes" article
STEP 1	$q \cdot d_1 = 0.5 \times 0.6 + 0.4 \times 0.3 + 0.7 \times 0.8 = 0.30 + 0.12 + 0.56 = 0.98$
STEP 2	$q \cdot d_2 = 0.5 \times 0.1 + 0.4 \times 0.9 + 0.7 \times 0.2 = 0.05 + 0.36 + 0.14 = 0.55$
STEP 3	$\|q\| = \sqrt{(0.25+0.16+0.49)} = \sqrt{0.90} \approx 0.949$
STEP 4	$\cos(q,d_1) = 0.98 / (0.949 \times \sqrt{1.09}) \approx 0.989$ # near-identical direction
STEP 5	$\cos(q,d_2) = 0.55 / (0.949 \times \sqrt{0.86}) \approx 0.625$
RESULT	Doc 1 wins (0.989 > 0.625) → returned as the top match

05 Norms & Distance

Types of norms

L1 norm: $||v||_1 = |v_1|+|v_2|+...+|v_n|$
L2 norm: $||v||_2 = \sqrt{(v_1^2+...+v_n^2)}$
L∞ norm: $||v||_\infty = \max(|v_i|)$

Distance metrics

Metric	Used in AI for
Euclidean $||a-b||_2$	k-NN, clustering, image similarity
Cosine $1-(a\cdot b/|a||b|)$	Text similarity, embeddings
Manhattan $||a-b||_1$	Sparse features, LASSO

1The Concept

A norm is any function that assigns a non-negative "size" to a vector and follows three rules: zero vector has zero norm, scaling a vector scales its norm proportionally, and the triangle inequality holds. Different norms emphasize different things. **L2 (Euclidean)** is the straight-line distance you learned in school. **L1 (Manhattan)** sums absolute values — like walking on a city grid where you can only go north/south/east/west. **L∞** takes the largest absolute component and ignores the rest.

Norms also define *distance*: the distance between two points a and b is just `||a − b||`. The choice of norm changes which points feel "close" — Manhattan thinks (3,0) and (0,3) are equally close to the origin, while Euclidean considers diagonal points closer.

2Why It Matters in AI

Norms appear in three places: (a) as **regularizers** added to the loss to prevent overfitting — L2 penalizes large weights smoothly, L1 pushes weights to exactly zero (creating sparse models); (b) as **distance metrics** in nearest-neighbor methods, clustering, and retrieval; (c) as **safety bounds** — gradient clipping rescales the gradient when its L2 norm exceeds a threshold, preventing exploding-gradient training failures.

3How It's Used — Step by Step

1. **For regularization:** compute $||w||^2 = \Sigma\ w_i^2$ across all model weights, multiply by hyperparameter λ, and add it to the loss: `L_total = L_task + λ·||w||²`.
2. **For gradient clipping:** compute `g_norm = ||∇L||₂`; if it exceeds a threshold T, rescale: `∇L ← (T / g_norm) · ∇L`. This caps each step's magnitude.
3. **For nearest-neighbor search:** for each candidate c, compute `||q − c||`, sort ascending, return the closest k.
4. **For L1 sparsity:** add `λ · Σ |wᵢ|` to the loss. The non-smooth shape of L1 at zero pushes many weights to exactly 0, yielding a sparse model.

4Worked Example

Compare norms for the vector *v = [3, −4, 0]* and apply gradient clipping to a gradient with the same shape.

Computing All Three Norms + Gradient Clipping

GIVEN	v = [3, −4, 0], gradient g = [3, −4, 0], clip threshold T = 2
L1	$||v||_1 = |3| + |-4| + |0| = 7$
L2	$||v||_2 = \sqrt{(9 + 16 + 0)} = \sqrt{25} = 5$
L∞	$||v||_\infty = \max(3, 4, 0) = 4$
CLIP	$||g||_2 = 5 > T=2$, so rescale: $g \leftarrow (2/5) \times g = [1.2, -1.6, 0]$
AFTER	$||g_clipped||_2 = \sqrt{(1.44+2.56+0)} = \sqrt{4} = 2$ ✓ (now safely bounded)

06 Linear Independence & Span

Definitions

Independent: no vector = combination of others
Dependent:

Span: all points reachable by combining vectors
Basis: minimal set spanning the full space
Standard 2D basis: $e_1=[1,0]$, $e_2=[0,1]$
Any 2D point = $a \cdot e_1 + b \cdot e_2$

Why it matters for AI

A full-rank weight matrix has independent rows — it can transform data into any direction without losing information. A rank-deficient matrix collapses some dimensions → information loss.

PCA finds truly independent directions in your data. Feature engineering tries to remove redundant (dependent) features before training.

1The Concept

A set of vectors $\{v_1, v_2, \ldots, v_k\}$ is **linearly independent** if the only way to write the zero vector as $c_1v_1 + c_2v_2 + \ldots + c_kv_k = 0$ is with all $c_i = 0$. If *any* non-trivial combination produces zero, at least one vector is "redundant" — expressible as a combination of the others. The **span** of a set is the collection of all reachable points using linear combinations. A **basis** is a minimal set whose span is the whole space — like the smallest possible set of LEGO pieces from which you can build everything.

Think of independence as "non-redundant information." Three independent vectors in 3D give you full freedom to reach any point. Three dependent vectors might all lie in a 2D plane — no matter how you combine them, you can't escape that plane.

2Why It Matters in AI

The **rank** of a weight matrix equals the number of independent rows (or columns). A rank-deficient matrix is "wasteful" — it has more parameters than information capacity. Researchers found that fine-tuning updates to large LLMs are surprisingly low-rank: a 4096×4096 matrix update can often be approximated by an outer product of two skinny matrices (4096×r) × (r×4096) with r≈8. This is the foundation of **LoRA**, which makes fine-tuning GPT-scale models tractable on a single GPU.

3How It's Used — Step by Step

1. **Stack vectors as rows of a matrix M.**
2. **Run row-reduction** (Gaussian elimination) to triangular form.
3. **Count non-zero rows.** That count is the rank — the number of truly independent directions.
4. **If rank < n,** some vectors are redundant. In LoRA, this insight inspires factoring $\Delta W \approx A \cdot B$ where A is d×r and B is r×d, with $r \ll d$.

4Worked Example

Are these three vectors independent? *$v_1 = [1, 2]$, $v_2 = [2, 4]$, $v_3 = [1, 0]$.*

Testing Linear Independence

GIVEN	$v_1 = [1, 2]$, $v_2 = [2, 4]$, $v_3 = [1, 0]$
STEP 1	Notice $v_2 = 2 \times v_1$ → v_1 and v_2 are dependent
STEP 2	v_3 is NOT a multiple of v_1 → v_1 and v_3 are independent
STEP 3	In 2D, the maximum number of independent vectors is 2
RESULT	Set has rank 2; $\{v_1, v_3\}$ forms a basis for $\mathbb{R}^2$; v_2 is redundant info
LORA	A 4096×4096 ΔW with rank 8 → store 4096×8 + 8×4096 = 65,536 nums instead of 16,777,216 — a 256× saving

07 Projection

Formula

proj_b(a) = (a·b / |b|²) × b

scalar projection = a·b / |b|

Geometric intuition

The projection of a onto b is the point on b's line that is closest to a. It's how much a "overlaps" with b. This is exactly what a neuron computes — how much does input x align with weight vector w?

1The Concept

Projecting *a* onto *b* means: of all the multiples of b, find the one that lies closest to a. The result is itself a vector along the b direction. The projection has two parts: a **scalar projection** (signed length: how far along b you go) and a **vector projection** (the actual vector pointing along b). The leftover piece `a − proj_b(a)` is perpendicular to b and is called the **residual** — it's everything in a that's NOT explained by b.

Projection is the simplest "explain a using b" operation. If b is a known direction, projection extracts how much of a lies along it. The residual is what's left unexplained — pure novelty.

2Why It Matters in AI

A single neuron is a projection: `z = w·x` measures how much input x aligns with the learned pattern w. Each filter in a CNN projects an image patch onto a learned feature direction. **Least-squares regression** is geometrically the projection of the target vector y onto the column space of the design matrix X. Even **self-supervised learning** often boils down to projecting representations and asking how much they agree.

3How It's Used — Step by Step

1. **Compute the dot product** of the two vectors: `a·b`.
2. **Compute |b|²** — the squared magnitude of the direction you're projecting onto.
3. **Divide** to get the scalar projection coefficient: `k = (a·b) / |b|²`.
4. **Multiply b by k** to get the projection vector: `proj_b(a) = k · b`.
5. **(Optional)** compute residual: `r = a − proj_b(a)`. This is what b cannot explain.

4Worked Example

Project *a = [3, 2]* onto *b = [4, 0]* (the x-axis direction).

Vector Projection in 2D

GIVEN	a = [3, 2], b = [4, 0]
STEP 1	a · b = 3×4 + 2×0 = 12
STEP 2	$\|b\|^2 = 4^2 + 0^2 = 16$
STEP 3	k = 12 / 16 = 0.75
STEP 4	proj_b(a) = 0.75 × [4, 0] = [3, 0]
STEP 5	residual = a − [3, 0] = [0, 2] # perpendicular to b ✓
MEANING	a has length 3 along b's direction, plus 2 of "new" perpendicular content

08 Matrices

Structure & notation

A ∈ ℝ^(m×n)
A[i][j] = element at row i, col j

Transpose: A^T flips rows↔cols
(A^T)[i][j] = A[j][i]
Shape: (m×n) → (n×m) after transpose

Special matrices in AI

Type	AI use
Identity I	Skip/residual connections
Weight W	Every neural layer
Q, K, V	Transformer attention
Embedding E	Word → vector lookup
Covariance C	PCA, batch norm

1The Concept

A matrix is a 2D grid of scalars, but the *useful* way to think of it is as a **function**: an m×n matrix W maps an n-dimensional input vector to an m-dimensional output vector via `y = Wx`. Each row of W defines one output dimension as a linear combination of the input components. The **shape (m × n)** tells you the function's signature: it eats n-dim inputs and produces m-dim outputs. Transpose W^T swaps rows and columns — the function flips direction (m-dim → n-dim).

Don't picture matrices as "tables of numbers." Picture them as boxes with arrows going in and out. $W: \mathbb{R}^n \to \mathbb{R}^m$. The numbers inside W are just the wiring details that determine how the inputs get blended into outputs.

2Why It Matters in AI

A neural network is fundamentally a stack of matrices interleaved with nonlinear activations: $x \to W_1 \to \sigma \to W_2 \to \sigma \to \dots \to \hat{y}$. The matrices are everything that's **learned** — billions of weights are nothing but the numbers inside W_1, W_2, etc. GPT's "knowledge" lives in these matrices. Training adjusts them via gradient descent. So mastering matrices = mastering the substrate of modern AI.

3How It's Used — Step by Step

1. **Define the shape** based on input/output dimensions. A layer mapping 768 → 3072 needs W of shape (3072 × 768).
2. **Initialize randomly** (e.g., Xavier or He initialization with carefully chosen scale).
3. **Forward pass:** for input $x \in \mathbb{R}^{768}$, compute $y = Wx + b \in \mathbb{R}^{3072}$. This applies the linear transformation.
4. **Backward pass** computes $\partial L/\partial W$, an m×n gradient matrix.
5. **Update:** $W \leftarrow W - \alpha \cdot \partial L/\partial W$ — the matrix is itself updated by another matrix.

4Worked Example

A tiny neural layer with 2 inputs and 3 outputs. Bias omitted for clarity.

A 3×2 Weight Matrix Acting on a 2-D Input

GIVEN	W = [[1, 2], [0, −1], [3, 1]], x = [4, 5]
ROW 1	$y_1 = 1\times4 + 2\times5 = 4 + 10 = 14$
ROW 2	$y_2 = 0\times4 + (-1)\times5 = 0 - 5 = -5$
ROW 3	$y_3 = 3\times4 + 1\times5 = 12 + 5 = 17$
RESULT	y = Wx = [14, −5, 17] ∈ $\mathbb{R}^3$ # 2D input has been "lifted" into 3D feature space
SHAPE	(3 × 2) · (2) = (3) ✓ inner dims match

09 Matrix Multiplication

The rule

$C = A \times B$

$C[i][j] = \Sigma_k A[i][k] \times B[k][j]$

$(m \times k) \times (k \times n) \rightarrow (m \times n)$

Neural forward pass

$z = W \cdot x + b$
$a = \sigma(z)$

$h_1 = \text{relu}(W_1 \cdot x + b_1)$
$h_2 = \text{relu}(W_2 \cdot h_1 + b_2)$
$\hat{y} = \text{softmax}(W_3 \cdot h_2 + b_3)$

1The Concept

To multiply *A (m×k)* by *B (k×n)*, the result *C (m×n)* has entry `C[i][j] = Σ A[i][p] · B[p][j]` over p = 1..k. Said differently: **C[i][j] = (row i of A) · (column j of B)**. The "inner" dimensions must match (k = k); the "outer" dimensions become the result's shape (m × n). Matrix multiplication is **not commutative** — generally `AB ≠ BA` — and it's **associative** — `(AB)C = A(BC)`, which lets us choose the cheapest order.

Think of A·B as composition of functions: first apply B (right-most acts first), then apply A. Two transformations stacked into one. Three layers of a neural network = one giant matrix product mathematically, though we don't fuse them in practice because of nonlinear activations between layers.

2Why It Matters in AI

Roughly **99% of GPU compute time** in training and inference is matrix multiplication. The forward pass of GPT is a chain of matmuls; backpropagation is matmuls in reverse. Modern AI hardware (TPUs, NVIDIA Tensor Cores) is purpose-built to do nothing but matmuls efficiently. Even attention boils down to two big matrix products: QK^T and `(softmax result)·V`. When researchers talk about "FLOPs" for a model, they're really counting matmul operations.

3How It's Used — Step by Step

1. **Verify shapes match:** A is (m×k), B must be (k×n). The shared k is the contraction dimension.
2. **For each output cell C[i][j]:** grab row i of A and column j of B.
3. **Take their dot product** — element-wise multiply, then sum.
4. **Place the scalar** into C[i][j]. Repeat for all m×n cells.
5. **Total cost:** m·n·k multiply-adds. A 4096×4096 by 4096×4096 product needs ~70 billion operations — done in milliseconds on a modern GPU.

4Worked Example

Multiply a 2×3 matrix by a 3×2 matrix to produce a 2×2 result.

2×3 · 3×2 → 2×2 Matrix Multiplication

A (2×3)	[[1, 2, 3], [4, 5, 6]]
B (3×2)	[[7, 8], [9, 10], [11, 12]]
C[0][0]	$\text{row}_0(A) \cdot \text{col}_0(B) = 1\times7 + 2\times9 + 3\times11 = 7 + 18 + 33 = 58$
C[0][1]	$\text{row}_0(A) \cdot \text{col}_1(B) = 1\times8 + 2\times10 + 3\times12 = 8 + 20 + 36 = 64$
C[1][0]	$\text{row}_1(A) \cdot \text{col}_0(B) = 4\times7 + 5\times9 + 6\times11 = 28 + 45 + 66 = 139$

C[1][0]	$row_1(A) \cdot col_0(B) = 4\times7 + 5\times9 + 6\times11 = 28 + 45 + 66 = 139$
C[1][1]	$row_1(A) \cdot col_1(B) = 4\times8 + 5\times10 + 6\times12 = 32 + 50 + 72 = 154$
RESULT	C = [[58, 64], [139, 154]] shape (2×2) ✓

10 Linear Transformations

1The Concept

A linear transformation is any function `T(x) = Wx` that satisfies two properties: `T(a+b) = T(a) + T(b)` and `T(c·a) = c·T(a)`. Geometrically that means: **the origin stays put**, **straight lines stay straight**, and **parallel lines stay parallel**. So a matrix W can rotate, scale, shear, or reflect — but it can't translate or curve. The columns of W tell you exactly where the basis vectors land: column 1 is the image of [1,0], column 2 is the image of [0,1].

Trick to read any 2×2 matrix W instantly: look at its two columns. Those are the new positions of the unit x-axis and unit y-axis. Everything else gets dragged along consistently.

2Why It Matters in AI

Each neural layer applies a linear transformation followed by a nonlinearity. The linear part *repositions* the data, the nonlinear part *bends* it. After many layers, classes that were tangled together in the input space become linearly separable in the final feature space. Visualizing layer outputs of a trained ResNet shows raw pixels gradually morphing into clean clusters — that morph is just hundreds of linear transformations stitched together.

3How It's Used — Step by Step

1. **Identify the desired behavior:** rotate by 30°, scale x by 2, shear, etc.
2. **Build the matrix** by writing where each basis vector should land. e_1`=[1,0]` goes in column 1; e_2`=[0,1]` goes in column 2.
3. **Apply** by multiplying: `x' = W·x`.
4. **Compose** multiple transformations by multiplying matrices: `C = B·A` means "first apply A, then B."
5. **Check det(W):** nonzero = invertible (info preserved); zero = collapse (info lost).

4Worked Example

Build a 90°-rotation matrix from scratch and apply it.

Constructing & Applying a 90° Rotation

GOAL	Rotate every point 90° counter-clockwise
STEP 1	Where does e_1=[1,0] go? Up to [0, 1] → column 1
STEP 2	Where does e_2=[0,1] go? Left to [−1, 0] → column 2
STEP 3	R = [[0, −1], [1, 0]]
APPLY	R · [3, 2] = [0×3 + (−1)×2, 1×3 + 0×2] = [−2, 3]
VERIFY	det(R) = 0×0 − (−1)×1 = 1 → preserves area, no flip ✓
RESULT	[3,2] rotated to [−2,3]; rotation preserves all lengths and shapes

11 Determinant & Inverse

Determinant

det([[a,b],[c,d]]) = ad − bc

det > 0: preserves orientation
det < 0: flips orientation
|det| = area scale factor
det = 0: COLLAPSES to lower dim!

Inverse

A^{-1}: undoes transformation A
$A \times A^{-1} = I$ (identity)

2×2 inverse:
$A^{-1} = (1/\det) \times [[d,-b],[-c,a]]$

Solving $Ax = b$: $x = A^{-1}b$

1The Concept

The **determinant** of a square matrix is a scalar that measures how the matrix scales volumes. In 2D, |det(W)| equals the area of the parallelogram formed by W's columns. Sign matters too: positive det preserves orientation, negative det flips it. **If det = 0**, the matrix collapses space into a lower dimension and *cannot be inverted*. The **inverse** A^{-1} "undoes" A: `A · A⁻¹ = I`. Only square, full-rank matrices have inverses.

Determinant = "stretchiness factor." det=2 doubles all areas. det=0.5 halves them. det=0 squashes everything flat — and you can never un-squash, just like you can't recover the third dimension after photographing a 3D scene.

2Why It Matters in AI

Three uses dominate. First, **checking invertibility**: solving `Ax = b` directly only works when det(A) ≠ 0. Second, **normalizing flows** (a class of generative models) require computing log|det(Jacobian)| at every step to track how probability mass changes. Third, the **singularity** in the loss landscape — places where the Hessian has det = 0 — corresponds to flat minima and saddle points where optimization gets stuck.

3How It's Used — Step by Step

1. **For a 2×2 matrix** A = [[a,b],[c,d]]: det = ad − bc. Quick rule: down-diagonal product minus up-diagonal product.
2. **If det ≠ 0,** the inverse is `A⁻¹ = (1/det) × [[d, −b], [−c, a]]` — swap diagonal, negate off-diagonal, divide by det.
3. **Verify:** compute $A \cdot A^{-1}$ and confirm it equals the identity I.
4. **Solve Ax = b:** apply A^{-1} to both sides → `x = A⁻¹ · b`. (In practice, libraries solve via LU decomposition for stability — but the idea is the same.)

4Worked Example

Solve a 2-equation system using the inverse: $2x + y = 5,\ \ x + 3y = 10$.

Computing det, inverse, and solving Ax = b

SYSTEM	A = [[2, 1], [1, 3]], b = [5, 10]
STEP 1	det(A) = 2×3 − 1×1 = 6 − 1 = 5 ≠ 0 ✓ invertible
STEP 2	A^{-1} = (1/5) × [[3, −1], [−1, 2]] = [[0.6, −0.2], [−0.2, 0.4]]
STEP 3	x = A^{-1} · b x_1 = 0.6×5 + (−0.2)×10 = 3 − 2 = 1 x_2 = (−0.2)×5 + 0.4×10 = −1 + 4 = 3
VERIFY	2(1) + 1(3) = 5 ✓ 1(1) + 3(3) = 10 ✓
RESULT	x = [1, 3] # unique solution because det ≠ 0

12 Solving Ax = b

Methods

$x = A^{-1}b$

[A | b] → row reduce → solution

$x = (A^TA)^{-1}A^Tb$
or solve: $\min ||Ax - b||_2^2$

Three possible outcomes

Unique solution — A is invertible (det≠0), exactly one x satisfies Ax=b. The system is perfectly determined.

No solution — overdetermined (more equations than unknowns). Use least squares — find x that gets as close as possible.

Infinite solutions — underdetermined (more unknowns than equations). Neural nets are always in this regime! Many weight combinations give zero training loss.

1The Concept

The system `Ax = b` asks: "is there a vector x such that A applied to x produces exactly b?" Three cases. **Square + full-rank A**: unique solution $x = A^{-1}b$. **Tall A (more equations than unknowns):** usually no exact solution exists, but we can find the x that minimizes $||Ax - b||^2$ — that's *least squares*. **Wide A (more unknowns than equations):** infinitely many solutions; we pick one with extra structure (e.g. minimum norm). Neural networks live in this third regime — vastly more parameters than training examples.

Geometrically, Ax travels through the column space of A — the set of all reachable outputs. If b is in that space, exact solution. If not, the best we can do is project b onto the column space and solve for that projection.

2Why It Matters in AI

Linear regression — the simplest ML model — is solved by the **normal equations**: $w = (X^TX)^{-1}X^Ty$. These come straight from least squares. Even when we use gradient descent for big problems, understanding the closed-form solution gives us the gold-standard answer to compare against. Beyond regression, every "physics-informed" or "constrained" optimization in AI ends up solving structured Ax = b systems many times.

3How It's Used — Step by Step (Least-Squares Linear Regression)

1. **Build the design matrix X** — each row is a training example, each column a feature (add a column of 1s for the intercept).
2. **Stack targets** y as a column vector.
3. **Compute** X^TX (small square matrix) and X^Ty (column vector).
4. **Solve** $(X^TX) \cdot w = X^Ty$ for w. In code: `w = np.linalg.solve(X.T @ X, X.T @ y)`.
5. **Predict** on a new x: $\hat{y} = w^Tx$.

4Worked Example

Fit a line $y = w_1 \cdot x + w_0$ through three points (1,2), (2,3), (4,5).

Linear Regression via Normal Equations

DATA	(1,2), (2,3), (4,5)
STEP 1	X = [[1,1], [1,2], [1,4]] y = [2, 3, 5] # col 0 = bias, col 1 = x
STEP 2	X^TX = [[3, 7], [7, 21]]
STEP 3	X^Ty = [10, 28]
STEP 4	det(X^TX) = 63 − 49 = 14

	$(X^TX)^{-1} = (1/14)\ [[21, -7],[-7, 3]]$
STEP 5	$w = (X^TX)^{-1} \cdot X^Ty = (1/14)\ [[21\times10 + (-7)\times28], [(-7)\times10 + 3\times28]]$ $= (1/14)\ [210 - 196, -70 + 84] = (1/14)[14, 14]$
RESULT	$w_0 = 1$, $w_1 = 1 \rightarrow$ line is $y = 1 + 1\cdot x = x + 1$ Check: x=1→2 ✓, x=2→3 ✓, x=4→5 ✓ (perfect fit on these 3 collinear points)

13 Rank & Null Space

Definitions

Rank(A) = #independent rows = #independent cols

Column space: all outputs A can produce
Null space: all x where Ax = 0

Rank + dim(Null space) = n columns

Rank in AI

Situation	Meaning
Full rank	Matrix uses all dimensions
Low rank	Compressible (LoRA!)
Rank 1	Outer product ($u \times v^T$)
Rank 0	Zero matrix

1The Concept

The **rank** of a matrix is the number of linearly independent rows (which always equals the number of independent columns). For an m×n matrix, rank ≤ min(m, n). The **column space** is the set of all outputs `Ax` can produce; its dimension equals the rank. The **null space** is the set of inputs that get sent to zero: `{x : Ax = 0}`. The famous **rank-nullity theorem** says: `rank(A) + dim(null(A)) = n`. Information either survives the matrix or gets destroyed — total dimensions accounted for.

Picture A as a funnel. Inputs in the null space drain out (lost forever). Whatever survives must lie in the column space. Rank tells you how wide the surviving channel is.

2Why It Matters in AI

Empirically, weight matrices in trained neural networks are often **much lower rank than their nominal size**. A 4096×4096 attention projection might effectively use only ~50 independent directions. This is the conceptual basis for LoRA (rank-r adapters), model distillation, and pruning. Conversely, encoder-decoder architectures intentionally create rank bottlenecks: the encoder squeezes rich data through a low-dimensional code, forcing the model to learn the most informative directions.

3How It's Used — Step by Step

1. **Run row reduction** on A to row-echelon form.
2. **Count non-zero rows** — that's the rank r.
3. **Find null space basis:** back-substitute the reduced form, treat free variables as parameters; the resulting vectors span null(A).
4. **Verify** rank-nullity: r + dim(null) should equal n.
5. **For low-rank approximation** in practice, use SVD: keep top-r singular values, zero out the rest, reconstruct.

4Worked Example

Find the rank and null space of *A = [[1, 2, 3], [2, 4, 6], [1, 1, 1]]*.

Computing Rank and Null Space

GIVEN	A = [[1, 2, 3], [2, 4, 6], [1, 1, 1]]
STEP 1	Row 2 − 2·Row 1 → [0, 0, 0] # row 2 was just 2× row 1!
STEP 2	Row 3 − Row 1 → [0, −1, −2]
RESULT	Echelon form: [[1,2,3], [0,−1,−2], [0,0,0]] → rank = 2
NULL	Solve Ax = 0: $x_3 = t$ (free); $x_2 = -2t$; $x_1 = -2x_2 - 3x_3 = 4t - 3t = t$ Null space basis: [1, −2, 1]
CHECK	rank + dim(null) = 2 + 1 = 3 = n ✓ (rank-nullity holds)

14 Eigenvectors & Eigenvalues

Definition

$Av = \lambda v$

Find eigenvalues: $\det(A - \lambda I) = 0$
Then for each λ: find v in $(A-\lambda I)v = 0$

What eigenvalues tell you

λ	Meaning
Large λ	High variance direction in data
λ = 0	Redundant direction (null space)
λ > 0 all	Positive definite (bowl-shaped loss)
Some λ < 0	Saddle point in loss surface!

1The Concept

An **eigenvector** of a square matrix A is a non-zero vector v that A only stretches: `Av = λv`. The scalar λ is its **eigenvalue**. Most vectors get rotated AND scaled when you apply A, but eigenvectors keep their direction — they're the matrix's "skeleton." Find them by solving `det(A − λI) = 0` for λ (the characteristic polynomial), then plug each λ back to find the corresponding v from `(A − λI)v = 0`. A symmetric n×n matrix always has n real, mutually perpendicular eigenvectors.

Eigenvectors are A's "preferred directions." Apply A a thousand times to a random vector — it will progressively align with the eigenvector having the largest |λ|. This is the secret of power iteration, PageRank, and the spectral methods that dominate scientific computing.

2Why It Matters in AI

Eigenvectors of a **covariance matrix** are exactly the principal components in PCA — the directions of maximum variance in data. Eigenvalues of the **loss Hessian** tell you whether you're at a minimum (all positive), saddle (mixed signs), or maximum (all negative); they also bound the safe learning rate as `α < 2 / λ_max`. Eigenvalues of an LLM's **weight matrices** reveal how unstable training is: if λ_max grows much larger than 1, the network amplifies noise exponentially through layers.

3How It's Used — Step by Step

1. **Set up:** compute `A − λI` (subtract λ from each diagonal entry).
2. **Take det(A − λI)** — it's a polynomial in λ called the characteristic polynomial.
3. **Set it to zero** and solve. The roots are the eigenvalues λ_1, λ_2, …
4. **For each λ_i:** solve `(A − λᵢI) v = 0` for v. Any non-zero solution is an eigenvector.
5. **(Optional) Normalize** each eigenvector to unit length for cleanliness.

4Worked Example

Find eigenvalues and eigenvectors of *A = [[4, 1], [2, 3]]*.

Eigenvalue Decomposition by Hand

GIVEN	A = [[4, 1], [2, 3]]
STEP 1	$A - \lambda I$ = [[4−λ, 1], [2, 3−λ]]
STEP 2	det = (4−λ)(3−λ) − (1)(2) = $\lambda^2 - 7\lambda + 12 - 2 = \lambda^2 - 7\lambda + 10$
STEP 3	$\lambda^2 - 7\lambda + 10 = 0 \rightarrow (\lambda-5)(\lambda-2) = 0 \rightarrow \lambda_1 = 5, \lambda_2 = 2$
Λ_1=5	$(A-5I)v = 0 \rightarrow [[-1, 1],[2, -2]]v=0 \rightarrow v_1 - v_2 = 0 \rightarrow v_1 = [1, 1]$
Λ_2=2	$(A-2I)v = 0 \rightarrow [[2, 1],[2, 1]]v=0 \rightarrow 2v_1 + v_2 = 0 \rightarrow v_2 = [1, -2]$
VERIFY	A·[1,1] = [4+1, 2+3] = [5, 5] = 5·[1,1] ✓
RESULT	eigenpairs: (5, [1, 1]) and (2, [1, −2])

15 PCA — Step by Step

1The Concept

Principal Component Analysis is "finding the best photographic angle" for high-dimensional data. PCA finds an orthogonal set of directions (the principal components) such that the first PC captures the most variance, the second captures the most remaining variance perpendicular to PC1, and so on. Mathematically, the principal components are the **eigenvectors of the data's covariance matrix**, ordered by their eigenvalues (which equal the variance along each PC).

PCA assumes that the directions where your data spreads the most are the directions that carry the most information. Often that's true (a long, thin point cloud) but not always — sometimes important signal lives in low-variance directions.

2Why It Matters in AI

PCA delivers **dimensionality reduction** for free: keep just the top-k principal components and you've compressed your data while losing minimal variance. This makes downstream models faster and less prone to overfitting. PCA is also the standard preprocessing for visualizing high-D embeddings (compress 768D → 2D for plotting), for noise reduction (low-variance directions are often noise), and as a baseline against which neural compression methods like autoencoders are compared.

3How It's Used — Step by Step

1. **Center the data:** subtract the mean of each column. Now the dataset has zero mean.
2. **Compute the covariance matrix:** `C = (1/(n−1)) · X^T · X`. This is d×d where d = number of features.
3. **Find eigenvectors and eigenvalues** of C. Sort by eigenvalue, descending.
4. **Pick top-k eigenvectors** as columns of a projection matrix $W \in \mathbb{R}^{d \times k}$.
5. **Project** data: `X_reduced = X_centered · W`. Each row is now a k-D representation.

4Worked Example

Reduce 4 points in 2D to 1D using PCA: (2,2), (3,3), (4,4), (5,5).

PCA on a Tiny Dataset

DATA	X = [[2,2], [3,3], [4,4], [5,5]] # perfectly correlated
STEP 1	Mean = [3.5, 3.5]; X_centered = [[−1.5,−1.5],[−0.5,−0.5],[0.5,0.5],[1.5,1.5]]
STEP 2	X^TX (sample) = [[5, 5],[5, 5]]; C = X^TX / (n−1) = (1/3)·[[5,5],[5,5]]
STEP 3	Eigenvalues of C: $\lambda_1 = 10/3 \approx 3.33$, $\lambda_2 = 0$ # 100% of variance on PC1!
STEP 4	Eigenvector for λ_1: $v_1 = [1/\sqrt{2}, 1/\sqrt{2}] \approx [0.707, 0.707]$
STEP 5	Project: X_centered · v_1 = [−2.121, −0.707, 0.707, 2.121]

RESULT 2D → 1D with zero info loss (variance explained = 100%)

16 SVD — Singular Value Decomposition

Formula

$A = U \Sigma V^T$

U (m×m): left singular vectors
Σ (m×n): diagonal, singular values $\sigma_1 \geq \sigma_2 \geq \ldots$
V (n×n): right singular vectors

$A_k \approx U[:,:k]\ \Sigma[:k,:k]\ V^T[:k,:]$

SVD vs PCA

Feature	PCA	SVD
Works on	Square covariance	Any matrix
Output	Eigenvectors	U, Σ, V^T
Stability	Can be unstable	Numerically stable
AI use	Visualization	Compression, LoRA

1The Concept

Every real matrix A (m×n, any shape) can be written as `A = U Σ Vᵀ` where U is m×m orthogonal, V is n×n orthogonal, and Σ is m×n with non-negative **singular values** $\sigma_1 \geq \sigma_2 \geq \ldots \geq 0$ along the diagonal. Geometrically: *any linear transformation = rotation, then axis-aligned stretch, then another rotation*. The singular values measure how much each direction is stretched. Number of non-zero singular values equals the rank.

SVD is PCA's more general cousin — PCA only works on covariance matrices (square, symmetric, positive-semidefinite), but SVD works on any matrix. In fact, PCA *is* SVD applied to the centered data matrix.

2Why It Matters in AI

SVD provides the **best rank-k approximation** of any matrix (Eckart–Young theorem): truncate Σ to its top-k singular values, and you have the closest possible rank-k matrix to A measured in Frobenius or spectral norm. This drives compression: **image compression** (keep top singular values, discard the small ones), **collaborative filtering** (Netflix-style recommendations factor the user-item matrix as $U \Sigma V^T$), and **LoRA** (re-parameterize ΔW as a rank-r product, motivated by the empirical low-rank-ness of fine-tuning updates).

3How It's Used — Step by Step

1. **Compute A^sA** (n×n) and find its eigenvectors $v_1, \ldots, v_n$ — these become V's columns.
2. **Eigenvalues of A^sA** are σ_i^2 — take square roots for singular values.
3. **Compute $u_i = (1/\sigma_i) \cdot A \cdot v_i$** for each non-zero σ_i — these become U's columns.
4. **Assemble:** $A = U \Sigma V^T$. Numerically use `np.linalg.svd(A)`.
5. **Truncate** to rank k by keeping the first k columns of U, the first k rows of V^T, and the top-k Σ entries: `A_k = U[:,:k] · Σ[:k,:k] · Vᵀ[:k,:]`.

4Worked Example

Approximate a 3×3 user-rating matrix using a rank-1 SVD.

Rank-1 Approximation via SVD

GIVEN A = [[5, 4, 0],
[4, 4, 1],
[0, 1, 5]] # 3 users × 3 movies

STEP 1 SVD computes $\sigma_1 \approx 9.04$, $\sigma_2 \approx 4.40$, $\sigma_3 \approx 1.06$

STEP 1	SVD computes $\sigma_1 \approx 9.64$, $\sigma_2 \approx 4.40$, $\sigma_3 \approx 1.00$
STEP 2	$u_1 \approx [0.62, 0.66, 0.42]$, $v_1 \approx [0.61, 0.65, 0.45]$
STEP 3	$A_1 = \sigma_1 \cdot u_1 \cdot v_1^T \approx$ rank-1 outer product
STEP 4	$A_1 \approx$ [[3.42, 3.65, 2.52], [3.64, 3.88, 2.69], [2.32, 2.47, 1.71]]
INSIGHT	A_1 captures the dominant pattern; missing entries can be filled in (e.g. user-1 movie-3 ≈ 2.52)

17 Tensors

Tensor dimensions in deep learning

Tensor	Typical shape	Represents
Image batch	(32, 3, 224, 224)	32 RGB photos
Text tokens	(8, 512)	8 sequences of 512 tokens
Embeddings	(8, 512, 768)	8 seq, each token 768D
Attention	(8, 12, 512, 512)	8 batch, 12 heads
Video	(4, 3, 30, 224, 224)	4 videos, 30 frames

Tensor operations

C = A + B, C = A * B

C = A @ B

x.reshape(32, -1)

x.permute(0, 2, 1)

1The Concept

A tensor is the natural extension of scalar (rank 0) → vector (rank 1) → matrix (rank 2) → 3D tensor → 4D tensor → … → arbitrary-rank array. The number of axes is the **rank**; the size along each axis is one of the **dimensions**. The shape `(32, 3, 224, 224)` means: rank-4, with 32 along axis 0, 3 along axis 1, 224 along axes 2 and 3. The total number of scalars stored is the product of the dimensions.

Tensors are bookkeeping. The numbers themselves are the same as in any array — what tensors give you is a clear convention for "which axis means what." Axis 0 = batch, axis 1 = channels, axes 2–3 = spatial — once a model fixes that convention, every operation is unambiguous.

2Why It Matters in AI

Real data is multi-axis. An image is (channels × height × width). A video adds a time axis. A batch of sequences is (batch × tokens × embedding). Multi-head attention adds a heads axis. Modern hardware (GPUs/TPUs) is built around **tensor cores** that perform contractions over chosen axes in parallel. Frameworks like PyTorch and JAX let you slice, reshape, and contract tensors with simple syntax — and that becomes the operational vocabulary of every deep-learning practitioner.

3How It's Used — Step by Step

1. **Plan the shape** for each tensor: batch size, channels, spatial/temporal dims, embedding size.
2. **Initialize** with `torch.randn(shape)`, `torch.zeros(shape)`, or load from data.
3. **Use broadcasting** for element-wise ops: when shapes differ, smaller tensors are virtually expanded along missing/size-1 axes.

4. **Contract with matmul:** `(A @ B)` contracts last axis of A with second-to-last of B; ideal for batched matrix multiplications.
5. **Reshape / permute** for I/O between layer types (flatten before a Linear layer, permute to swap heads and tokens before attention).

4Worked Example

Compute total numbers and bytes for one Transformer attention tensor.

Tensor Shape Math for Multi-Head Attention

SETUP	batch=8, heads=12, seq_len=512, d_k=64
Q	shape = (8, 12, 512, 64) → total = 8·12·512·64 = 3,145,728 numbers
K	same shape (8, 12, 512, 64)
QK^s	contract last axes → shape (8, 12, 512, 512) = 25,165,824 numbers
BYTES	25,165,824 × 4 (float32) ≈ 96 MB just for the attention scores tensor
WHY CARE	Quadratic in seq_len → at 4k tokens this becomes 6 GB → why FlashAttention & sparse attention exist

18 Gram-Schmidt & Orthogonality

1The Concept

Gram-Schmidt takes any set of linearly independent vectors and produces an **orthonormal basis** — vectors that are all unit length and mutually perpendicular. Process: keep the first vector, normalize it. For each next vector, **subtract its projections** onto every already-orthogonalized vector (so what remains is perpendicular to all of them), then normalize. The output: an orthonormal set spanning the same subspace as the input.

It's like building a coordinate system from scratch using whatever vectors you happen to have. The "subtract the projection" step removes overlap; the "normalize" step makes sure each axis is the same length.

2Why It Matters in AI

Orthogonal matrices are **numerically stable** — they don't amplify errors. **QR decomposition** (any matrix A = Q·R where Q is orthogonal) uses Gram-Schmidt under the hood and is preferred over normal-equations for solving least squares. Orthogonal initialization of weight matrices (specifically RNNs/Transformers) helps training stability. Multi-head attention works best when different heads learn (approximately) orthogonal features — orthogonalization regularizers explicitly enforce this.

3How It's Used — Step by Step

1. **Take v_1,** the first input vector. Normalize: $u_1 = v_1 / |v_1|$.
2. **For v_2:** compute its projection onto u_1: $p = (v_2 \cdot u_1)\ u_1$. Subtract: $w_2 = v_2 - p$. Normalize: $u_2 = w_2 / |w_2|$.
3. **For v_3:** subtract projections onto both u_1 and u_2: $w_3 = v_3 - (v_3 \cdot u_1)u_1 - (v_3 \cdot u_2)u_2$. Normalize.
4. **Continue** for any remaining vectors. Result: an orthonormal basis $\{u_1, u_2, \ldots\}$.

4Worked Example

Orthogonalize $v_1 = [1, 1, 0]$ and $v_2 = [1, 0, 1]$.

Gram-Schmidt on Two 3-D Vectors

GIVEN	$v_1 = [1, 1, 0]$, $v_2 = [1, 0, 1]$
STEP 1	$\lvert v_1\rvert = \sqrt{(1+1+0)} = \sqrt{2} \to u_1 = [1/\sqrt{2}, 1/\sqrt{2}, 0] \approx [0.707, 0.707, 0]$
STEP 2	$v_2 \cdot u_1 = 1\cdot0.707 + 0\cdot0.707 + 1\cdot0 = 0.707$
STEP 3	$\text{proj} = 0.707\cdot u_1 = [0.5, 0.5, 0]$
STEP 4	$w_2 = v_2 - \text{proj} = [1-0.5, 0-0.5, 1-0] = [0.5, -0.5, 1]$
STEP 5	$\lvert w_2\rvert = \sqrt{(0.25+0.25+1)} = \sqrt{1.5} \approx 1.225 \to u_2 \approx [0.408, -0.408, 0.816]$
VERIFY	$u_1 \cdot u_2 = 0.707\cdot0.408 + 0.707\cdot(-0.408) + 0 = 0$ ✓ orthogonal

VERIFY	$u_1 \cdot u_2 = 0.707 \cdot 0.408 + 0.707 \cdot (-0.408) + 0 = 0$ ✓ orthogonal
RESULT	$\{u_1, u_2\}$ forms orthonormal basis for the same 2-D plane as $\{v_1, v_2\}$

19 Gradient Vectors

What is a gradient?

$\nabla L(w) = [\partial L/\partial w_1, \partial L/\partial w_2, \ldots, \partial L/\partial w_n]$

To minimize: $w \leftarrow w - \alpha \nabla L$

Gradient field

At every point in weight space, the gradient points uphill. The loss surface is like a hilly landscape — the gradient at any location points toward the nearest peak (uphill). Training = following the gradient downhill to find the valley (minimum loss).

1The Concept

The gradient of a scalar function $L(w_1, w_2, \ldots, w_n)$ is the vector of all partial derivatives: `∇L = [∂L/∂w₁, ∂L/∂w₂, …, ∂L/∂wₙ]`. It points in the direction of **steepest ascent** — the direction along which L grows fastest. The magnitude $|\nabla L|$ tells you how steep the slope is. Critically, gradients are *local*: they only describe the immediate neighborhood, not far-away regions. The negative gradient, $-\nabla L$, is the direction of steepest descent — exactly what we need for minimization.

If the loss is a hilly landscape and you're standing at point w, the gradient is a tiny arrow at your feet pointing uphill. Walk against it = downhill. Take many small steps and (usually) you'll reach a valley.

2Why It Matters in AI

Training is just **repeated gradient descent** with the gradient telling each weight how to change. Without gradients, we'd have to try every possible weight combination — combinatorially impossible for billions of parameters. The chain rule (computed efficiently by backprop) gives us all the partials simultaneously in one backward pass, making training feasible. Modern variants like Adam, RMSProp, and SGD-momentum all use the gradient as input but adapt it (per-parameter scaling, momentum, decay) before applying.

3How It's Used — Step by Step

1. **Define a loss function** L(w) that measures error and is differentiable in every weight.
2. **For each weight w_i,** compute $\partial L/\partial w_i$ — the rate of change of L when w_i alone is nudged.
3. **Stack these partials** into the gradient vector ∇L.
4. **Move in the negative direction:** `w ← w − α · ∇L`. Loss decreases (locally).
5. **Stop conditions:** when $|\nabla L| \approx 0$ (flat region) or after a budget of steps.

4Worked Example

Compute the gradient of $L(x, y) = x^2 + 2y^2$ at the point (3, 1) and take one descent step.

Gradient and One Step of Gradient Descent

LOSS	$L(x, y) = x^2 + 2y^2$, point = (3, 1), $\alpha = 0.1$
∂L/∂X	$d/dx\ (x^2 + 2y^2) = 2x = 2 \cdot 3 = 6$
∂L/∂Y	$d/dy\ (x^2 + 2y^2) = 4y = 4 \cdot 1 = 4$
∇L	$\nabla L = [6, 4]$ # points uphill from (3,1)
|∇L|	$\|\nabla L\| = \sqrt{(36+16)} = \sqrt{52} \approx 7.21$ # steep!
STEP	new = $(3,1) - 0.1 \cdot [6,4] = (3 - 0.6, 1 - 0.4) = (2.4, 0.6)$
L OLD	$3^2 + 2 \cdot 1^2 = 9 + 2 = 11$
L NEW	$2.4^2 + 2 \cdot 0.6^2 = 5.76 + 0.72 = 6.48$ → loss dropped from 11 to 6.48 ✓

20 Gradient Descent

1The Concept

Gradient descent is the iterative rule `w ← w − α · ∇L(w)`. Each step nudges the weights opposite to the gradient direction. The **learning rate α** controls step size: too small → slow convergence; too large → overshoot or divergence. **Stochastic gradient descent (SGD)** uses one mini-batch's gradient instead of the full dataset's — noisy but vastly faster per step. Variants like **Momentum**, **RMSProp**, and **Adam** add running averages of past gradients to dampen oscillations and adapt α per-parameter.

A safe heuristic: maximum stable learning rate ≈ 2 / λ_max(Hessian). Below this, convergence; above, divergence. This is why "warmup" exists — at initialization the Hessian is poorly conditioned, so we ramp α up gradually.

2Why It Matters in AI

Every modern neural network is trained by gradient descent or one of its variants. Pretraining a 70B-parameter LLM is hundreds of thousands of gradient-descent steps on trillions of tokens. The choice of optimizer (SGD vs Adam vs AdamW), learning rate, schedule, and batch size are arguably *the* most consequential choices in any training run — get them wrong and the loss blows up or never decreases.

3How It's Used — Step by Step (One Training Step)

1. **Sample a mini-batch** of training examples.
2. **Forward pass:** compute predictions ŷ and loss L.
3. **Backward pass:** compute gradient ∇L w.r.t. every weight.
4. **(Optional) clip gradient** if its L2 norm exceeds a threshold.
5. **Update:** for plain SGD, `w ← w − α · ∇L`. For Adam, scale by running averages of first and second moments first.
6. **Repeat** for many epochs / steps. Track loss; check validation periodically.

4Worked Example

Run 3 steps of gradient descent on $L(w) = w^2$ starting at w = 4 with α = 0.3.

Three Iterations of Gradient Descent

SETUP	$L(w) = w^2$, $\nabla L = 2w$, $w_0 = 4$, $\alpha = 0.3$
ITER 1	$\nabla L(4) = 8$; $w_1 = 4 - 0.3 \cdot 8 = 4 - 2.4 = 1.6$; $L = 2.56$
ITER 2	$\nabla L(1.6) = 3.2$; $w_2 = 1.6 - 0.96 = 0.64$; $L = 0.41$
ITER 3	$\nabla L(0.64) = 1.28$; $w_3 = 0.64 - 0.384 = 0.256$; $L = 0.066$
RESULT	Loss dropped 16 → 2.56 → 0.41 → 0.066 — converging toward w=0 (the minimum)
TRY A=1.1	$w_1 = 4 - 1.1 \cdot 8 = -4.8$; $w_2 = +5.76$; diverges! → α exceeds 2/L″ = 1.0

21 Backpropagation

1The Concept

Backpropagation is just the **chain rule** applied to the computation graph of a model. For a composition `L = f(g(h(w)))`, the chain rule says `∂L/∂w = ∂L/∂f · ∂f/∂g · ∂g/∂h · ∂h/∂w`. The trick is doing this **efficiently** for billions of parameters: backprop walks the graph in reverse, multiplying gradients as it goes, reusing intermediate results so the cost is roughly the same as the forward pass — not exponential as it would be if computed naively.

Think of it as accountability. The loss is bad. Backprop walks back through the network asking each operation, "How much of this error is your fault?" — and assigns blame to its inputs and weights proportionally.

2Why It Matters in AI

Without backprop, training deep networks would be infeasible — the cost of computing each gradient one at a time

scales with parameter count. Backprop computes *all* gradients in a single backward pass at the same cost as one forward pass. Every framework (PyTorch, TensorFlow, JAX) implements it as **reverse-mode automatic differentiation**: record every operation in a graph, then walk backward applying transposed Jacobians.

3How It's Used — Step by Step

1. **Forward pass:** compute and cache every intermediate ($z_1 = W_1x$, $a_1 = \sigma(z_1)$, $z_2 = W_2a_1$, ...) until reaching the loss L.
2. **Initialize** the upstream gradient at the loss: `δ = ∂L/∂L = 1`.
3. **Walk backward.** At each operation, multiply δ by the local Jacobian to get the gradient w.r.t. that operation's inputs.
4. **For weights** at each linear layer: `∂L/∂W = δ · xᵀ` (outer product). Save this for the optimizer.
5. **For inputs** at each linear layer: `∂L/∂x = Wᵀ · δ`. Pass this further back as the new δ.
6. **Optimizer step** uses the accumulated weight gradients: `W ← W − α · ∂L/∂W`.

4Worked Example

Tiny 1-layer network: $\hat{y} = w \cdot x$, loss = $\frac{1}{2}(\hat{y} - y)^2$. Compute ∂L/∂w by hand.

Backprop Through One Linear Layer + Squared Loss

GIVEN	x = 2, y = 5, w = 3 → ŷ = w·x = 6
FORWARD	L = ½·(ŷ − y)² = ½·(6 − 5)² = ½·1 = 0.5
∂L/∂Ŷ	= ŷ − y = 6 − 5 = 1 # derivative of squared loss
∂Ŷ/∂W	= x = 2 # since ŷ = w·x
CHAIN	∂L/∂w = ∂L/∂ŷ · ∂ŷ/∂w = 1 · 2 = 2
UPDATE	w ← w − 0.1·2 = 3 − 0.2 = 2.8
VERIFY	new ŷ = 2.8·2 = 5.6; new L = ½·0.6² = 0.18 — dropped from 0.5 ✓

22 Attention Mechanism

1The Concept

Each token in the sequence produces three vectors via three learned projection matrices: a **Query Q**, a **Key K**, and a **Value V**. The token "asks" what it's looking for via Q. Every other token "advertises" what it has via K. The dot product Q·K measures how relevant each other token is to the asker. Normalize these scores with softmax → mixing weights. Use those weights to mix the V vectors → the new representation. Formula: `Attention(Q, K, V) = softmax(QKᵀ / √d) · V`.

Q = "what am I looking for?" · K = "what am I about?" · V = "what info do I provide?" The Q·K dot product is similarity; softmax turns it into a probability distribution over tokens; the weighted sum of V's is the output. Three matrix multiplies + a softmax = the most important operation in modern AI.

2Why It Matters in AI

Attention solved the long-standing problem of **long-range dependencies** in sequences — RNNs forgot, LSTMs were slow, but attention lets every token see every other token in O(1) hops. This unlocked GPT, BERT, and the entire LLM revolution. Vision Transformers apply the same operation to image patches. Diffusion models use cross-attention between image patches and text tokens. Even DNA-folding (AlphaFold) uses attention. The √d divisor keeps softmax in a sensitive regime; multi-head attention runs the operation in parallel with different Q/K/V projections to capture different relationships simultaneously.

3How It's Used — Step by Step

1. **Project** the input embedding matrix X into Q, K, V via three learned weight matrices: Q = X·W_Q, K = X·W_K, V = X·W_V.
2. **Compute scores** $S = Q \cdot K^T$ — a matrix where S[i][j] is "how much should token i pay attention to token j?"

3. **Scale** by 1/√d to keep softmax stable: S = S / √d_k.
4. **Softmax** over each row of S so weights sum to 1: A = softmax(S).
5. **Weight the values:** output = A · V. Each row of output is a weighted sum of V's, with the attention pattern of the corresponding query.
6. **(Multi-head)** repeat steps 1–5 in parallel with different W_Q/K/V's, then concatenate outputs and project.

4Worked Example

Three tokens with 2-D Q/K/V vectors. Compute attention output for token 1.

Self-Attention on a 3-Token Sequence

Q	Q = [[1, 0], [0, 1], [1, 1]] # query for each token
K	K = [[1, 0], [0, 1], [1, 1]] # keys
V	V = [[2, 0], [0, 2], [4, 4]] # values; d=2 → √d ≈ 1.414
STEP 1	$Q[0] \cdot K^T$ = [1·1+0·0, 1·0+0·1, 1·1+0·1] = [1, 0, 1]
STEP 2	scaled = [1, 0, 1] / 1.414 = [0.707, 0, 0.707]
STEP 3	exp = [2.028, 1, 2.028]; sum = 5.056
STEP 4	softmax weights ≈ [0.401, 0.198, 0.401]
STEP 5	output[0] = 0.401·[2,0] + 0.198·[0,2] + 0.401·[4,4] = [0.802, 0] + [0, 0.396] + [1.604, 1.604] = [2.406, 2.000]
MEANING	Token 1 attends mostly to itself and token 3 (which had a similar key); ignores token 2 less. Output is a context-aware blend.

23 Softmax

1The Concept

Softmax takes a vector of arbitrary real numbers (logits) and converts them into a probability distribution: every output is between 0 and 1, and they all sum to exactly 1. The formula is **$\text{softmax}(x)_i = \exp(x_i/T) / \Sigma_j \exp(x_j/T)$**. The exponential ensures positivity and amplifies differences — a logit that is just slightly higher than the others ends up with a disproportionately large share of the probability. The temperature T scales the logits before exponentiation: T < 1 sharpens (winner takes more), T > 1 flattens (more uniform).

💡 Think of softmax as a "soft" version of argmax. Argmax says "the biggest one wins everything"; softmax says "the biggest one wins most, but the others get a slice proportional to how close they were."

2Why It Matters in AI

Almost every neural network classifier ends with softmax — it converts raw scores into something a loss function (cross-entropy) can compare to a true label. Inside transformers, softmax is what turns attention scores into weights that sum to 1, so each token's new representation is a proper weighted average of others. Softmax is also fully differentiable, so gradients flow through it cleanly during backprop. Without softmax there is no probabilistic LLM output, no attention mechanism, and no standard classification head.

3How It's Used — Step by Step

1. **Get logits** from the final linear layer (one number per class or vocabulary token).
2. **Subtract the max** from every logit. This is a numerical-stability trick — it prevents exp() from overflowing on large positive numbers — and gives identical results because the constant cancels out in the ratio.
3. **Divide by temperature T** if you want to control sharpness (T=1 leaves it alone).
4. **Exponentiate** each shifted logit.
5. **Sum** all the exponentials.
6. **Divide** each exponential by that sum — those are your probabilities.
7. **Sample** from the distribution (LLM generation) or take argmax (classification).

4Worked Example

An LLM produces three logits for the next token: `[2, 1, 0]`. Let's see how temperature changes the distribution.

Softmax with T=1 vs T=0.5

GIVEN	logits x = [2, 1, 0]
STEP 1	Stability shift: subtract max=2 → [0, −1, −2]
T=1	exp([0, −1, −2]) = [1.000, 0.368, 0.135]
SUM	1.000 + 0.368 + 0.135 = 1.503
RESULT	softmax = [0.665, 0.245, 0.090] — moderately confident
T=0.5	scale logits by 1/T=2 → [4, 2, 0]; shift to [0, −2, −4]; exp = [1, 0.135, 0.018]
RESULT	softmax = [0.867, 0.117, 0.016] — much sharper, top option dominates

Same logits, very different distributions. At T=0.5 the model is "confident"; at T=1.5 it would be "creative". This single dial is what creativity sliders in chatbots actually control.

24 Convolution as Matrix Multiplication

1The Concept

A convolution slides a small grid of weights — the **kernel** (also called a filter) — across a larger input grid. At every position, it computes a dot product between the kernel and the patch of input it currently overlaps, and writes that single number into the output. The kernel is the same at every position: this is **parameter sharing**. A 3×3 kernel has only 9 weights, yet it produces a feature map covering the whole image. Different kernels detect different patterns: one might fire on vertical edges, another on horizontal edges, another on a specific colour blob.

💡 A convolution is a dot product, repeated. The kernel is a tiny "template," and at each position the output answers the question: *how much does this patch of the image look like the template?*

2Why It Matters in AI

Convolutions encode three powerful priors that match how images actually work: **locality** (nearby pixels are related), **translation invariance** (a cat is a cat whether it's in the top-left or bottom-right), and **parameter sharing** (one edge detector works everywhere). A fully connected layer on a 224×224 RGB image would need ~150K weights per neuron; a 3×3 conv needs 27. That parameter efficiency is why CNNs dominated computer vision for a decade and still power the vision side of multimodal models. Vision transformers eventually replaced them in many tasks, but they internally cut the image into patches — a discrete echo of the same locality prior.

3How It's Used — Step by Step

1. **Choose kernel size** (3×3 is most common) and the number of kernels in the layer (e.g. 64).
2. **Position the kernel** at the top-left of the input.
3. **Element-wise multiply** the kernel with the input patch underneath, then **sum**. Write the result in the output map.
4. **Slide** the kernel by the stride (usually 1) and repeat until the whole image is covered.
5. **Apply activation** (typically ReLU) to the output map.
6. **Stack multiple kernels** — each produces its own output map (channel). 64 kernels → 64 output maps.
7. **im2col trick:** in practice GPUs flatten every patch into a row of a big matrix and do one giant matrix multiply with the stacked kernels. Convolution is fundamentally a matmul in disguise.

4Worked Example

Let's compute one output cell of a Sobel-x kernel (vertical edge detector) over a patch with a clear vertical edge — dark on the left, bright on the right.

Sobel-x on a vertical edge patch

PATCH	[[1,1,9], [1,1,9], [1,1,9]] — left column dark, right column bright
KERNEL	[[−1,0,1], [−2,0,2], [−1,0,1]]
STEP 1	Element-wise product: [[−1,0,9], [−2,0,18], [−1,0,9]]
STEP 2	Sum row-by-row: 8 + 16 + 8 = 32

RESULT	Output cell = +32 — strong positive response, vertical edge detected
SANITY	On a flat patch [[5,5,5], [5,5,5], [5,5,5]]: every row gives −5+0+5 = 0. Total = 0 — no edge, no signal. ✓

A ResNet-50 performs roughly 4 billion such dot products per image. Every one is exactly this: place kernel, multiply, sum. The model learns *which* kernels to use, not how to apply them.

25 Vector Embeddings

1The Concept

An embedding is a dense vector — usually 128 to 4096 numbers — that represents the meaning of something (a word, a sentence, an image, a user, a product) as a point in a high-dimensional space. The geometry is the meaning: items with similar meaning sit close together, dissimilar items sit far apart. Each dimension doesn't have to mean anything specific by itself — meaning emerges from *relative* positions. That's why "king − man + woman ≈ queen" works in a well-trained word embedding space.

💡 An embedding is meaning-as-coordinates. Once your concepts live in vector space, "what's similar to X?" becomes "what vectors are nearest to X?" — a problem your laptop can solve millions of times per second.

2Why It Matters in AI

Embeddings are the universal interface of modern AI. Search, retrieval-augmented generation (RAG), recommendations, deduplication, clustering, anomaly detection, and multimodal models like CLIP all reduce to the same recipe: encode everything as vectors, then use distance or dot product to compare them. They turn fuzzy human concepts ("documents about climate policy", "users who liked this") into operations a GPU can do at scale: a single matrix multiplication can compare one query against millions of items in milliseconds.

3How It's Used — Step by Step

1. **Train an encoder** — Word2Vec, BERT, sentence-transformers, CLIP — to map inputs into vectors using contrastive or self-supervised objectives that pull similar pairs together and push dissimilar ones apart.
2. **Encode your corpus**: run every document/image/product through the encoder once and store the resulting vectors in a vector database (FAISS, Pinecone, pgvector).
3. **Encode the query** at runtime using the same encoder, producing one query vector.
4. **Compute similarity** between the query vector and every stored vector — typically cosine similarity (dot product of normalised vectors).
5. **Take the top-k** by similarity score. These are the most relevant items.
6. **Use the results**: as RAG context for an LLM, as recommendations, as search results, as nearest-neighbour classifications.

4Worked Example

Tiny 3-D embeddings to make the geometry visible. We'll measure how similar "cat" is to "kitten" versus "car" using cosine similarity.

Cosine similarity in toy embedding space

GIVEN	cat = [0.9, 0.1, 0.2], kitten = [0.85, 0.15, 0.18], car = [0.1, 0.8, 0.7]
STEP 1	Norms: ‖cat‖ = √(0.81+0.01+0.04) = 0.927, ‖kitten‖ = √0.7774 = 0.882, ‖car‖ = √1.14 = 1.068
STEP 2	cat · kitten = 0.9·0.85 + 0.1·0.15 + 0.2·0.18 = 0.765 + 0.015 + 0.036 = 0.816
STEP 3	cos(cat, kitten) = 0.816 / (0.927 · 0.882) = 0.816 / 0.818 ≈ 0.997
STEP 4	cat · car = 0.9·0.1 + 0.1·0.8 + 0.2·0.7 = 0.09 + 0.08 + 0.14 = 0.31
STEP 5	cos(cat, car) = 0.31 / (0.927 · 1.068) = 0.31 / 0.990 ≈ 0.313
RESULT	cat↔kitten ≈ 0.997 (nearly identical direction), cat↔car ≈ 0.313 (very different). Geometry has captured semantics.

A real RAG system does exactly this calculation, just with 1536-dimensional vectors and millions of documents. The encoder is what does the heavy lifting; once it's trained, retrieval is just a fast matrix multiplication followed by a top-k.

26 Complete Cheat Sheet

26 Complete Cheat Sheet

PART III

Probability & Statistics for AI

Distributions, Bayes, information theory, sampling, inference

Print-friendly reading edition

01 Sample Spaces & Events

The vocabulary of randomness — what an outcome is and how we measure its chance

🎲

Plain English: Roll a die. The set {1,2,3,4,5,6} of all things that could happen is the *sample space*. Any subset — like "even numbers" {2,4,6} — is an *event*. Probability assigns each event a number between 0 (impossible) and 1 (certain). In AI, the "die roll" is a model's prediction, a coin flip in dropout, or which token to sample next.

```
P(A) ∈ [0, 1]
P(Ω) = 1
P(A ∪ B) = P(A) + P(B) − P(A ∩ B)
```

AI Deep Dive — Sample Spaces & Events

🎲 The Concept

Three Kolmogorov axioms define probability. **(1)** $P(A) \geq 0$ for any event A. **(2)** $P(\Omega) = 1$ — the entire sample space has probability 1. **(3)** For disjoint events, $P(A \cup B) = P(A) + P(B)$. Everything else — conditional probability, Bayes, expectation, distributions — is downstream of these three. The sample space Ω is the set of all possible outcomes; an event is a subset of Ω; probability is a function from events to [0, 1] that respects the axioms.

💡 ***Intuition:*** *A sample space is the "menu of everything possible." An event is checking a few items on that menu. Probability is the area on the menu those items occupy, when the whole menu has area 1.*

? Why AI needs this

Every AI prediction is implicitly an event in some sample space. Defining that space precisely is half the modelling problem.

- Classifier output: sample space = the set of class labels {cat, dog, …}
- LLM next token: sample space = vocabulary (50K+ tokens)
- Dropout: each neuron's "keep/drop" is one Bernoulli sample space
- Reinforcement learning: states, actions, rewards each have their own sample space

⚙ How AI uses it

Categorical sampling lives in `torch.distributions`:

```
probs = torch.softmax(logits, dim=-1)
# Sample one outcome from the sample space:
dist = torch.distributions.Categorical(probs)
token = dist.sample()
# Or measure probability of an event:
p_top5 = probs.topk(5).values.sum()
```

🧮 Worked Example

Two fair dice. Sample space has 36 ordered outcomes (a, b) with a, b ∈ {1, …, 6}. Compute P(sum = 7) two ways: counting and a quick simulation.

P(sum = 7) on two fair dice

GIVEN	$\lvert\Omega\rvert = 36$ equally likely outcomes
STEP 1	Event A = {sum = 7} = {(1,6),(2,5),(3,4),(4,3),(5,2),(6,1)} → $\lvert A\rvert = 6$
STEP 2	$P(A) = \lvert A\rvert / \lvert\Omega\rvert = 6 / 36 = 1/6 \approx 0.1667$
SIM	Simulate 10,000 rolls → expect ≈ 1667 sevens; typical observed ~1655–1685 (±2σ)
RESULT	P(sum = 7) = 1/6. The "ridge" at 7 is why most board-game movement bell-curves around it — and why convolution of two uniforms is triangular.

02 Random Variables

A function from the sample space to numbers — discrete or continuous

Plain English: A random variable is just a rule that turns each outcome into a number. Flip 3 coins → X = number of heads ($X \in \{0, 1, 2, 3\}$, *discrete*). Measure a stranger's height → $X \in \mathbb{R}^+$ (*continuous*). Every quantity in ML — a logit, a loss value, a noise sample — is a random variable.

AI Deep Dive — Random Variables

The Concept

A random variable X is a function from the sample space Ω to ℝ. Discrete RVs take values in a countable set (integers, labels) and are described by a **probability mass function** $P(X = x)$. Continuous RVs take values in a continuum (real numbers) and are described by a **probability density function** $p(x)$, where probabilities are areas: $P(a \le X \le b) = \int_a^b p(x)\,dx$. Two RVs are *identically distributed* if they have the same PMF/PDF; they are *independent* if knowing one tells you nothing about the other.

Intuition: *"Random variable" sounds like a thing, but it's really a recipe — a function attached to a chance experiment that says "if outcome ω happens, the number is X(ω)." Whenever you write E[X] or Var(X), you're talking about that recipe.*

? Why AI needs this

Every quantity inside a model is a random variable — when training, when sampling, when measuring uncertainty.

- Weights initialised from $N(0, \sigma^2)$ → each weight is a continuous RV
- Predicted class is a discrete RV taking values in the label set
- Sampling temperature T scales logits before turning them into RVs
- Diffusion models add Gaussian RV noise at every timestep

How AI uses it

Both discrete and continuous RVs are first-class:

```
# Discrete RV: predicted class
dist = Categorical(logits=model(x))
y_hat = dist.sample()
# Continuous RV: weight init / latent code
z = torch.randn(batch, latent_dim)  # N(0, 1)
# Reparameterized RV in VAE:
z = mu + sigma * torch.randn_like(mu)
```

Worked Example

Let X = number of heads in 3 fair coin flips. List the PMF, compute E[X] and Var(X) directly from the definition.

PMF, mean, and variance of X = #heads in 3 flips

GIVEN	3 fair coin flips, X = number of heads, $\lvert\Omega\rvert = 8$
STEP 1	$P(X=0) = 1/8$, $P(X=1) = 3/8$, $P(X=2) = 3/8$, $P(X=3) = 1/8$ (sums to 1 ✓)
STEP 2	$E[X] = \Sigma\, x \cdot P(X=x) = 0\cdot 1/8 + 1\cdot 3/8 + 2\cdot 3/8 + 3\cdot 1/8 = 12/8 = 1.5$
STEP 3	$E[X^2] = 0 + 1\cdot 3/8 + 4\cdot 3/8 + 9\cdot 1/8 = 24/8 = 3$
STEP 4	$Var(X) = E[X^2] - E[X]^2 = 3 - 2.25 = 0.75$
RESULT	$X \sim Binomial(3, 0.5)$ → mean = np = 1.5, variance = np(1−p) = 0.75. Direct PMF and the binomial formula agree.

03 PMF, PDF & CDF

Three ways to fully describe one random variable

📊

Plain English: The PMF (discrete) or PDF (continuous) is the "shape" — how likely each value is. The CDF $F(x) = P(X \le x)$ tells you how much probability has accumulated by x. CDFs always go from 0 to 1, left to right. Sampling from a distribution is just inverse-CDF: draw $u \in [0, 1]$ uniformly, then $x = F^{-1}(u)$.

```
PMF: P(X = x); PDF: p(x), with P(X ∈ [a,b]) = ∫ₐᵇ p(x) dx
CDF: F(x) = P(X ≤ x); ∫-∞^∞ p(x) dx = 1
F'(x) = p(x)
```

AI Deep Dive — PMF, PDF & CDF

📚 The Concept

PMF, PDF, and CDF are three lenses on the same random variable. The **PMF** works for discrete RVs and gives an actual probability at each value. The **PDF** works for continuous RVs and is a *density* — p(x) by itself isn't a probability (can be >1!) but its integral over an interval is. The **CDF** $F(x) = P(X \le x)$ works for both, and it's the universal currency: the PDF is just F's derivative, and F^{-1} is what every random sampler is implicitly computing. CDFs are also how we compare distributions — Kolmogorov-Smirnov tests look at max $|F_1(x) - F_2(x)|$.

💡 ***Intuition:*** *PDF is "instantaneous likelihood per unit x" (think speed). CDF is accumulated probability up to x (think distance). They're related by the same calculus that links rates to totals.*

? Why AI needs this

Every probabilistic model is fundamentally a PDF or PMF. Training maximises log-PDF on the data; sampling inverts the CDF.

- Negative log-likelihood loss = $-\log p(x; \theta)$ summed over data → minimised in training
- Inverse-CDF sampling: $u \sim \text{Uniform} \rightarrow F^{-1}(u)$ gives a sample from the target
- Quantile regression predicts $F^{-1}(\tau)$ instead of the mean — robust to outliers
- Calibration: a well-calibrated classifier has F(predicted) ≈ uniform on test labels

⚙ How AI uses it

PyTorch `distributions` exposes log-PDF and CDF directly:

```
d = torch.distributions.Normal(mu, sigma)
log_p = d.log_prob(x)  # log PDF
F_x   = d.cdf(x)       # CDF
sample = d.sample()    # inverse-CDF under the hood
nll = -log_p.mean()    # NLL loss
```

🧮 Worked Example

For $X \sim N(0, 1)$: use the Normal CDF Φ to compute $P(X \le 1)$, $P(X > 2)$, and the 95% interval.

Standard Normal probabilities via Φ

GIVEN	$X \sim N(0, 1)$, CDF $\Phi(x) = P(X \le x)$
STEP 1	$P(X \le 1) = \Phi(1) \approx 0.8413$
STEP 2	$P(X > 2) = 1 - \Phi(2) = 1 - 0.9772 = 0.0228$ → only ~2.3% of mass beyond 2σ
STEP 3	95% central interval: $\Phi^{-1}(0.025) = -1.96$, $\Phi^{-1}(0.975) = +1.96$
RESULT	95% of standard-normal mass lies in $[-1.96, +1.96]$. This is the source of the "1.96" in every confidence interval and "2σ event" headline.

04 Expectation & Variance

The center of a distribution and how widely it spreads

⚖️

Plain English: Expectation E[X] is the long-run average if you sampled X infinitely many times — the balance point of the PDF. Variance Var(X) measures how far values typically stray from that balance point. Squared distance, on average. Standard deviation $\sigma = \sqrt{Var}$ puts that spread back into the original units. Training loss is an expectation; gradient descent is trying to minimise it.

```
E[X] = Σ x·P(x)  or  ∫ x·p(x) dx
Var(X) = E[(X − μ)²] = E[X²] − (E[X])²
E[aX + b] = a·E[X] + b
Var(aX + b) = a²·Var(X)
```

AI Deep Dive — Expectation & Variance

📚 The Concept

Expectation is a *linear operator*: E[aX + bY] = aE[X] + bE[Y] for any RVs X, Y and constants a, b — even if X and Y are dependent. That linearity is doing enormous work in ML: it's why averaging mini-batch gradients is unbiased, why Monte Carlo estimators are unbiased, why E[loss] over a dataset can be split into per-example terms. Variance, by contrast, is *non-linear*: Var(X + Y) = Var(X) + Var(Y) only when X, Y are independent. The standard deviation σ has the same units as X — that's what makes it interpretable. The Cauchy-Schwarz inequality and Chebyshev's inequality are both expectation-based bounds that drop out of these definitions.

💡 ***Intuition:*** *Expectation tells you where the center of mass sits; variance tells you how much the mass smears out. The bias-variance decomposition of generalisation error is exactly this split applied to a learning algorithm's predictions.*

? Why AI needs this

Loss functions are expectations; optimisers minimise them. Gradient noise is a variance; reducing it speeds convergence.

- Empirical risk minimisation: minimise $(1/n)\Sigma\, \ell(x_i; \theta) \approx E_x[\ell(X; \theta)]$
- Bias-variance tradeoff: test error = bias^2 + variance + irreducible noise
- Adam's v_t tracks the second moment $E[g^2]$ of gradients
- Reparameterization trick reduces variance of VAE/policy-gradient estimators

⚙ How AI uses it

E[X] and Var(X) appear directly in optimiser state and statistics:

```
# Mini-batch loss ≈ E_x[ℓ(X; θ)]
loss = ℓ(model(x), y).mean()
loss.backward()
# Adam tracks first moment (mean) and second moment
# m_t ≈ E[g_t], v_t ≈ E[g_t²]
# Variance reduction (control variates):
# E[f(X) - g(X)] + E[g(X)]  with chosen g
```

🧮 Worked Example

For Bernoulli(p): compute E[X] and Var(X). When is variance maximal?

Bernoulli mean and variance

GIVEN	$X \in \{0, 1\}$, $P(X=1) = p$, $P(X=0) = 1 - p$
STEP 1	$E[X] = 0\cdot(1-p) + 1\cdot p = p$
STEP 2	$E[X^2] = 0^2\cdot(1-p) + 1^2\cdot p = p$ (since $X^2 = X$ for Bernoulli)
STEP 3	$Var(X) = E[X^2] - E[X]^2 = p - p^2 = p(1 - p)$
MAX	$d/dp\ [p(1-p)] = 1 - 2p = 0 \rightarrow p = 0.5$, max variance = 0.25
RESULT	A fair coin (p = 0.5) is the most uncertain Bernoulli. This is also why dropout uses $p \approx 0.5$: maximum injected noise per neuron.

05 Conditional Probability & Independence

P(A | B) — the rule that learns from evidence

🔗

Plain English: P(A | B) — "probability of A *given* B" — narrows the sample space to just the part where B happened, then asks how much of that part is also A. If knowing B doesn't change A's probability (P(A|B) = P(A)), the events are *independent*. Conditioning is the engine of all probabilistic reasoning, including every Bayesian update.

🎲

Plain English: A *Bernoulli(p)* is a single biased coin: heads with probability p, tails with 1−p. A *Binomial(n, p)* is the count of heads in n independent flips. In AI: a single dropout decision is Bernoulli; the number of dropped neurons in a layer is Binomial. Every binary classifier outputs a Bernoulli distribution.

🎯

Plain English: A Categorical distribution puts probability on K mutually exclusive options — class 0, class 1, …, class K−1. Softmax is the function that takes K real-valued logits and turns them into a Categorical: positive, sums to 1, monotone in the logits. Every multi-class classifier ends with a softmax. So does every LLM, every Mixture-of-Experts router, every reinforcement-learning policy.

🔔

Plain English: The Normal distribution N(μ, σ²) has the iconic bell shape: symmetric around mean μ, spread σ. About 68% of mass is within μ±σ, 95% within μ±2σ, 99.7% within μ±3σ. The Central Limit Theorem says any large average tends toward Normal — which is why noise, errors, and aggregated quantities are so often modelled as Gaussian.

🎚

Plain English: Beta(α, β) is a distribution over a number p ∈ [0, 1] — useful when p itself is uncertain (the bias of a coin you haven't fully observed). Dirichlet generalises this to a vector that sums to 1 (a Categorical's parameters). They're the "conjugate priors" for Bernoulli and Categorical: posterior is the same family, just with updated counts.

⏱

Plain English: Exponential(λ) models the *time until* the next random event when events arrive at rate λ — server requests, photon hits, customer arrivals. Poisson(λ) models the *count* of those events in a fixed window. Memoryless: knowing you've already waited 5 min doesn't change how long you'll wait next.

🗺

Plain English: A joint distribution p(x, y) describes two RVs simultaneously — every pair of values gets a probability (or density). The *marginal* p(x) ignores y by summing/integrating it out. The *conditional* p(x | y) is the distribution of x once y is observed. In ML, every supervised learning problem is about the joint p(x, y) — and any model is some assumption on how it factorises.

🔄

Plain English: Bayes' theorem flips conditional probabilities. You know P(evidence | hypothesis) — the likelihood — and your prior belief P(hypothesis); Bayes gives you what you want, P(hypothesis | evidence) — the *posterior*. It's the engine of every spam filter, medical diagnostic, and Bayesian neural network.

🌡

Plain English: Entropy H(X) is the expected "surprise" of a random variable, measured in bits (log base 2) or nats (log base e). A fair coin has H = 1 bit; a biased near-deterministic coin has H ≈ 0. High entropy = lots of uncertainty = hard to predict. Low entropy = peaky distribution = easy to predict. Information theory's foundation, and the loss function for every classifier.

◿

Plain English: KL divergence $D_{KL}(P \parallel Q)$ measures how much P "differs" from Q in nats (or bits). It's not a true distance (asymmetric, no triangle inequality) but it's non-negative and zero iff P = Q. Cross-entropy $H(P, Q)$ is the expected NLL of true distribution P under predicted distribution Q. Identity: $H(P, Q) = H(P) + D_{KL}(P \parallel Q)$. Minimise cross-entropy ⇔ minimise KL.

🎯

Plain English: You have data, you pick a distribution family (e.g., Gaussian), and MLE asks: which specific parameters in that family make my data most likely? Take the joint likelihood $\prod p(x_i \mid \theta)$, take its log to turn product into sum, take the derivative w.r.t. θ, set to zero. Almost every supervised learning algorithm is MLE in disguise — including least-squares regression and softmax classifiers.

⚓

Plain English: Maximum a Posteriori (MAP) finds the θ that maximises the *posterior* $p(\theta \mid \text{data}) \propto p(\text{data} \mid \theta)\, p(\theta)$. It's MLE's regularised cousin: instead of letting the data alone choose θ, you tug toward your prior. Strong prior = small weights → L2 regularisation. Heavy-tailed prior → robustness. Sparse prior → L1.

🌊

Plain English: MLE and MAP collapse parameter uncertainty to one number. Full Bayesian inference keeps the whole posterior $p(\theta \mid \text{data})$ — and to make a prediction, integrates over all plausible θ weighted by their posterior probability. Result: predictions come with calibrated uncertainty for free. The catch: posteriors are usually intractable, so we sample (MCMC) or approximate (variational inference).

🎲

Plain English: Most posterior integrals are impossible analytically. Solution: replace $E[f(X)]$ with $(1/n)\, \Sigma\, f(x_i)$ using samples. Direct sampling works for nice distributions (uniform, Normal, Beta). For nastier targets, use rejection sampling, importance sampling, or Markov-chain Monte Carlo — each trades off setup cost against sample quality.

🌪

Plain English: Two miracles of probability. *LLN:* the sample mean of n i.i.d. RVs converges to the true mean as n grows. That's why training on more data works. *CLT:* the sample mean (suitably scaled) is approximately Normal — regardless of the shape of the underlying distribution. That's why Gaussians appear everywhere and why confidence intervals always look like ±1.96·SE.

⚖️

Plain English: Frame a null hypothesis H_0 ("nothing is happening") and an alternative H_1. Compute a test statistic from your data. Ask: assuming H_0 is true, how rare is a statistic at least this extreme? That probability is the *p-value*. Small p (typically < 0.05) → reject H_0 in favour of H_1. The math behind every A/B test, model comparison, and offline evaluation experiment.

```
P(A | B) = P(A ∩ B) / P(B)
A ⊥ B  ⇔  P(A ∩ B) = P(A)·P(B)  ⇔  P(A|B) = P(A)
Chain rule: P(X₁,…,Xₙ) = P(X₁)·P(X₂|X₁)·…
```

```
Bernoulli(p): P(X=1) = p,   E[X] = p,   Var(X) = p(1-p)
Binomial(n, p): P(X=k) = C(n,k) p^k (1-p)^(n-k),   E[X] = np,   Var(X) = np(1-p)
```

```
softmax(z)ᵢ = exp(zᵢ) / Σⱼ exp(zⱼ)
With temperature T: softmax(z / T)ᵢ
argmax(z) = argmax(softmax(z))
```

```
N(μ, σ²):   p(x) = (1/√(2πσ²)) · exp(-(x-μ)² / (2σ²))
Standard normal: μ = 0, σ = 1;   Z = (X - μ)/σ
Sum: X₁ ~ N(μ₁, σ₁²), X₂ ~ N(μ₂, σ₂²) independent → X₁+X₂ ~ N(μ₁+μ₂, σ₁²+σ₂²)
```

```
Beta(α, β): p(x) ∝ x^(α−1) (1−x)^(β−1),   x ∈ [0,1]
Mean = α/(α+β),   Mode = (α−1)/(α+β−2) for α,β > 1
Conjugacy: Beta(α, β) + observe k successes / n−k failures → Beta(α+k, β+n−k)
```

```
Poisson(λ): P(X=k) = e^(−λ) λ^k / k!,   E[X] = Var(X) = λ
Exponential(λ): p(t) = λ e^(−λt),   E[T] = 1/λ,   Var(T) = 1/λ²
Connection: if events arrive Poisson(λ) per unit time, gaps between events are Exp(λ).
```

```
Joint: p(x, y); Marginal: p(x) = ∫ p(x, y) dy; Conditional: p(x | y) = p(x, y) / p(y)
X ⊥ Y  ⇔  p(x, y) = p(x) p(y) for all x, y
Bivariate Normal: X | Y = y  ~  N(μ_x + ρ(σ_x/σ_y)(y − μ_y),   (1−ρ²) σ_x²)
```

```
P(H | E) = P(E | H) · P(H) / P(E)
posterior ∝ likelihood × prior
Gaussian conjugate update: μ_post = (μ_0/σ_0² + x/σ_L²) / (1/σ_0² + 1/σ_L²)
```

```
H(X) = −Σ p(x) log p(x)
0 ≤ H(X) ≤ log K
Joint: H(X, Y) = H(X) + H(Y | X); Mutual info: I(X; Y) = H(X) − H(X | Y)
```

```
D_KL(P ‖ Q) = Σ P(x) log(P(x) / Q(x)) ≥ 0
H(P, Q) = −Σ P(x) log Q(x)
H(P, Q) = H(P) + D_KL(P ‖ Q)
```

```
θ_MLE = argmax_θ ∏ᵢ p(xᵢ | θ) = argmax_θ Σᵢ log p(xᵢ | θ)
For X ~ N(μ, σ²):   μ̂ = (1/n) Σ xᵢ,   σ̂² = (1/n) Σ (xᵢ − μ̂)²
NLL(θ) = − Σᵢ log p(xᵢ | θ)
```

```
θ_MAP = argmax_θ p(θ | data) = argmax_θ [ log p(data | θ) + log p(θ) ]
L2 regulariser ⇔ Gaussian prior on weights: log p(w) = −λ‖w‖² / 2 + const
L1 regulariser ⇔ Laplace prior on weights → sparsity
```

```
posterior: p(θ | D) = p(D | θ) p(θ) / p(D); p(D) = ∫ p(D | θ) p(θ) dθ
posterior predictive: p(x_new | D) = ∫ p(x_new | θ) p(θ | D) dθ
Beta-Binomial conjugacy: posterior ∝ θ^(α+h−1) (1−θ)^(β+n−h−1) → Beta(α+h, β+n−h)
```

```
Monte Carlo: E[f(X)] ≈ (1/n) Σ f(xᵢ),   xᵢ ~ p
Importance: E_p[f(X)] = E_q[f(X) p(X)/q(X)]
Rejection: accept x ~ g if u·M·g(x) ≤ p(x),   u ~ Uniform(0, 1)
```

```
LLN: (1/n) Σ Xᵢ → E[X]
CLT: √n · (X̄ − μ) / σ → N(0, 1)
Standard error: SE(X̄) = σ / √n
```

```
z = (X̄ − μ₀) / (σ / √n)
p-value (two-sided) = 2·(1 − Φ(|z|))
Reject H₀ at level α  ⇔  p < α
Type I error = α (false positive); Type II error = β (miss the effect); power = 1 − β
```

AI Deep Dive — Conditional & Independence

> **🎲 The Concept**
>
> P(A | B) = P(A ∩ B) / P(B), defined whenever P(B) > 0. It "zooms in" on B and renormalises so the new sample space has total probability 1. **Independence** is the special case where conditioning doesn't change anything: A ⊥ B iff P(A | B) = P(A) iff P(A ∩ B) = P(A)P(B). The **chain rule** $P(X_1, \ldots, X_n) = P(X_1) \cdot P(X_2 \mid X_1) \cdot \ldots \cdot P(X_n \mid X_1, \ldots, X_{n-1})$ decomposes any joint distribution into a sequence of conditionals — which is exactly how an autoregressive language model factors the probability of a sentence. *Conditional independence* X ⊥ Y | Z is the workhorse of graphical

models: it says Z "shields" X from Y.

> 💡 ***Intuition:*** *"Given B" is a verbal restriction of the universe to where B is true. Independence says: the universe-restriction didn't matter. Most useful AI structure (HMMs, Bayes nets, transformers) is built from carefully chosen conditional-independence assumptions.*

? Why AI needs this

Conditioning is how every model handles "given the input, predict the output." It's the entire point of supervised learning.

- Classifiers learn P(y | x) — the conditional distribution of label given features
- LLMs learn P(token_t | tokens_{<t}) — chain rule applied to text
- Naive Bayes *assumes* features are conditionally independent given the label
- Causal inference: P(Y | do(X)) is conditioning on an *intervention*, not observation

⚙ How AI uses it

The chain rule is the structural identity that makes autoregressive LLMs work:

```
# LLM: P(sentence) = ∏ P(token_t | tokens_
```

🧮 Worked Example

The base-rate fallacy. Disease prevalence 0.1%, test sensitivity 99%, specificity 95%. A patient tests positive — what's the actual probability they're sick?

P(disease | positive test)

GIVEN	P(D) = 0.001, P(+ | D) = 0.99, P(+ | ¬D) = 0.05
STEP 1	P(+ ∩ D) = P(+ | D) · P(D) = 0.99 · 0.001 = 0.00099
STEP 2	P(+ ∩ ¬D) = P(+ | ¬D) · P(¬D) = 0.05 · 0.999 = 0.04995
STEP 3	P(+) = P(+ ∩ D) + P(+ ∩ ¬D) = 0.00099 + 0.04995 = 0.05094
STEP 4	P(D | +) = P(+ ∩ D) / P(+) = 0.00099 / 0.05094 ≈ 0.0194
RESULT	Only ~1.94% chance you actually have the disease! False positives (5% of 99.9% of people) swamp the true positives (99% of 0.1%). The base rate dominates.

AI Deep Dive — Bernoulli & Binomial

📖 The Concept

Bernoulli(p) is the simplest non-trivial distribution: a single 0/1 outcome with probability p of being 1. Binomial(n, p) is what you get when you sum n independent Bernoulli(p) variables — the count of "successes" in n trials. The Binomial PMF $P(X = k) = C(n, k)\, p^k\, (1-p)^{n-k}$ has three pieces: the combinatorial coefficient C(n, k) counting which trials were successes, p^k for those k successes, and $(1-p)^{n-k}$ for the n−k failures. Mean np and variance np(1−p) follow from linearity of expectation applied to the Bernoulli sum. As n grows with p fixed, Binomial converges to Normal (CLT); as n grows with np fixed, it converges to Poisson (rare-event regime).

> 💡 ***Intuition:*** *Bernoulli = "yes/no question." Binomial = "how many yes-es out of n questions?" These two cover an enormous fraction of real-world counting situations, from A/B test conversions to spam-filter precision.*

? Why AI needs this

Binary classification, dropout, and many evaluation metrics are Bernoulli/Binomial under the hood.

- Sigmoid output of a binary classifier IS the parameter p of a Bernoulli
- BCEWithLogitsLoss = negative log-likelihood under Bernoulli
- Dropout: each neuron Bernoulli(p_keep); a layer's kept-count is Binomial
- Accuracy on a test set: #correct out of n is Binomial — bounds the eval uncertainty

⚙ How AI uses it

Binary cross-entropy IS Bernoulli negative log-likelihood:

```
p = torch.sigmoid(logit)        # Bernoulli parameter
# NLL of label y under Bernoulli(p):
loss = -(y*torch.log(p) + (1-y)*torch.log(1-p))
# Equivalently:
loss = F.binary_cross_entropy_with_logits(logit, y)
# Dropout: Bernoulli mask
mask = torch.bernoulli(torch.full_like(x, 0.8))
```

🧮 Worked Example

Test-set accuracy uncertainty. A model gets 850 of 1000 examples right — what's the standard error on this estimate of true accuracy?

Confidence interval for accuracy = 0.85, n = 1000

GIVEN	k = 850 correct, n = 1000, model treats each example as Bernoulli(p)
STEP 1	Estimate $\hat{p} = k/n = 0.85$
STEP 2	Var of count = $np(1-p) = 1000 \cdot 0.85 \cdot 0.15 = 127.5$
STEP 3	$SE(\hat{p}) = \sqrt{\hat{p}(1-\hat{p})/n} = \sqrt{0.1275/1000} \approx 0.0113$
STEP 4	95% CI $\approx \hat{p} \pm 1.96 \cdot SE = 0.85 \pm 0.022 \rightarrow [0.828, 0.872]$
RESULT	Reporting "85% accuracy" without ±2.2 percentage points is meaningless. With n = 100 the CI widens to ±7%. Always pair an accuracy with its sample size.

AI Deep Dive — Categorical & Softmax

🎲 The Concept

A Categorical(p_1, …, p_K) distribution is fully specified by K non-negative numbers that sum to 1. Softmax is the canonical way to *parameterise* such a distribution from K free real numbers (logits). Three properties make softmax the right choice. **(1)** Translation invariance: softmax(z + c) = softmax(z). **(2)** Order-preserving: rankings of z_i and p_i match. **(3)** Smooth and differentiable everywhere — argmax isn't, which is why argmax can't be backpropagated through but softmax can. The temperature T is a knob that interpolates between argmax ($T \rightarrow 0$) and uniform ($T \rightarrow \infty$), giving exact control over how peaky the distribution is — top-p, top-k, and nucleus sampling all play in this temperature regime.

💡 ***Intuition:*** *Logits are "scores." Softmax converts scores into a probability budget — the difference between two scores controls how much budget separates them, exponentially. Doubling the gap squares the relative odds.*

? Why AI needs this

Multi-class classification, language modelling, attention, and RL policies all output Categorical distributions parameterised by softmax.

- Output layer for K-class classifier: linear → softmax → cross-entropy loss
- Each LLM step samples from softmax(logits / T) over the vocabulary
- Attention weights are softmax over similarity scores
- Policy gradient RL: π(a | s) = softmax of action-value scores

⚙ How AI uses it

Always use the numerically stable forms (log-softmax, F.cross_entropy):

```
probs = F.softmax(logits, dim=-1)
log_probs = F.log_softmax(logits, dim=-1)
# Cross-entropy combines both stably:
loss = F.cross_entropy(logits, labels)
# Sample with temperature:
dist = Categorical(logits=logits / T)
token = dist.sample()
```

🧮 Worked Example

Logits [3, 1, 0]. Compute softmax at T = 1 and T = 2; see how temperature flattens the distribution.

Softmax of [3, 1, 0] at temperature 1 vs 2

GIVEN	logits z = [3, 1, 0]

T = 1	exp(z) = [e^3, e^1, e^0] = [20.09, 2.72, 1.00], Σ = 23.81
→ PROBS	softmax = [0.844, 0.114, 0.042] — class 0 dominates
T = 2	z / T = [1.5, 0.5, 0]; exp = [4.48, 1.65, 1.00], Σ = 7.13
→ PROBS	softmax = [0.629, 0.231, 0.140] — much flatter
RESULT	Same logits, T = 1 → 84.4% on class 0; T = 2 → 62.9%. LLM "creativity" knobs literally are this T.

AI Deep Dive — Gaussian

📚 The Concept

The Normal distribution $N(\mu, \sigma^2)$ has PDF $p(x) = (2\pi\sigma^2)^{-1/2} \exp(-(x-\mu)^2 / (2\sigma^2))$. Three remarkable properties make it ubiquitous. **(1) Closure under linear operations:** if $X \sim N(\mu, \sigma^2)$, then $aX + b \sim N(a\mu + b, a^2\sigma^2)$. Sums of independent Normals are Normal. **(2) Maximum entropy** among distributions with fixed mean and variance — it's the "most uncertain" choice given those constraints. **(3) Central Limit Theorem:** sums of many independent variables — regardless of their original shape — approach a Normal. The multivariate generalisation $N(\mu, \Sigma)$ keeps all these properties and parameterises a covariance matrix Σ that controls correlations between dimensions.

> 💡 ***Intuition:*** *The Normal is the lazy, default, "no further information" choice for a continuous quantity. That's not a bug; the maximum-entropy property says it's mathematically the most honest distribution consistent with knowing only the mean and variance.*

? Why AI needs this

Gaussian is the default noise model, the default initialisation, the default prior, and the latent space of every VAE.

- Weight init: N(0, 2/n) (He init) tuned so activations stay variance-1 layer-to-layer
- VAE latent $z \sim N(0, I)$; reparameterisation: $z = \mu + \sigma \odot \varepsilon$ with $\varepsilon \sim N(0, I)$
- Diffusion: each forward step adds $N(0, \beta_t I)$ noise to the data
- MSE loss = NLL under $N(\text{prediction}, \sigma^2)$ — least-squares is Gaussian MLE

⚙ How AI uses it

torch.randn samples standard normal; reparameterisation enables VAE backprop:

```
eps = torch.randn_like(mu)            # N(0, I)
z = mu + sigma * eps                  # N(mu, sigma^2)
nll = 0.5 * ((x - mu)/sigma).pow(2) \
      + torch.log(sigma) + 0.5*math.log(2*math.pi)
# He initialization (for ReLU layers)
nn.init.kaiming_normal_(layer.weight)
```

🧮 Worked Example

VAE reparameterisation. Encoder outputs μ = 2.0 and σ = 0.5 for one latent dimension. Sample z; show why the trick keeps gradients flowing.

Reparameterisation: z = μ + σε

GIVEN	μ = 2.0, σ = 0.5; want z ~ N(2.0, 0.25)
NAIVE	z = torch.normal(μ, σ) → no gradient w.r.t. μ, σ (sampling is non-differentiable)
TRICK	Sample ε ~ N(0, 1) once, then z = μ + σ·ε. Suppose ε = 0.4 → z = 2.0 + 0.5·0.4 = 2.2
∂Z/∂M	= 1 (gradient flows directly)
∂Z/∂Σ	= ε = 0.4 (gradient flows; randomness pushed into ε which has no params)
RESULT	Same sample z = 2.2 with full gradient connectivity. This trick — separating "where to sample" from "how to randomise" — is the entire mechanical reason VAEs and policy gradient RL can be trained by backprop.

AI Deep Dive — Beta & Dirichlet

📚 The Concept

Beta(α, β) is a distribution on the unit interval; Dirichlet(α_1, …, α_K) is a distribution on the (K−1)-simplex (vectors of

K non-negative numbers summing to 1). Both are *conjugate priors*: combine Beta(α, β) with Bernoulli/Binomial data and you get back another Beta with simply updated parameters. The hyperparameters α, β behave like *pseudo-counts* — Beta(2, 2) acts like "I've already seen one success and one failure," giving a prior centred at 0.5 but uncertain. Beta(10, 10) is the same centre but much more confident. Setting α = β = 1 gives the uniform distribution — no information. Dirichlet plays the same role for K-class probability vectors and is the prior in latent Dirichlet allocation (LDA), the backbone of topic modelling.

> 💡 ***Intuition:*** *Treat α and β as "fake data" you pretend to have observed before seeing real data. Conjugacy then says: real data + fake data → updated belief, with no integration required. It's the cleanest closed-form Bayesian update available.*

? Why AI needs this

Beta/Dirichlet show up wherever you put a prior on a probability — A/B testing, Thompson sampling, topic models, Bayesian neural nets.

- Thompson sampling for multi-armed bandits maintains Beta(α, β) per arm
- Bayesian A/B testing: prior Beta(1, 1), posterior Beta(1+wins, 1+losses)
- LDA: each document is Dirichlet over topics, each topic Dirichlet over words
- Label smoothing turns one-hot targets into Dirichlet means → softer cross-entropy

⚙ How AI uses it

Conjugate updating is one line:

```
alpha, beta = 1, 1   # uniform prior
for win in observations:           # Bernoulli stream
    if win: alpha += 1
    else:   beta  += 1
# Mean of Beta = alpha / (alpha + beta)
# Thompson sample: torch.distributions.Beta(a, b).sample()
dist = torch.distributions.Dirichlet(torch.tensor([1.,1.,1.]))
theta = dist.sample()  # 3-class probability vector
```

🧮 Worked Example

You're A/B-testing a button colour. Prior Beta(1, 1) (uniform). Observe 7 clicks out of 10 visitors. Compute posterior, mean, and 95% credible interval.

Bayesian A/B test: Beta(1,1) → posterior after 7/10

GIVEN	Prior: Beta(1, 1) = Uniform[0, 1]; data: 7 successes, 3 failures
STEP 1	Posterior = Beta(1 + 7, 1 + 3) = Beta(8, 4)
STEP 2	Posterior mean = α/(α+β) = 8/12 ≈ 0.667
STEP 3	Posterior mode = (α−1)/(α+β−2) = 7/10 = 0.700 (= MLE)
STEP 4	95% credible interval ≈ [0.39, 0.89] (from Beta(8, 4) quantiles)
RESULT	Best estimate ~67% click rate, but a wide interval. With 100 visits and 70 clicks, posterior would be Beta(71, 31), mean 0.696, CI ≈ [0.604, 0.778] — much tighter. Sample size shrinks the credible interval.

AI Deep Dive — Exponential & Poisson

🧠 The Concept

Poisson and Exponential are two sides of the same coin: they describe a process where events happen independently at constant average rate λ. **Poisson(λ)** counts how many events happen in a unit time window; mean and variance both equal λ — a striking constraint that lets you spot Poisson data fast (Var ≈ Mean). **Exponential(λ)** describes the time between consecutive events; it's the unique continuous distribution that is *memoryless*: P(T > s + t | T > s) = P(T > t). The Exponential's discrete cousin is the Geometric distribution. Together they form the building blocks of Markov chains, queuing theory, and survival analysis.

> 💡 ***Intuition:*** *Imagine raindrops on a roof. Poisson tells you how many drops in a minute; Exponential tells you how long until the next drop. Same physics, two different questions, one parameter λ.*

? Why AI needs this

Rare-event counts, time-to-event modelling, RL reward sparsity, and policy gradients with discounting all draw on

Poisson/Exponential.

- NLP: word frequencies in long-tail vocab approximately Poisson per document
- Click-through-rate (CTR) models: clicks per impression often Poisson-Gamma
- Survival analysis: time-to-failure modelled with Exponential or Weibull (its generalisation)
- RL: discount factor $\gamma^t = e^{(-\lambda t)}$ for an effective discount rate $\lambda = -\ln \gamma$

⚙ How AI uses it

Sampling and likelihood are one line each:

```
poisson = torch.distributions.Poisson(rate=3.0)
counts = poisson.sample((1000,))   # event counts
exp = torch.distributions.Exponential(rate=3.0)
gaps = exp.sample((1000,))         # waiting times
# NLL of observed count k under Poisson(lambda):
nll = lam - k*torch.log(lam) + torch.lgamma(k+1)
```

🧮 Worked Example

A web service averages 3 requests per second. What's P(exactly 5 requests in the next second)? And what's the average wait until the next request?

Poisson + Exponential at rate λ = 3 / sec

GIVEN	Arrivals are Poisson($\lambda = 3$ per second), gaps Exponential($\lambda = 3$)
STEP 1	$P(X = 5) = e^{-3} \cdot 3^5 / 5! = 0.0498 \cdot 243 / 120 = 0.1008$
STEP 2	$P(X = 0) = e^{-3} \approx 0.0498$ (idle second)
STEP 3	$E[\text{gap}] = 1/\lambda = 1/3 \approx 0.333$ sec; Var = 1/9
STEP 4	$P(\text{gap} > 1 \text{ sec}) = e^{-3 \cdot 1} = e^{-3} \approx 0.0498$ — same as P(0 arrivals in 1 sec). Two views, same answer.
RESULT	10% chance of exactly 5 requests; ~5% chance of going a full second with none. Both follow from one rate parameter — capacity planning loves this.

AI Deep Dive — Joint Distributions

🗐 The Concept

Two RVs X, Y have a *joint* p(x, y) over the product space. From it you recover the *marginal* by summing out: $p(x) = \Sigma_y\, p(x, y)$ (or integrating). The *conditional* is $p(x \mid y) = p(x, y) / p(y)$ — and rearranging gives the chain rule $p(x, y) = p(y) \cdot p(x \mid y) = p(x) \cdot p(y \mid x)$, which is what makes Bayes' theorem work. The bivariate Normal shows the geometry beautifully: contour ellipses tilted by correlation ρ. Conditioning on Y = y slices the ellipse and gives another Normal, with mean shifted along the regression line and variance shrunk by $(1 - \rho^2)$. This shrinkage *is* "explained variance" — R^2 in linear regression literally equals ρ^2. For more than two variables, the joint factorises along whatever conditional-independence structure your model assumes (Naive Bayes, Bayes nets, HMMs, transformers).

> 💡 ***Intuition:*** *A joint distribution is a 2D heatmap. Marginalising = projecting onto one axis. Conditioning = slicing along a fixed value of the other axis and re-normalising. The slice is always thinner than the projection — that thinning is the information gain.*

? Why AI needs this

Every supervised learner targets either p(y | x) (discriminative) or p(x, y) (generative). The factorisation choice *is* the modelling choice.

- Discriminative models (logistic regression, neural nets) parameterise p(y | x) directly
- Generative models (Naive Bayes, GMM, diffusion) parameterise p(x, y) or p(x)
- Latent variable models: $p(x) = \int p(x \mid z)\, p(z)\, dz$ — marginalise out z
- Multivariate Gaussians: covariance matrix = full joint structure for Normal data

⚙ How AI uses it

Multivariate Normals are first-class in deep learning libraries:

```
mu = torch.zeros(2)
cov = torch.tensor([[1.0, 0.7], [0.7, 1.0]])  # rho=0.7
mvn = torch.distributions.MultivariateNormal(mu, cov)
samples = mvn.sample((1000,))   # joint samples
log_p = mvn.log_prob(x)          # joint log-density
# Latent variable model: marginal via Monte Carlo
# log p(x) ≈ log mean_z exp(log p(x|z) + log p(z))
```

🧮 Worked Example

(X, Y) ~ bivariate Normal with $\mu_X = \mu_Y = 0$, $\sigma_X = \sigma_Y = 1$, $\rho = 0.7$. Compute the marginal of X and the conditional of X given Y = 1.

Bivariate Normal: marginal X and conditional X | Y = 1

GIVEN	$\mu_X = \mu_Y = 0$, $\sigma_X = \sigma_Y = 1$, $\rho = 0.7$
STEP 1	Marginal: X ~ N(0, 1) (joint Normal → each marginal is Normal with same μ, σ)
STEP 2	Conditional mean: $E[X \mid Y=1] = \mu_X + \rho(\sigma_X/\sigma_Y)(1 - \mu_Y) = 0 + 0.7 \cdot 1 \cdot 1 = 0.7$
STEP 3	Conditional variance: $Var(X \mid Y=1) = (1 - \rho^2)\,\sigma_X^2 = (1 - 0.49) \cdot 1 = 0.51$
STEP 4	So X | Y = 1 ~ N(0.7, 0.51) — narrower ($\sigma \approx 0.71$) than marginal X ~ N(0, 1)
RESULT	Knowing Y = 1 shifts our best guess of X from 0 to 0.7 and shrinks the variance by 49%. That 49% is exactly ρ^2 — the "R^2 explained" by Y. This is linear regression, derived from joint Normality.

AI Deep Dive — Bayes' Theorem

📚 The Concept

Bayes' theorem rewrites the joint p(H, E) two ways and equates them: p(H, E) = p(H) p(E | H) = p(E) p(H | E). Solve for p(H | E): p(H | E) = p(E | H) p(H) / p(E). The pieces have names. **Prior** p(H) — what you believed about H before seeing E. **Likelihood** p(E | H) — how plausible the evidence is under each hypothesis. **Marginal evidence** p(E) = ∫ p(E | H) p(H) dH — a normalising constant. **Posterior** p(H | E) — your updated belief. The deep insight: posterior ∝ likelihood × prior, with the evidence acting only as a normaliser. Multiplying repeatedly is sequential Bayesian updating — every new observation moves the posterior closer to the truth, weighted by how informative each observation is. With Gaussian likelihoods and priors, posteriors stay Gaussian and *precisions* ($1/\sigma^2$) add cleanly.

💡 ***Intuition:*** *Prior is "what I knew." Likelihood is "what this evidence says, assuming each hypothesis." Posterior is the multiplied compromise. The stronger the prior or the more decisive the likelihood, the more it dominates the product.*

? Why AI needs this

Bayesian methods sit underneath probabilistic ML, regularisation theory, and the entire workflow of model uncertainty.

- L2 regularisation = MAP under a Gaussian prior on weights
- L1 regularisation = MAP under a Laplace prior (sparsity-inducing)
- Bayesian neural nets place priors on weights, infer posteriors from data
- Naive Bayes classifier: P(class | features) ∝ P(features | class) P(class)
- Spam filtering, fraud detection, medical AI all rest on Bayes

⚙ How AI uses it

Bayes is everywhere from regularisation to amortised inference:

```
# L2 regularization = Gaussian prior on weights
loss = nll + 0.5 * lam * (w**2).sum()  # MAP
# Bayesian linear regression closed form
# Posterior precision = prior precision + sigma^-2 X^T X
# Variational inference (used in BNN, VAE):
# minimize KL(q(w) || p(w | data))
```

🧮 Worked Example

Spam filter using Bayes. Prior P(spam) = 0.4. The word "free" appears in 60% of spam, 5% of ham. An email contains "free" — what's P(spam | "free")?

P(spam | "free") via Bayes' theorem

GIVEN	P(spam) = 0.4, P("free" | spam) = 0.6, P("free" | ham) = 0.05
STEP 1	P("free" ∩ spam) = 0.6 · 0.4 = 0.24
STEP 2	P("free" ∩ ham) = 0.05 · 0.6 = 0.03
STEP 3	P("free") = 0.24 + 0.03 = 0.27
STEP 4	P(spam | "free") = 0.24 / 0.27 ≈ 0.889
RESULT	Seeing "free" raised the spam probability from 40% (prior) to 88.9% (posterior). Naive Bayes generalises this to many words assumed conditionally independent given the class.

AI Deep Dive — Entropy & Information

🗎 The Concept

Entropy $H(X) = -E[\log p(X)] = -\Sigma\, p(x) \log p(x)$ measures the average information content of one sample from p. The information of a single outcome with probability p is $-\log p$ — rare outcomes carry more bits. Three milestone properties. **(1)** $H(X) \geq 0$ with equality iff X is deterministic. **(2)** Among distributions on K outcomes, the uniform distribution maximises H at log K. **(3)** Conditioning never increases entropy: $H(X \mid Y) \leq H(X)$, with equality iff $X \perp Y$. **Mutual information** $I(X; Y) = H(X) - H(X \mid Y)$ measures how much knowing Y reduces uncertainty in X — a non-negative, symmetric, unit-free measure of dependence. Differential entropy generalises this to continuous RVs, with one quirk: differential entropy can be negative.

> 💡 ***Intuition:*** *Entropy answers "how many yes/no questions, on average, to learn the outcome?" A fair die has $\log_2 6 \approx 2.58$ bits — you can pin it down in roughly 3 well-chosen questions. Mutual information says "how many questions does Y save you when guessing X?"*

? Why AI needs this

Entropy and mutual information appear in nearly every loss function, regulariser, and information bottleneck.

- Cross-entropy loss = expected NLL = entropy of true labels + KL divergence to predictions
- Decision trees split on feature with maximum information gain (= mutual information)
- InfoNCE / contrastive losses are lower bounds on mutual information
- RL exploration: entropy bonus on policy keeps π from collapsing too fast (PPO)
- Information bottleneck: compress X but preserve I(Z; Y) for the task

⚙ How AI uses it

Entropy is one line; mutual info typically a contrastive estimator:

```
probs = F.softmax(logits, dim=-1)
log_probs = F.log_softmax(logits, dim=-1)
entropy = -(probs * log_probs).sum(dim=-1)   # per-sample H
# RL policy regularization (PPO):
loss = pg_loss - ent_coef * entropy.mean()
# InfoNCE (mutual info estimator)
loss = -F.log_softmax(scores, dim=1)[:, 0].mean()
```

🧮 Worked Example

Compute the entropy of a uniform distribution over 4 outcomes vs a near-deterministic one. Verify the bounds $0 \leq H \leq \log_2 4$.

Entropy of uniform vs near-deterministic over 4 outcomes

UNIFORM	p = [0.25, 0.25, 0.25, 0.25]
H	$H = -4 \cdot (0.25 \cdot \log_2 0.25) = -4 \cdot (0.25 \cdot (-2)) = 2$ bits $= \log_2 4$ ✓ (max)
PEAKY	p = [0.97, 0.01, 0.01, 0.01]
H	$H = -0.97 \cdot \log_2 0.97 - 3 \cdot (0.01 \cdot \log_2 0.01) = 0.0426 + 3 \cdot 0.0664 = 0.244$ bits
RESULT	Same support of 4 outcomes, but peaky distribution carries ~12% of the uniform's entropy. A neural-net classifier with peaked softmax has low entropy (high confidence). PPO adds entropy bonus to keep policy from getting peaky too fast.

AI Deep Dive — KL & Cross-Entropy

📚 The Concept

KL divergence $D_{KL}(P \| Q) = E_P[\log(P/Q)]$ measures the expected number of extra bits needed to encode samples from P using a code optimised for Q. **It's not symmetric.** "Forward KL" $D_{KL}(P \| Q)$ penalises Q being zero where P has mass — Q is forced to be *mode-covering*, spreading itself to cover all of P's modes. "Reverse KL" $D_{KL}(Q \| P)$ penalises Q having mass where P is zero — Q becomes *mode-seeking*, latching onto a single mode. This asymmetry is the core distinction between maximum-likelihood training (forward KL) and variational inference (reverse KL). Cross-entropy $H(P, Q) = H(P) + D_{KL}(P \| Q)$: since H(P) doesn't depend on Q, minimising cross-entropy is equivalent to minimising KL — that's why all classifiers optimise cross-entropy. Jensen-Shannon divergence symmetrises KL; the f-divergence family generalises both.

> 💡 ***Intuition:*** *Cross-entropy is a "bad encoder" cost: how many bits per sample do you waste using Q when P is the truth? KL is the wasted-bits-only part. Two distributions agree fully → no waste, KL = 0.*

? Why AI needs this

KL and cross-entropy *are* the loss functions of modern ML.

- Multi-class classification: cross-entropy = NLL = forward KL up to a constant
- VAE objective: ELBO = $E[\log p(x \mid z)] - KL(q(z \mid x) \| p(z))$ — both directions appear
- RLHF / DPO / PPO: KL penalty keeps fine-tuned policy near reference policy
- Knowledge distillation: student matches teacher's softmax — minimise KL(teacher ‖ student)
- GAN's f-divergence variants generalise vanilla JS-divergence GANs

⚙ How AI uses it

F.cross_entropy and F.kl_div both built in:

```
loss = F.cross_entropy(logits, labels)
# KL between two softmax distributions:
log_q = F.log_softmax(student_logits, dim=-1)
p     = F.softmax(teacher_logits, dim=-1)
kl = F.kl_div(log_q, p, reduction='batchmean')
# RLHF / PPO trust-region term:
kl_to_ref = (log_pi_new - log_pi_ref).mean()
```

🧮 Worked Example

True P = [0.7, 0.3], predicted Q = [0.5, 0.5]. Compute $KL(P \| Q)$ and cross-entropy $H(P, Q)$ in nats.

KL and cross-entropy on a 2-outcome problem

GIVEN	P = [0.7, 0.3], Q = [0.5, 0.5] (using natural log → units: nats)
KL TERM 1	$0.7 \cdot \ln(0.7 / 0.5) = 0.7 \cdot \ln(1.4) = 0.7 \cdot 0.3365 = 0.2356$
KL TERM 2	$0.3 \cdot \ln(0.3 / 0.5) = 0.3 \cdot \ln(0.6) = 0.3 \cdot (-0.5108) = -0.1533$
KL SUM	$D_{KL}(P \| Q) = 0.2356 - 0.1533 = 0.0823$ nats (≈ 0.119 bits)
ENTROPY OF P	$H(P) = -0.7 \cdot \ln 0.7 - 0.3 \cdot \ln 0.3 = 0.2497 + 0.3612 = 0.6109$ nats
CE	$H(P, Q) = H(P) + KL = 0.6109 + 0.0823 = 0.6931$ nats (= ln 2, since Q is uniform)
RESULT	Q "wastes" 0.082 nats per sample by being uniform when P leans toward class 1. Training would push Q's first probability toward 0.7 — cross-entropy gradient does exactly that.

AI Deep Dive — Maximum Likelihood

📚 The Concept

MLE is a *frequentist* point estimator: pick the θ that maximises the likelihood $L(\theta) = \prod p(x_i \mid \theta)$. Working in log-space turns product into sum and avoids underflow: $\log L(\theta) = \Sigma \log p(x_i \mid \theta)$. MLE has lovely asymptotic guarantees: under mild conditions it's **consistent** ($\theta \to \theta_{true}$ as $n \to \infty$), **asymptotically efficient** (lowest possible variance, attaining the Cramér-Rao bound), and **asymptotically Normal** ($\theta \approx N(\theta_{true}, I^{-1}/n)$ where I is the Fisher information). On finite samples, MLE can be *biased* — for example, the MLE of σ^2 in a Gaussian uses 1/n, but the unbiased estimator uses 1/(n−1) (Bessel's correction). MLE also *overfits* when models are flexible: maximising the likelihood without regularisation is the cause of overfitting, fixed in topic 16 by adding a prior.

💡 ***Intuition:*** *MLE asks: "given my data, which parameter makes this exact data set most plausible?" When the answer is the sample mean, sample variance, or the proportion that worked, that's MLE giving you the obvious estimator — which is exactly why those classical estimators are so common.*

? Why AI needs this

Most loss functions in ML *are* negative log-likelihoods under specific distributional assumptions.

- MSE = NLL under Gaussian noise: MLE of regression weights is least-squares
- Cross-entropy = NLL under Categorical labels: MLE of classifier params
- Logistic regression = MLE of Bernoulli probabilities given features
- Language modelling = MLE on next-token Categorical, summed over corpus

⚙ How AI uses it

Training a deep network *is* MLE by gradient descent on NLL:

```
# Gaussian regression: MLE = least squares
nll = 0.5*((y - mu)/sigma)**2 + log(sigma)
# Equivalently
loss = F.mse_loss(model(x), y)  # MLE of mu, fixed sigma
# Classifier: MLE of softmax params
loss = F.cross_entropy(logits, labels)
# LM: MLE summed over tokens
loss = F.cross_entropy(logits.flatten(0,1), tokens.flatten())
```

🧮 Worked Example

Five data points {2, 3, 4, 5, 6} drawn from $N(\mu, \sigma^2)$. Compute the MLE for μ and σ^2.

Closed-form Gaussian MLE on 5 points

GIVEN	$x = [2, 3, 4, 5, 6]$, assume $x_i \sim N(\mu, \sigma^2)$ i.i.d.
STEP 1	$\log L = \Sigma \log p(x_i \mid \mu, \sigma^2)$; take $\partial/\partial\mu = 0 \rightarrow \hat{\mu} = (1/n) \Sigma x_i$
STEP 2	$\hat{\mu} = (2 + 3 + 4 + 5 + 6) / 5 = 20 / 5 = 4$
STEP 3	$\partial/\partial\sigma^2 = 0 \rightarrow \hat{\sigma}^2 = (1/n) \Sigma (x_i - \hat{\mu})^2 = (4 + 1 + 0 + 1 + 4) / 5 = 10 / 5 = 2$
NOTE	Unbiased $\hat{\sigma}^2$ uses $1/(n-1) = 10/4 = 2.5$ (Bessel correction); MLE uses $1/n = 2.0 \rightarrow$ biased low
RESULT	$\hat{\mu} = 4$, $\hat{\sigma}^2 = 2$ (MLE). Linear regression's normal equations are exactly this argument applied to a vector of weights.

AI Deep Dive — MAP & Priors

📚 The Concept

MAP estimation maximises the posterior $p(\theta \mid \text{data}) \propto p(\text{data} \mid \theta)\, p(\theta)$. Taking logs: argmax of $[\log p(\text{data} \mid \theta) + \log p(\theta)]$. The first term is the log-likelihood (drives MLE); the second is the log-prior (the regulariser). The crucial AI insight is the *exact correspondence* between regularisation and Bayesian priors. Adding $\lambda\|w\|^2 / 2$ to the loss is mathematically identical to assuming a Gaussian prior $w \sim N(0, 1/\lambda)$. Adding $\lambda\|w\|_1$ corresponds to a Laplace prior, which has a sharp peak at zero and is responsible for the sparsity-inducing behaviour of L1. Heavy-tailed priors (e.g. Student-t, Cauchy) yield robust estimators that ignore outliers. As $n \rightarrow \infty$, the prior contribution becomes negligible compared to the likelihood — MAP → MLE — which is why with enough data, the regulariser barely matters.

💡 ***Intuition:*** *A prior is "data you brought from home." It pulls your estimate toward your prior beliefs in proportion to its strength. The strength is measured in pseudo-counts — Beta(2, 2) is "I've already seen one of each" — so 1 real flip plus this prior is effectively 3 observations.*

? Why AI needs this

Every weight regulariser is a MAP prior. Choosing one IS doing Bayesian modelling.

- Weight decay (AdamW): Gaussian prior on w → L2 regulariser → MAP
- L1 / Lasso: Laplace prior → induces sparsity → feature selection
- Dropout as Bayesian approximation (Gal & Ghahramani, 2016)
- Label smoothing: Dirichlet prior on label distribution → softer targets
- "Pretrain + fine-tune" is informally MAP: pretrained weights are a prior

⚙ How AI uses it

Weight decay = L2 = Gaussian prior:

```
# L2 regularization is MAP under N(0, 1/lambda) prior on weights
optimizer = torch.optim.AdamW(model.parameters(),
    lr=1e-3, weight_decay=0.01)   # weight_decay = lambda
# Equivalently in the loss:
reg = 0.5 * lam * sum((p**2).sum() for p in params)
loss = nll + reg                    # MAP objective
```

🧮 Worked Example

You see 7 heads in 10 flips. With a Beta(2, 2) prior, what is the MAP estimate of the coin's bias? Compare to MLE.

MLE vs MAP for a Bernoulli bias

GIVEN	7H / 10 flips, prior Beta(2, 2)
MLE	$\hat{p}$_MLE = 7/10 = 0.700 (no regularisation)
POSTERIOR	Beta(α + h, β + n − h) = Beta(2 + 7, 2 + 3) = Beta(9, 5)
MAP	mode of Beta(9, 5) = (α − 1)/(α + β − 2) = 8/12 = 0.667
EFFECT	Prior pulled MAP from 0.700 toward 0.5 by 0.033. With 100 flips and 70 heads → MAP = (72−1)/(102−2) = 71/100 = 0.710 — barely shifted
RESULT	Beta(2, 2) acts like 2 extra "fake" observations, one of each. With small data, this matters; with large data, the data swamps the prior. This is exactly how weight decay tames small-data overfitting and gracefully lets go for large-data training.

AI Deep Dive — Bayesian Inference

📚 The Concept

Full Bayesian inference produces a *distribution* over parameters, not a point estimate. Posterior: p(θ | D) ∝ p(D | θ) p(θ). For most realistic models the marginal evidence p(D) = ∫ p(D | θ) p(θ) dθ is intractable, so we use one of three strategies: **(1) conjugate priors** — closed-form updates (Beta-Binomial, Normal-Normal, Dirichlet-Multinomial). **(2) MCMC** — Metropolis-Hastings, Hamiltonian Monte Carlo, NUTS — sample from the posterior without ever computing p(D). **(3) Variational inference (VI)** — pick a tractable family q(θ; φ) and minimise KL(q ‖ posterior) — turns inference into optimisation; this is what VAEs do. The headline benefit: predictions made by integrating over the posterior — the *posterior predictive* p(x_new | D) = ∫ p(x_new | θ) p(θ | D) dθ — incorporate parameter uncertainty automatically. They're calibrated; they don't pretend they know more than they do.

> 💡 ***Intuition:*** *MLE/MAP say "the answer is θ." Bayesian says "the answer is this whole cloud of plausible θ, weighted by how well each fits the data." For prediction, the cloud generates a wider, calibrated forecast — not just the single best guess.*

? Why AI needs this

Calibrated uncertainty matters wherever decisions cost real money — medical, autonomous driving, active learning, RL exploration.

- Bayesian neural nets give per-prediction uncertainty (Gal & Ghahramani 2016 with dropout)
- Active learning: query points where posterior predictive uncertainty is highest
- Bandit / Thompson sampling: action drawn from posterior over reward params
- Bayesian optimisation (BO) for hyperparameter search uses GP posteriors
- Modern LLM uncertainty quantification uses MC dropout or deep ensembles as Bayesian approximations

⚙ How AI uses it

Conjugate update is one line; PyMC and NumPyro handle MCMC:

```
# Beta-Binomial (closed form)
alpha, beta = 2, 2          # prior pseudo-counts
alpha += heads;  beta += (n - heads)   # posterior
# Posterior predictive: P(next=H | D) = alpha/(alpha+beta)
# MCMC sketch (NumPyro)
def model(data):
    theta = numpyro.sample('theta', dist.Beta(2, 2))
    numpyro.sample('y', dist.Bernoulli(theta), obs=data)
mcmc = MCMC(NUTS(model), num_warmup=500, num_samples=2000)
```

🧮 Worked Example

Coin with prior Beta(2, 2). After 50 flips with 30 heads, give the posterior, its mean, and predictive probability for the next flip being heads.

Beta-Binomial: 50 flips, 30 heads

GIVEN	prior Beta(2, 2), data: 30 heads, 20 tails
POSTERIOR	Beta(2 + 30, 2 + 20) = Beta(32, 22)
MEAN	$\alpha / (\alpha + \beta) = 32 / 54 \approx 0.5926$
VARIANCE	$\alpha\beta / [(\alpha+\beta)^2(\alpha+\beta+1)] = 32 \cdot 22 / (54^2 \cdot 55) \approx 0.00440 \rightarrow SD \approx 0.0663$
95% CI	≈ mean ± 2·SD ≈ [0.460, 0.725] (approx; exact via Beta quantiles)
PREDICT	P(next = H | data) = posterior mean = 0.5926. Not 30/50 = 0.60 (MLE) — the prior pulls slightly toward 0.5. This is calibrated forecasting: the posterior predictive accounts for both the data and the residual uncertainty.

AI Deep Dive — Sampling Methods

📐 The Concept

Monte Carlo replaces expectations with sample averages: $E[f(X)] \approx (1/n) \Sigma f(x_i)$. The estimator is unbiased and the law of large numbers gives convergence; the central limit theorem gives error bars of order $1/\sqrt{n}$ — a remarkable feature, because this rate *doesn't depend on dimension*. Different problems need different sampling techniques. **Inverse-CDF sampling** works when F^{-1} is easy. **Rejection sampling** wraps a tight envelope $M \cdot g(x)$ around the target; acceptance rate is $1/M$. **Importance sampling** samples from a proposal q and reweights by $p(x)/q(x)$, avoiding rejection but introducing variance. **MCMC** (Metropolis-Hastings, Gibbs, HMC, NUTS) builds a Markov chain whose stationary distribution is the target — needs only an unnormalised target density, perfect for posteriors. **Quasi-Monte Carlo** uses low-discrepancy sequences instead of random samples for better deterministic error rates.

💡 ***Intuition:*** *Sampling turns a hard integral into many easy operations. The price you pay is variance, but the law of large numbers gives $1/\sqrt{n}$ convergence in any dimension — better than grid integration as soon as $d \geq 4$.*

? Why AI needs this

Sampling is everywhere: VAEs, diffusion, RL policies, LLM decoding, Bayesian inference, even gradient estimators.

- VAE/policy gradient: reparameterised samples for low-variance gradients
- Diffusion models: every generated image is the end of a long sampling chain
- LLM decoding: top-k, top-p, nucleus sampling are all sample-from-softmax variants
- RL exploration: Thompson sampling draws θ from the posterior to pick actions
- Bayesian neural nets / probabilistic programs: posterior MCMC via NumPyro, Stan

⚙ How AI uses it

NumPyro & PyTorch make modern sampling one-liners:

```
# Inverse CDF
u = torch.rand(1000); x = dist.icdf(u)
# Importance sampling estimator
w = p(samples) / q(samples)
estimate = (w * f(samples)).mean() / w.mean()
# MCMC (NumPyro NUTS)
mcmc = MCMC(NUTS(model), num_warmup=500, num_samples=2000)
mcmc.run(rng_key, data)
# LM nucleus sampling: top-p subset, renormalise, sample
```

🧮 Worked Example

Use rejection sampling to draw from Beta(2, 5) using a Uniform(0, 1) proposal. What's the acceptance rate?

Rejection sampling Beta(2, 5) with Uniform proposal

TARGET	$p(x)$ = Beta(2, 5) PDF, mode at $x = (2-1)/(2+5-2) = 0.2$
STEP 1	Peak height = $p(0.2) \approx 2.46$ (using $B(2,5) = 1/30$ normaliser)
STEP 2	Proposal $q(x)$ = Uniform(0, 1) → $q(x) = 1$. Need $M \geq \max(p/q) = 2.46$. Pick $M = 2.5$.
STEP 3	For each draw: $x \sim U(0,1)$; $u \sim U(0,1)$; accept if $u \cdot 2.5 \leq p(x)$
STEP 4	Acceptance rate = $1/M = 0.40$ → ~40% of proposals are accepted
RESULT	To get 1000 Beta(2, 5) samples, expect ~2500 proposals. Higher M = lower efficiency. In high dimensions, M explodes exponentially — that's why MCMC replaces rejection for nontrivial Bayesian problems.

AI Deep Dive — LLN & CLT

📚 The Concept

Both theorems concern $\bar{X}_n = (1/n) \Sigma X_i$ for i.i.d. samples X_i with mean μ and finite variance σ^2. **Law of Large Numbers** says $\bar{X}_n \to \mu$ — the sample mean converges to the true mean. There are two flavours: weak LLN (convergence in probability) and strong LLN (almost-sure convergence). LLN justifies estimating expectations by averages, which underpins all of empirical risk minimisation. **Central Limit Theorem** goes further: it characterises *how fast* $\bar{X}_n$ converges. The standardised sample mean $\sqrt{n}\,(\bar{X}_n - \mu) / \sigma$ converges in distribution to N(0, 1), regardless of the source distribution's shape. Two consequences: $SE = \sigma/\sqrt{n}$ shrinks at the universal "square-root rate," and approximate confidence intervals $\bar{X}_n \pm 1.96 \cdot SE$ work for almost any parent distribution given enough data. CLT extends to martingales (martingale CLT) and to dependent data (mixing CLT) — both used in modern theory of stochastic gradient descent.

💡 ***Intuition:*** *LLN says "averages stabilise." CLT says "averages stabilise in a Gaussian-shaped way." Together they explain why almost every well-behaved measurement in science looks Gaussian, and why the standard error always looks like $\sigma/\sqrt{n}$.*

? Why AI needs this

Mini-batch SGD, evaluation metrics, model averaging, and bootstrap CIs all rely on LLN/CLT.

- Mini-batch gradient $\approx E[\nabla L(X; \theta)]$ — unbiased by LLN; variance shrinks as batch grows (CLT)
- Test-set loss → true risk by LLN — that's why eval sets work
- Bootstrap confidence intervals: resample to get SE without parametric assumptions
- Deep ensemble averaging stabilises predictions via LLN
- SGD convergence theory uses martingale CLT under stochastic gradient noise

⚙ How AI uses it

Standard error and z-score are workhorse calculations:

```
x = test_losses                  # length n
mean = x.mean()
se = x.std(unbiased=True) / n**0.5    # SE(mean) by CLT
ci95 = (mean - 1.96*se, mean + 1.96*se)
# Mini-batch SGD gradient noise:
# Var(g_batch) = Var(g_single) / batch_size  (CLT)
```

🧮 Worked Example

Source: Uniform(0, 1). Take n = 30 samples and average. What's the approximate distribution of the mean?

CLT applied to U(0, 1) with n = 30

SOURCE	$X \sim$ Uniform(0, 1) → $\mu = 0.5$, $\sigma^2 = 1/12 \approx 0.0833$
STEP 1	Mean of sample mean: $E[\bar{X}] = \mu = 0.5$
STEP 2	Variance of sample mean: $Var(\bar{X}) = \sigma^2 / n = 0.0833 / 30 \approx 0.00278$
STEP 3	$SE = \sqrt{Var(\bar{X})} = \sqrt{0.00278} \approx 0.0527$
CLT	$\bar{X} \approx N(0.5, 0.00278)$, so 95% CI $\approx 0.5 \pm 1.96 \cdot 0.0527 = [0.397, 0.603]$
RESULT	Source is uniform but the sample mean of 30 draws is essentially Gaussian around 0.5 with width ±0.10. To

halve the SE: quadruple n. To shrink it 10×: need 100× more data — the universal 1/√n curse and blessing of statistical learning.

AI Deep Dive — Hypothesis Testing

📚 The Concept

Hypothesis testing is decision-making under uncertainty. Set up the null hypothesis H_0 (the conservative default — "the new model is no better than the old one") and alternative H_1. Compute a test statistic T from data; under H_0 it has a known distribution. The **p-value** is $P(|T| \geq |t_observed| \mid H_0)$ — the probability of seeing data this extreme or more if H_0 were true. Reject H_0 at level α when $p < \alpha$; α is the false-positive rate (Type I error) you're willing to tolerate. The **power** $1 - \beta$ is the probability of correctly rejecting H_0 when H_1 is true; it grows with sample size and effect size, and shrinks with stricter α. Modern alternatives include confidence intervals (often more informative than p-values), Bayesian posteriors (give $P(H_1 \mid data)$ directly), and the false-discovery rate (FDR / Benjamini-Hochberg) for many simultaneous tests.

💡 ***Intuition:*** *A p-value is "if there were really no effect, how surprising would these data be?" Small p = surprising under H_0 = evidence against H_0. It's NOT $P(H_0 \mid data)$, and it's NOT effect size — only a measure of how compatible the data are with the null world.*

? Why AI needs this

Every model comparison, A/B test, and "is this regression coefficient real?" question is hypothesis testing.

- Online A/B testing: did variant B beat A? — z-test or sequential test
- Comparing model accuracies: McNemar's test for paired predictions
- Multiple comparisons (e.g. ablation studies): Bonferroni or FDR correction
- Permutation tests: resample labels to build the null distribution
- Sequential testing for early-stop in long experiments (mSPRT, always-valid p-values)

⚙ How AI uses it

z- and t-tests are one line; SciPy / statsmodels handle most cases:

```
from scipy import stats
# z-test (large n, known sigma)
z = (xbar - mu0) / (sigma / n**0.5)
p = 2 * (1 - stats.norm.cdf(abs(z)))   # two-sided
# Two-sample t-test (e.g., model A vs B)
t, p = stats.ttest_ind(losses_A, losses_B)
# Paired test for matched predictions
t, p = stats.ttest_rel(scores_A, scores_B)
```

🧮 Worked Example

You hypothesise the population mean is 5.0. Sample n = 100 with mean 5.2 and known σ = 1.0. Compute z, p, and decide at α = 0.05.

Two-sided z-test of μ = 5.0

GIVEN	H_0: μ = 5.0; H_1: μ ≠ 5.0; $\bar{X}$ = 5.2, n = 100, σ = 1.0, α = 0.05
STEP 1	SE = σ / √n = 1.0 / 10 = 0.1
STEP 2	z = (5.2 − 5.0) / 0.1 = 2.0
STEP 3	p = 2·(1 − Φ(2.0)) = 2·(1 − 0.9772) = 0.0455
STEP 4	Compare: p = 0.0455 < α = 0.05 → reject H_0
CI	95% CI = 5.2 ± 1.96·0.1 = [5.004, 5.396] — does not include 5.0 ✓
RESULT	Reject H_0 at the 5% level. But p = 0.045 vs 0.055 is barely-different evidence; report the CI alongside. With n = 1000, the same $\bar{X}$ = 5.2 gives z = 6.3 and p ≈ $3 \cdot 10^{-10}$ — a tiny effect can be screamingly significant given enough data, which is why effect size matters more than significance alone.

21Cheat SheetEvery formula from all 20 topics — one bookmark, complete reference

Axioms

$P(A) \geq 0 \cdot P(\Omega)=1 \cdot P(\cup A_i) = \Sigma P(A_i)$ for disjoint

Foundation of all probability

Complement

$P(A^c) = 1 - P(A)$

Probability not-A

Union (inclusion-exclusion)

$P(A \cup B) = P(A) + P(B) - P(A \cap B)$

Avoid double-counting

Conditional

$P(A|B) = P(A \cap B) / P(B)$

Update belief given evidence

Chain rule

$P(A,B,C) = P(A) \cdot P(B|A) \cdot P(C|A,B)$

Decompose joints — autoregressive LMs

Independence

$P(A \cap B) = P(A) \cdot P(B) \Leftrightarrow P(A|B)=P(A)$

Naive Bayes assumption

Bayes' theorem

$P(H|D) = P(D|H) \cdot P(H) / P(D)$

Posterior ∝ likelihood · prior

Total probability

$P(D) = \Sigma_i P(D|H_i) \cdot P(H_i)$

Bayes denominator (marginalisation)

06 Bernoulli & Binomial

Single coin flips and counts of successes — the building blocks of dropout

Plain English: A *Bernoulli(p)* is a single biased coin: heads with probability p, tails with 1−p. A *Binomial(n, p)* is the count of heads in n independent flips. In AI: a single dropout decision is Bernoulli; the number of dropped neurons in a layer is Binomial. Every binary classifier outputs a Bernoulli distribution.

```
Bernoulli(p): P(X=1) = p,   E[X] = p,   Var(X) = p(1-p)
Binomial(n, p): P(X=k) = C(n,k) p^k (1-p)^(n-k),   E[X] = np,   Var(X) = np(1-p)
```

AI Deep Dive — Bernoulli & Binomial

The Concept

Bernoulli(p) is the simplest non-trivial distribution: a single 0/1 outcome with probability p of being 1. Binomial(n, p) is what you get when you sum n independent Bernoulli(p) variables — the count of "successes" in n trials. The Binomial PMF $P(X = k) = C(n, k)\, p^k\, (1-p)^{n-k}$ has three pieces: the combinatorial coefficient C(n, k) counting which trials were successes, p^k for those k successes, and $(1-p)^{n-k}$ for the n−k failures. Mean np and variance np(1−p) follow from linearity of expectation applied to the Bernoulli sum. As n grows with p fixed, Binomial converges to Normal (CLT); as n grows with np fixed, it converges to Poisson (rare-event regime).

Intuition: *Bernoulli = "yes/no question." Binomial = "how many yes-es out of n questions?" These two cover an enormous fraction of real-world counting situations, from A/B test conversions to spam-filter precision.*

? Why AI needs this

Binary classification, dropout, and many evaluation metrics are Bernoulli/Binomial under the hood.

- Sigmoid output of a binary classifier IS the parameter p of a Bernoulli
- BCEWithLogitsLoss = negative log-likelihood under Bernoulli
- Dropout: each neuron Bernoulli(p_keep); a layer's kept-count is Binomial
- Accuracy on a test set: #correct out of n is Binomial — bounds the eval uncertainty

How AI uses it

Binary cross-entropy IS Bernoulli negative log-likelihood:

```
p = torch.sigmoid(logit)        # Bernoulli parameter
# NLL of label y under Bernoulli(p):
loss = -(y*torch.log(p) + (1-y)*torch.log(1-p))
# Equivalently:
loss = F.binary_cross_entropy_with_logits(logit, y)
# Dropout: Bernoulli mask
mask = torch.bernoulli(torch.full_like(x, 0.8))
```

Worked Example

Test-set accuracy uncertainty. A model gets 850 of 1000 examples right — what's the standard error on this estimate of true accuracy?

Confidence interval for accuracy = 0.85, n = 1000

GIVEN	k = 850 correct, n = 1000, model treats each example as Bernoulli(p)
STEP 1	Estimate $\hat{p} = k/n = 0.85$
STEP 2	Var of count = np(1−p) = 1000·0.85·0.15 = 127.5
STEP 3	$SE(\hat{p}) = \sqrt{\hat{p}(1-\hat{p})/n} = \sqrt{0.1275/1000} \approx 0.0113$
STEP 4	95% CI ≈ $\hat{p} \pm 1.96 \cdot SE = 0.85 \pm 0.022 \rightarrow [0.828, 0.872]$
RESULT	Reporting "85% accuracy" without ±2.2 percentage points is meaningless. With n = 100 the CI widens to ±7%. Always pair an accuracy with its sample size.

07 Categorical & Softmax

The distribution behind every classifier and every LLM token

🎯

Plain English: A Categorical distribution puts probability on K mutually exclusive options — class 0, class 1, …, class K−1. Softmax is the function that takes K real-valued logits and turns them into a Categorical: positive, sums to 1, monotone in the logits. Every multi-class classifier ends with a softmax. So does every LLM, every Mixture-of-Experts router, every reinforcement-learning policy.

```
softmax(z)ᵢ = exp(zᵢ) / Σⱼ exp(zⱼ)
With temperature T: softmax(z / T)ᵢ
argmax(z) = argmax(softmax(z))
```

AI Deep Dive — Categorical & Softmax

📚 The Concept

A Categorical(p_1, …, p_K) distribution is fully specified by K non-negative numbers that sum to 1. Softmax is the canonical way to *parameterise* such a distribution from K free real numbers (logits). Three properties make softmax the right choice. **(1)** Translation invariance: softmax(z + c) = softmax(z). **(2)** Order-preserving: rankings of z_i and p_i match. **(3)** Smooth and differentiable everywhere — argmax isn't, which is why argmax can't be backpropagated through but softmax can. The temperature T is a knob that interpolates between argmax (T → 0) and uniform (T → ∞), giving exact control over how peaky the distribution is — top-p, top-k, and nucleus sampling all play in this temperature regime.

💡 ***Intuition:*** *Logits are "scores." Softmax converts scores into a probability budget — the difference between two scores controls how much budget separates them, exponentially. Doubling the gap squares the relative odds.*

? Why AI needs this

Multi-class classification, language modelling, attention, and RL policies all output Categorical distributions parameterised by softmax.

- Output layer for K-class classifier: linear → softmax → cross-entropy loss
- Each LLM step samples from softmax(logits / T) over the vocabulary
- Attention weights are softmax over similarity scores
- Policy gradient RL: π(a | s) = softmax of action-value scores

⚙ How AI uses it

Always use the numerically stable forms (log-softmax, F.cross_entropy):

```
probs = F.softmax(logits, dim=-1)
log_probs = F.log_softmax(logits, dim=-1)
# Cross-entropy combines both stably:
loss = F.cross_entropy(logits, labels)
# Sample with temperature:
dist = Categorical(logits=logits / T)
token = dist.sample()
```

🧮 Worked Example

Logits [3, 1, 0]. Compute softmax at T = 1 and T = 2; see how temperature flattens the distribution.

Softmax of [3, 1, 0] at temperature 1 vs 2

GIVEN	logits z = [3, 1, 0]
T = 1	exp(z) = [e^3, e^1, e^0] = [20.09, 2.72, 1.00], Σ = 23.81
→ PROBS	softmax = [0.844, 0.114, 0.042] — class 0 dominates
T = 2	z / T = [1.5, 0.5, 0]; exp = [4.48, 1.65, 1.00], Σ = 7.13
→ PROBS	softmax = [0.629, 0.231, 0.140] — much flatter
RESULT	Same logits, T = 1 → 84.4% on class 0; T = 2 → 62.9%. LLM "creativity" knobs literally are this T.

08 Gaussian (Normal) Distribution

The bell curve — and why it's everywhere in AI

🔔

Plain English: The Normal distribution $N(\mu, \sigma^2)$ has the iconic bell shape: symmetric around mean μ, spread σ. About 68% of mass is within $\mu \pm \sigma$, 95% within $\mu \pm 2\sigma$, 99.7% within $\mu \pm 3\sigma$. The Central Limit Theorem says any large average tends toward Normal — which is why noise, errors, and aggregated quantities are so often modelled as Gaussian.

```
N(μ, σ²):   p(x) = (1/√(2πσ²)) · exp(−(x−μ)² / (2σ²))
Standard normal: μ = 0, σ = 1;   Z = (X − μ)/σ
Sum: X₁ ~ N(μ₁, σ₁²), X₂ ~ N(μ₂, σ₂²) independent → X₁+X₂ ~ N(μ₁+μ₂, σ₁²+σ₂²)
```

AI Deep Dive — Gaussian

📚 The Concept

The Normal distribution $N(\mu, \sigma^2)$ has PDF $p(x) = (2\pi\sigma^2)^{-1/2} \exp(-(x-\mu)^2 / (2\sigma^2))$. Three remarkable properties make it ubiquitous. **(1) Closure under linear operations:** if $X \sim N(\mu, \sigma^2)$, then $aX + b \sim N(a\mu + b, a^2\sigma^2)$. Sums of independent Normals are Normal. **(2) Maximum entropy** among distributions with fixed mean and variance — it's the "most uncertain" choice given those constraints. **(3) Central Limit Theorem:** sums of many independent variables — regardless of their original shape — approach a Normal. The multivariate generalisation $N(\mu, \Sigma)$ keeps all these properties and parameterises a covariance matrix Σ that controls correlations between dimensions.

💡 ***Intuition:*** *The Normal is the lazy, default, "no further information" choice for a continuous quantity. That's not a bug; the maximum-entropy property says it's mathematically the most honest distribution consistent with knowing only the mean and variance.*

? Why AI needs this

Gaussian is the default noise model, the default initialisation, the default prior, and the latent space of every VAE.

- Weight init: $N(0, 2/n)$ (He init) tuned so activations stay variance-1 layer-to-layer
- VAE latent $z \sim N(0, I)$; reparameterisation: $z = \mu + \sigma \odot \varepsilon$ with $\varepsilon \sim N(0, I)$
- Diffusion: each forward step adds $N(0, \beta_t I)$ noise to the data
- MSE loss = NLL under $N(\text{prediction}, \sigma^2)$ — least-squares is Gaussian MLE

⚙ How AI uses it

torch.randn samples standard normal; reparameterisation enables VAE backprop:

```
eps = torch.randn_like(mu)            # N(0, I)
z = mu + sigma * eps                  # N(mu, sigma^2)
nll = 0.5 * ((x - mu)/sigma).pow(2) \
      + torch.log(sigma) + 0.5*math.log(2*math.pi)
# He initialization (for ReLU layers)
nn.init.kaiming_normal_(layer.weight)
```

🧮 Worked Example

VAE reparameterisation. Encoder outputs $\mu = 2.0$ and $\sigma = 0.5$ for one latent dimension. Sample z; show why the trick keeps gradients flowing.

Reparameterisation: $z = \mu + \sigma\varepsilon$

GIVEN	$\mu = 2.0$, $\sigma = 0.5$; want $z \sim N(2.0, 0.25)$
NAIVE	z = torch.normal(μ, σ) → no gradient w.r.t. μ, σ (sampling is non-differentiable)
TRICK	Sample $\varepsilon \sim N(0, 1)$ once, then $z = \mu + \sigma \cdot \varepsilon$. Suppose $\varepsilon = 0.4 \rightarrow z = 2.0 + 0.5 \cdot 0.4 = 2.2$
∂Z/∂M	= 1 (gradient flows directly)
∂Z/∂Σ	= ε = 0.4 (gradient flows; randomness pushed into ε which has no params)
RESULT	Same sample z = 2.2 with full gradient connectivity. This trick — separating "where to sample" from "how to randomise" — is the entire mechanical reason VAEs and policy gradient RL can be trained by backprop.

09 Beta & Dirichlet Distributions

Distributions over probabilities — the natural priors for Bernoulli & Categorical

🎚

Plain English: Beta(α, β) is a distribution over a number p ∈ [0, 1] — useful when p itself is uncertain (the bias of a coin you haven't fully observed). Dirichlet generalises this to a vector that sums to 1 (a Categorical's parameters). They're the "conjugate priors" for Bernoulli and Categorical: posterior is the same family, just with updated counts.

```
Beta(α, β): p(x) ∝ x^(α−1) (1−x)^(β−1),   x ∈ [0,1]
Mean = α/(α+β),   Mode = (α−1)/(α+β−2) for α,β > 1
Conjugacy: Beta(α, β) + observe k successes / n−k failures → Beta(α+k, β+n−k)
```

AI Deep Dive — Beta & Dirichlet

📚 The Concept

Beta(α, β) is a distribution on the unit interval; Dirichlet(α_1, …, α_K) is a distribution on the (K−1)-simplex (vectors of K non-negative numbers summing to 1). Both are *conjugate priors*: combine Beta(α, β) with Bernoulli/Binomial data and you get back another Beta with simply updated parameters. The hyperparameters α, β behave like *pseudo-counts* — Beta(2, 2) acts like "I've already seen one success and one failure," giving a prior centred at 0.5 but uncertain. Beta(10, 10) is the same centre but much more confident. Setting α = β = 1 gives the uniform distribution — no information. Dirichlet plays the same role for K-class probability vectors and is the prior in latent Dirichlet allocation (LDA), the backbone of topic modelling.

💡 ***Intuition:*** *Treat α and β as "fake data" you pretend to have observed before seeing real data. Conjugacy then says: real data + fake data → updated belief, with no integration required. It's the cleanest closed-form Bayesian update available.*

? Why AI needs this

Beta/Dirichlet show up wherever you put a prior on a probability — A/B testing, Thompson sampling, topic models, Bayesian neural nets.

- Thompson sampling for multi-armed bandits maintains Beta(α, β) per arm
- Bayesian A/B testing: prior Beta(1, 1), posterior Beta(1+wins, 1+losses)
- LDA: each document is Dirichlet over topics, each topic Dirichlet over words
- Label smoothing turns one-hot targets into Dirichlet means → softer cross-entropy

⚙ How AI uses it

Conjugate updating is one line:

```
alpha, beta = 1, 1   # uniform prior
for win in observations:           # Bernoulli stream
    if win: alpha += 1
    else:   beta  += 1
# Mean of Beta = alpha / (alpha + beta)
# Thompson sample: torch.distributions.Beta(a, b).sample()
dist = torch.distributions.Dirichlet(torch.tensor([1.,1.,1.]))
theta = dist.sample()  # 3-class probability vector
```

🧮 Worked Example

You're A/B-testing a button colour. Prior Beta(1, 1) (uniform). Observe 7 clicks out of 10 visitors. Compute posterior, mean, and 95% credible interval.

Bayesian A/B test: Beta(1,1) → posterior after 7/10

GIVEN	Prior: Beta(1, 1) = Uniform[0, 1]; data: 7 successes, 3 failures
STEP 1	Posterior = Beta(1 + 7, 1 + 3) = Beta(8, 4)
STEP 2	Posterior mean = α/(α+β) = 8/12 ≈ 0.667
STEP 3	Posterior mode = (α−1)/(α+β−2) = 7/10 = 0.700 (= MLE)
STEP 4	95% credible interval ≈ [0.39, 0.89] (from Beta(8, 4) quantiles)

STEP 4 95% credible interval = [0.55, 0.85] (from Beta(8, 4) quantiles)

RESULT Best estimate ~67% click rate, but a wide interval. With 100 visits and 70 clicks, posterior would be Beta(71, 31), mean 0.696, CI ≈ [0.604, 0.778] — much tighter. Sample size shrinks the credible interval.

---

10 Exponential & Poisson Distributions

Waiting times and rare-event counts — the math of arrivals

⏱

Plain English: Exponential(λ) models the *time until* the next random event when events arrive at rate λ — server requests, photon hits, customer arrivals. Poisson(λ) models the *count* of those events in a fixed window. Memoryless: knowing you've already waited 5 min doesn't change how long you'll wait next.

```
Poisson(λ): P(X=k) = e^(−λ) λ^k / k!,   E[X] = Var(X) = λ
Exponential(λ): p(t) = λ e^(−λt),   E[T] = 1/λ,   Var(T) = 1/λ²
Connection: if events arrive Poisson(λ) per unit time, gaps between events are Exp(λ).
```

AI Deep Dive — Exponential & Poisson

🧠 The Concept

Poisson and Exponential are two sides of the same coin: they describe a process where events happen independently at constant average rate λ. **Poisson(λ)** counts how many events happen in a unit time window; mean and variance both equal λ — a striking constraint that lets you spot Poisson data fast (Var ≈ Mean). **Exponential(λ)** describes the time between consecutive events; it's the unique continuous distribution that is *memoryless*: P(T > s + t | T > s) = P(T > t). The Exponential's discrete cousin is the Geometric distribution. Together they form the building blocks of Markov chains, queuing theory, and survival analysis.

💡 ***Intuition:*** *Imagine raindrops on a roof. Poisson tells you how many drops in a minute; Exponential tells you how long until the next drop. Same physics, two different questions, one parameter λ.*

? Why AI needs this

Rare-event counts, time-to-event modelling, RL reward sparsity, and policy gradients with discounting all draw on Poisson/Exponential.

- NLP: word frequencies in long-tail vocab approximately Poisson per document
- Click-through-rate (CTR) models: clicks per impression often Poisson-Gamma
- Survival analysis: time-to-failure modelled with Exponential or Weibull (its generalisation)
- RL: discount factor $\gamma^t = e^{(-\lambda t)}$ for an effective discount rate $\lambda = -\ln \gamma$

⚙ How AI uses it

Sampling and likelihood are one line each:

```
poisson = torch.distributions.Poisson(rate=3.0)
counts = poisson.sample((1000,))   # event counts
exp = torch.distributions.Exponential(rate=3.0)
gaps = exp.sample((1000,))         # waiting times
# NLL of observed count k under Poisson(lambda):
nll = lam - k*torch.log(lam) + torch.lgamma(k+1)
```

🧮 Worked Example

A web service averages 3 requests per second. What's P(exactly 5 requests in the next second)? And what's the average wait until the next request?

Poisson + Exponential at rate λ = 3 / sec

GIVEN Arrivals are Poisson(λ = 3 per second), gaps Exponential(λ = 3)

STEP 1 $P(X = 5) = e^{-3} \cdot 3^5 / 5! = 0.0498 \cdot 243 / 120 = 0.1008$

STEP 2 $P(X = 0) = e^{-3} \approx 0.0498$ (idle second)

STEP 3 $E[gap] = 1/\lambda = 1/3 \approx 0.333$ sec; Var = 1/9

STEP 3	E[gap] = 1/λ = 1/3 = 0.333 sec, Var = 1/9
STEP 4	P(gap > 1 sec) = $e^{-3.1}$ = e^{-3} ≈ 0.0498 — same as P(0 arrivals in 1 sec). Two views, same answer.
RESULT	10% chance of exactly 5 requests; ~5% chance of going a full second with none. Both follow from one rate parameter — capacity planning loves this.

11 Joint, Marginal & Conditional Distributions

Two (or more) random variables, considered together

🗺

Plain English: A joint distribution p(x, y) describes two RVs simultaneously — every pair of values gets a probability (or density). The *marginal* p(x) ignores y by summing/integrating it out. The *conditional* p(x | y) is the distribution of x once y is observed. In ML, every supervised learning problem is about the joint p(x, y) — and any model is some assumption on how it factorises.

```
Joint: p(x, y); Marginal: p(x) = ∫ p(x, y) dy; Conditional: p(x | y) = p(x, y) / p(y)
X ⊥ Y  ⇔  p(x, y) = p(x) p(y) for all x, y
Bivariate Normal: X | Y = y  ~  N(μ_x + ρ(σ_x/σ_y)(y − μ_y),   (1−ρ²) σ_x²)
```

AI Deep Dive — Joint Distributions

📚 The Concept

Two RVs X, Y have a *joint* p(x, y) over the product space. From it you recover the *marginal* by summing out: p(x) = Σ_y p(x, y) (or integrating). The *conditional* is p(x | y) = p(x, y) / p(y) — and rearranging gives the chain rule p(x, y) = p(y) · p(x | y) = p(x) · p(y | x), which is what makes Bayes' theorem work. The bivariate Normal shows the geometry beautifully: contour ellipses tilted by correlation ρ. Conditioning on Y = y slices the ellipse and gives another Normal, with mean shifted along the regression line and variance shrunk by (1 − ρ^2). This shrinkage *is* "explained variance" — R^2 in linear regression literally equals ρ^2. For more than two variables, the joint factorises along whatever conditional-independence structure your model assumes (Naive Bayes, Bayes nets, HMMs, transformers).

💡 ***Intuition:*** *A joint distribution is a 2D heatmap. Marginalising = projecting onto one axis. Conditioning = slicing along a fixed value of the other axis and re-normalising. The slice is always thinner than the projection — that thinning is the information gain.*

? Why AI needs this

Every supervised learner targets either p(y | x) (discriminative) or p(x, y) (generative). The factorisation choice *is* the modelling choice.

- Discriminative models (logistic regression, neural nets) parameterise p(y | x) directly
- Generative models (Naive Bayes, GMM, diffusion) parameterise p(x, y) or p(x)
- Latent variable models: p(x) = ∫ p(x | z) p(z) dz — marginalise out z
- Multivariate Gaussians: covariance matrix = full joint structure for Normal data

⚙ How AI uses it

Multivariate Normals are first-class in deep learning libraries:

```
mu = torch.zeros(2)
cov = torch.tensor([[1.0, 0.7], [0.7, 1.0]])  # rho=0.7
mvn = torch.distributions.MultivariateNormal(mu, cov)
samples = mvn.sample((1000,))   # joint samples
log_p = mvn.log_prob(x)          # joint log-density
# Latent variable model: marginal via Monte Carlo
# log p(x) ≈ log mean_z exp(log p(x|z) + log p(z))
```

🧮 Worked Example

(X, Y) ~ bivariate Normal with μ_X = μ_Y = 0, σ_X = σ_Y = 1, ρ = 0.7. Compute the marginal of X and the conditional of X given Y = 1.

Bivariate Normal: marginal X and conditional X | Y = 1

GIVEN	μ_X = μ_Y = 0, σ_X = σ_Y = 1, ρ = 0.7

STEP 1	Marginal: $X \sim N(0, 1)$ (joint Normal → each marginal is Normal with same μ, σ)
STEP 2	Conditional mean: $E[X \mid Y=1] = \mu_X + \rho(\sigma_X/\sigma_Y)(1 - \mu_Y) = 0 + 0.7 \cdot 1 \cdot 1 = 0.7$
STEP 3	Conditional variance: $Var(X \mid Y=1) = (1 - \rho^2)\,\sigma_X^2 = (1 - 0.49) \cdot 1 = 0.51$
STEP 4	So $X \mid Y = 1 \sim N(0.7, 0.51)$ — narrower ($\sigma \approx 0.71$) than marginal $X \sim N(0, 1)$
RESULT	Knowing $Y = 1$ shifts our best guess of X from 0 to 0.7 and shrinks the variance by 49%. That 49% is exactly ρ^2 — the "R^2 explained" by Y. This is linear regression, derived from joint Normality.

---

12 Bayes' Theorem

How beliefs update when evidence arrives

🔄

Plain English: Bayes' theorem flips conditional probabilities. You know P(evidence | hypothesis) — the likelihood — and your prior belief P(hypothesis); Bayes gives you what you want, P(hypothesis | evidence) — the *posterior*. It's the engine of every spam filter, medical diagnostic, and Bayesian neural network.

```
P(H | E) = P(E | H) · P(H) / P(E)
posterior ∝ likelihood × prior
Gaussian conjugate update: μ_post = (μ_0/σ_0² + x/σ_L²) / (1/σ_0² + 1/σ_L²)
```

AI Deep Dive — Bayes' Theorem

📚 The Concept

Bayes' theorem rewrites the joint p(H, E) two ways and equates them: $p(H, E) = p(H)\,p(E \mid H) = p(E)\,p(H \mid E)$. Solve for p(H | E): $p(H \mid E) = p(E \mid H)\,p(H) / p(E)$. The pieces have names. **Prior** p(H) — what you believed about H before seeing E. **Likelihood** p(E | H) — how plausible the evidence is under each hypothesis. **Marginal evidence** $p(E) = \int p(E \mid H)\,p(H)\,dH$ — a normalising constant. **Posterior** p(H | E) — your updated belief. The deep insight: posterior ∝ likelihood × prior, with the evidence acting only as a normaliser. Multiplying repeatedly is sequential Bayesian updating — every new observation moves the posterior closer to the truth, weighted by how informative each observation is. With Gaussian likelihoods and priors, posteriors stay Gaussian and *precisions* ($1/\sigma^2$) add cleanly.

💡 ***Intuition:*** *Prior is "what I knew." Likelihood is "what this evidence says, assuming each hypothesis." Posterior is the multiplied compromise. The stronger the prior or the more decisive the likelihood, the more it dominates the product.*

? Why AI needs this

Bayesian methods sit underneath probabilistic ML, regularisation theory, and the entire workflow of model uncertainty.

- L2 regularisation = MAP under a Gaussian prior on weights
- L1 regularisation = MAP under a Laplace prior (sparsity-inducing)
- Bayesian neural nets place priors on weights, infer posteriors from data
- Naive Bayes classifier: P(class | features) ∝ P(features | class) P(class)
- Spam filtering, fraud detection, medical AI all rest on Bayes

⚙ How AI uses it

Bayes is everywhere from regularisation to amortised inference:

```
# L2 regularization = Gaussian prior on weights
loss = nll + 0.5 * lam * (w**2).sum()  # MAP
# Bayesian linear regression closed form
# Posterior precision = prior precision + sigma^-2 X^T X
# Variational inference (used in BNN, VAE):
# minimize KL(q(w) || p(w | data))
```

🧮 Worked Example

Spam filter using Bayes. Prior P(spam) = 0.4. The word "free" appears in 60% of spam, 5% of ham. An email contains "free" — what's P(spam | "free")?

P(spam | "free") via Bayes' theorem

GIVEN	P(spam) = 0.4, P("free" | spam) = 0.6, P("free" | ham) = 0.05
STEP 1	P("free" ∩ spam) = 0.6 · 0.4 = 0.24
STEP 2	P("free" ∩ ham) = 0.05 · 0.6 = 0.03
STEP 3	P("free") = 0.24 + 0.03 = 0.27
STEP 4	P(spam | "free") = 0.24 / 0.27 ≈ 0.889
RESULT	Seeing "free" raised the spam probability from 40% (prior) to 88.9% (posterior). Naive Bayes generalises this to many words assumed conditionally independent given the class.

13 Entropy & Information

How much uncertainty a distribution actually carries — measured in bits

🌡

Plain English: Entropy H(X) is the expected "surprise" of a random variable, measured in bits (log base 2) or nats (log base e). A fair coin has H = 1 bit; a biased near-deterministic coin has H ≈ 0. High entropy = lots of uncertainty = hard to predict. Low entropy = peaky distribution = easy to predict. Information theory's foundation, and the loss function for every classifier.

```
H(X) = −Σ p(x) log p(x)
0 ≤ H(X) ≤ log K
Joint: H(X, Y) = H(X) + H(Y | X); Mutual info: I(X; Y) = H(X) − H(X | Y)
```

AI Deep Dive — Entropy & Information

📚 The Concept

Entropy H(X) = −E[log p(X)] = −Σ p(x) log p(x) measures the average information content of one sample from p. The information of a single outcome with probability p is −log p — rare outcomes carry more bits. Three milestone properties. **(1)** H(X) ≥ 0 with equality iff X is deterministic. **(2)** Among distributions on K outcomes, the uniform distribution maximises H at log K. **(3)** Conditioning never increases entropy: H(X | Y) ≤ H(X), with equality iff X ⊥ Y. **Mutual information** I(X; Y) = H(X) − H(X | Y) measures how much knowing Y reduces uncertainty in X — a non-negative, symmetric, unit-free measure of dependence. Differential entropy generalises this to continuous RVs, with one quirk: differential entropy can be negative.

💡 ***Intuition:*** *Entropy answers "how many yes/no questions, on average, to learn the outcome?" A fair die has $\log_2 6 \approx 2.58$ bits — you can pin it down in roughly 3 well-chosen questions. Mutual information says "how many questions does Y save you when guessing X?"*

? Why AI needs this

Entropy and mutual information appear in nearly every loss function, regulariser, and information bottleneck.

- Cross-entropy loss = expected NLL = entropy of true labels + KL divergence to predictions
- Decision trees split on feature with maximum information gain (= mutual information)
- InfoNCE / contrastive losses are lower bounds on mutual information
- RL exploration: entropy bonus on policy keeps π from collapsing too fast (PPO)
- Information bottleneck: compress X but preserve I(Z; Y) for the task

⚙ How AI uses it

Entropy is one line; mutual info typically a contrastive estimator:

```
probs = F.softmax(logits, dim=-1)
log_probs = F.log_softmax(logits, dim=-1)
entropy = -(probs * log_probs).sum(dim=-1)   # per-sample H
# RL policy regularization (PPO):
loss = pg_loss - ent_coef * entropy.mean()
# InfoNCE (mutual info estimator)
loss = -F.log_softmax(scores, dim=1)[:, 0].mean()
```

🧮 Worked Example

Compute the entropy of a uniform distribution over 4 outcomes vs a near-deterministic one. Verify the bounds $0 \le H \le \log_2 4$.

Entropy of uniform vs near-deterministic over 4 outcomes

UNIFORM	p = [0.25, 0.25, 0.25, 0.25]
H	$H = -4\cdot(0.25\cdot\log_2 0.25) = -4\cdot(0.25\cdot(-2)) = 2$ bits $= \log_2 4$ ✓ (max)
PEAKY	p = [0.97, 0.01, 0.01, 0.01]
H	$H = -0.97\cdot\log_2 0.97 - 3\cdot(0.01\cdot\log_2 0.01) = 0.0426 + 3\cdot0.0664 = 0.244$ bits
RESULT	Same support of 4 outcomes, but peaky distribution carries ~12% of the uniform's entropy. A neural-net classifier with peaked softmax has low entropy (high confidence). PPO adds entropy bonus to keep policy from getting peaky too fast.

14 KL Divergence & Cross-Entropy

The loss function behind every classifier — and the regulariser in every VAE

📐

Plain English: KL divergence D_KL(P ‖ Q) measures how much P "differs" from Q in nats (or bits). It's not a true distance (asymmetric, no triangle inequality) but it's non-negative and zero iff P = Q. Cross-entropy H(P, Q) is the expected NLL of true distribution P under predicted distribution Q. Identity: H(P, Q) = H(P) + D_KL(P ‖ Q). Minimise cross-entropy ⇔ minimise KL.

```
D_KL(P ‖ Q) = Σ P(x) log(P(x) / Q(x)) ≥ 0
H(P, Q) = -Σ P(x) log Q(x)
H(P, Q) = H(P) + D_KL(P ‖ Q)
```

AI Deep Dive — KL & Cross-Entropy

📚 The Concept

KL divergence D_KL(P ‖ Q) = E_P[log(P/Q)] measures the expected number of extra bits needed to encode samples from P using a code optimised for Q. **It's not symmetric.** "Forward KL" D_KL(P ‖ Q) penalises Q being zero where P has mass — Q is forced to be *mode-covering*, spreading itself to cover all of P's modes. "Reverse KL" D_KL(Q ‖ P) penalises Q having mass where P is zero — Q becomes *mode-seeking*, latching onto a single mode. This asymmetry is the core distinction between maximum-likelihood training (forward KL) and variational inference (reverse KL). Cross-entropy H(P, Q) = H(P) + D_KL(P ‖ Q): since H(P) doesn't depend on Q, minimising cross-entropy is equivalent to minimising KL — that's why all classifiers optimise cross-entropy. Jensen-Shannon divergence symmetrises KL; the f-divergence family generalises both.

💡 ***Intuition:*** *Cross-entropy is a "bad encoder" cost: how many bits per sample do you waste using Q when P is the truth? KL is the wasted-bits-only part. Two distributions agree fully → no waste, KL = 0.*

? Why AI needs this

KL and cross-entropy *are* the loss functions of modern ML.

- Multi-class classification: cross-entropy = NLL = forward KL up to a constant
- VAE objective: ELBO = E[log p(x | z)] − KL(q(z | x) ‖ p(z)) — both directions appear
- RLHF / DPO / PPO: KL penalty keeps fine-tuned policy near reference policy
- Knowledge distillation: student matches teacher's softmax — minimise KL(teacher ‖ student)
- GAN's f-divergence variants generalise vanilla JS-divergence GANs

⚙ How AI uses it

F.cross_entropy and F.kl_div both built in:

```
loss = F.cross_entropy(logits, labels)
# KL between two softmax distributions:
log_q = F.log_softmax(student_logits, dim=-1)
p     = F.softmax(teacher_logits, dim=-1)
kl = F.kl_div(log_q, p, reduction='batchmean')
# RLHF / PPO trust-region term:
kl_to_ref = (log_pi_new - log_pi_ref).mean()
```

🧮 Worked Example

True P = [0.7, 0.3], predicted Q = [0.5, 0.5]. Compute KL(P ‖ Q) and cross-entropy H(P, Q) in nats.

KL and cross-entropy on a 2-outcome problem

GIVEN	P = [0.7, 0.3], Q = [0.5, 0.5] (using natural log → units: nats)
KL TERM 1	0.7 · ln(0.7 / 0.5) = 0.7 · ln(1.4) = 0.7 · 0.3365 = 0.2356
KL TERM 2	0.3 · ln(0.3 / 0.5) = 0.3 · ln(0.6) = 0.3 · (−0.5108) = −0.1533
KL SUM	D_KL(P ‖ Q) = 0.2356 − 0.1533 = 0.0823 nats (≈ 0.119 bits)
ENTROPY OF P	H(P) = −0.7·ln 0.7 − 0.3·ln 0.3 = 0.2497 + 0.3612 = 0.6109 nats
CE	H(P, Q) = H(P) + KL = 0.6109 + 0.0823 = 0.6931 nats (= ln 2, since Q is uniform)
RESULT	Q "wastes" 0.082 nats per sample by being uniform when P leans toward class 1. Training would push Q's first probability toward 0.7 — cross-entropy gradient does exactly that.

15 Maximum Likelihood Estimation (MLE)

Pick the parameters that make the observed data most plausible

🎯

Plain English: You have data, you pick a distribution family (e.g., Gaussian), and MLE asks: which specific parameters in that family make my data most likely? Take the joint likelihood ∏ p(x_i | θ), take its log to turn product into sum, take the derivative w.r.t. θ, set to zero. Almost every supervised learning algorithm is MLE in disguise — including least-squares regression and softmax classifiers.

```
θ_MLE = argmax_θ ∏ᵢ p(xᵢ | θ) = argmax_θ Σᵢ log p(xᵢ | θ)
For X ~ N(μ, σ²):   μ̂ = (1/n) Σ xᵢ,   σ̂² = (1/n) Σ (xᵢ − μ̂)²
NLL(θ) = − Σᵢ log p(xᵢ | θ)
```

AI Deep Dive — Maximum Likelihood

📚 The Concept

MLE is a *frequentist* point estimator: pick the θ that maximises the likelihood L(θ) = ∏ p(x_i | θ). Working in log-space turns product into sum and avoids underflow: log L(θ) = Σ log p(x_i | θ). MLE has lovely asymptotic guarantees: under mild conditions it's **consistent** (θ → θ_true as n → ∞), **asymptotically efficient** (lowest possible variance, attaining the Cramér-Rao bound), and **asymptotically Normal** (θ ≈ N(θ_true, I^{-1}/n) where I is the Fisher information). On finite samples, MLE can be *biased* — for example, the MLE of σ^2 in a Gaussian uses 1/n, but the unbiased estimator uses 1/(n−1) (Bessel's correction). MLE also *overfits* when models are flexible: maximising the likelihood without regularisation is the cause of overfitting, fixed in topic 16 by adding a prior.

💡 ***Intuition:*** *MLE asks: "given my data, which parameter makes this exact data set most plausible?" When the answer is the sample mean, sample variance, or the proportion that worked, that's MLE giving you the obvious estimator — which is exactly why those classical estimators are so common.*

? Why AI needs this

Most loss functions in ML *are* negative log-likelihoods under specific distributional assumptions.

- MSE = NLL under Gaussian noise: MLE of regression weights is least-squares
- Cross-entropy = NLL under Categorical labels: MLE of classifier params
- Logistic regression = MLE of Bernoulli probabilities given features
- Language modelling = MLE on next-token Categorical, summed over corpus

⚙ How AI uses it

Training a deep network *is* MLE by gradient descent on NLL:

```
# Gaussian regression: MLE = least squares
nll = 0.5*((y - mu)/sigma)**2 + log(sigma)
# Equivalently
loss = F.mse_loss(model(x), y)  # MLE of mu, fixed sigma
# Classifier: MLE of softmax params
loss = F.cross_entropy(logits, labels)
# LM: MLE summed over tokens
loss = F.cross_entropy(logits.flatten(0,1), tokens.flatten())
```

🧮 Worked Example

Five data points {2, 3, 4, 5, 6} drawn from $N(\mu, \sigma^2)$. Compute the MLE for μ and σ^2.

Closed-form Gaussian MLE on 5 points

GIVEN	$x = [2, 3, 4, 5, 6]$, assume $x_i \sim N(\mu, \sigma^2)$ i.i.d.
STEP 1	$\log L = \Sigma \log p(x_i \mid \mu, \sigma^2)$; take $\partial/\partial\mu = 0 \rightarrow \hat{\mu} = (1/n) \Sigma x_i$
STEP 2	$\hat{\mu} = (2 + 3 + 4 + 5 + 6) / 5 = 20 / 5 = 4$
STEP 3	$\partial/\partial\sigma^2 = 0 \rightarrow \hat{\sigma}^2 = (1/n) \Sigma (x_i - \hat{\mu})^2 = (4 + 1 + 0 + 1 + 4) / 5 = 10 / 5 = 2$
NOTE	Unbiased $\hat{\sigma}^2$ uses $1/(n-1) = 10/4 = 2.5$ (Bessel correction); MLE uses $1/n = 2.0 \rightarrow$ biased low
RESULT	$\hat{\mu} = 4$, $\hat{\sigma}^2 = 2$ (MLE). Linear regression's normal equations are exactly this argument applied to a vector of weights.

---

16 MAP Estimation & Priors

MLE plus a prior — equivalent to weight regularisation

⚓

Plain English: Maximum a Posteriori (MAP) finds the θ that maximises the *posterior* $p(\theta \mid \text{data}) \propto p(\text{data} \mid \theta)\, p(\theta)$. It's MLE's regularised cousin: instead of letting the data alone choose θ, you tug toward your prior. Strong prior = small weights → L2 regularisation. Heavy-tailed prior → robustness. Sparse prior → L1.

```
θ_MAP = argmax_θ p(θ | data) = argmax_θ [ log p(data | θ) + log p(θ) ]
L2 regulariser ⇔ Gaussian prior on weights: log p(w) = −λ‖w‖² / 2 + const
L1 regulariser ⇔ Laplace prior on weights → sparsity
```

AI Deep Dive — MAP & Priors

🧠 The Concept

MAP estimation maximises the posterior $p(\theta \mid \text{data}) \propto p(\text{data} \mid \theta)\, p(\theta)$. Taking logs: argmax of $[\log p(\text{data} \mid \theta) + \log p(\theta)]$. The first term is the log-likelihood (drives MLE); the second is the log-prior (the regulariser). The crucial AI insight is the *exact correspondence* between regularisation and Bayesian priors. Adding $\lambda\|w\|^2 / 2$ to the loss is mathematically identical to assuming a Gaussian prior $w \sim N(0, 1/\lambda)$. Adding $\lambda\|w\|_1$ corresponds to a Laplace prior, which has a sharp peak at zero and is responsible for the sparsity-inducing behaviour of L1. Heavy-tailed priors (e.g. Student-t, Cauchy) yield robust estimators that ignore outliers. As $n \rightarrow \infty$, the prior contribution becomes negligible compared to the likelihood — MAP → MLE — which is why with enough data, the regulariser barely matters.

💡 ***Intuition:*** *A prior is "data you brought from home." It pulls your estimate toward your prior beliefs in proportion to its strength. The strength is measured in pseudo-counts — Beta(2, 2) is "I've already seen one of each" — so 1 real flip plus this prior is effectively 3 observations.*

? Why AI needs this

Every weight regulariser is a MAP prior. Choosing one IS doing Bayesian modelling.

- Weight decay (AdamW): Gaussian prior on w → L2 regulariser → MAP
- L1 / Lasso: Laplace prior → induces sparsity → feature selection
- Dropout as Bayesian approximation (Gal & Ghahramani, 2016)
- Label smoothing: Dirichlet prior on label distribution → softer targets
- "Pretrain + fine-tune" is informally MAP: pretrained weights are a prior

⚙ How AI uses it

Weight decay = L2 = Gaussian prior:

```
# L2 regularization is MAP under N(0, 1/lambda) prior on weights
optimizer = torch.optim.AdamW(model.parameters(),
    lr=1e-3, weight_decay=0.01)   # weight_decay = lambda
# Equivalently in the loss:
reg = 0.5 * lam * sum((p**2).sum() for p in params)
loss = nll + reg                    # MAP objective
```

🧮 Worked Example

You see 7 heads in 10 flips. With a Beta(2, 2) prior, what is the MAP estimate of the coin's bias? Compare to MLE.

MLE vs MAP for a Bernoulli bias

GIVEN	7H / 10 flips, prior Beta(2, 2)
MLE	p̂_MLE = 7/10 = 0.700 (no regularisation)
POSTERIOR	Beta(α + h, β + n − h) = Beta(2 + 7, 2 + 3) = Beta(9, 5)
MAP	mode of Beta(9, 5) = (α − 1)/(α + β − 2) = 8/12 = 0.667
EFFECT	Prior pulled MAP from 0.700 toward 0.5 by 0.033. With 100 flips and 70 heads → MAP = (72−1)/(102−2) = 71/100 = 0.710 — barely shifted
RESULT	Beta(2, 2) acts like 2 extra "fake" observations, one of each. With small data, this matters; with large data, the data swamps the prior. This is exactly how weight decay tames small-data overfitting and gracefully lets go for large-data training.

---

17 Bayesian Inference

Don't pick a single θ — keep the whole posterior distribution

🌊

Plain English: MLE and MAP collapse parameter uncertainty to one number. Full Bayesian inference keeps the whole posterior p(θ | data) — and to make a prediction, integrates over all plausible θ weighted by their posterior probability. Result: predictions come with calibrated uncertainty for free. The catch: posteriors are usually intractable, so we sample (MCMC) or approximate (variational inference).

```
posterior: p(θ | D) = p(D | θ) p(θ) / p(D); p(D) = ∫ p(D | θ) p(θ) dθ
posterior predictive: p(x_new | D) = ∫ p(x_new | θ) p(θ | D) dθ
Beta-Binomial conjugacy: posterior ∝ θ^(α+h-1) (1-θ)^(β+n-h-1) → Beta(α+h, β+n-h)
```

AI Deep Dive — Bayesian Inference

📚 The Concept

Full Bayesian inference produces a *distribution* over parameters, not a point estimate. Posterior: p(θ | D) ∝ p(D | θ) p(θ). For most realistic models the marginal evidence p(D) = ∫ p(D | θ) p(θ) dθ is intractable, so we use one of three strategies: **(1) conjugate priors** — closed-form updates (Beta-Binomial, Normal-Normal, Dirichlet-Multinomial). **(2) MCMC** — Metropolis-Hastings, Hamiltonian Monte Carlo, NUTS — sample from the posterior without ever computing p(D). **(3) Variational inference (VI)** — pick a tractable family q(θ; φ) and minimise KL(q ‖ posterior) — turns inference into optimisation; this is what VAEs do. The headline benefit: predictions made by integrating over

the posterior — the *posterior predictive* $p(x_new \mid D) = \int p(x_new \mid \theta)\, p(\theta \mid D)\, d\theta$ — incorporate parameter uncertainty automatically. They're calibrated; they don't pretend they know more than they do.

> 💡 ***Intuition:*** *MLE/MAP say "the answer is θ." Bayesian says "the answer is this whole cloud of plausible θ, weighted by how well each fits the data." For prediction, the cloud generates a wider, calibrated forecast — not just the single best guess.*

? Why AI needs this

Calibrated uncertainty matters wherever decisions cost real money — medical, autonomous driving, active learning, RL exploration.

- Bayesian neural nets give per-prediction uncertainty (Gal & Ghahramani 2016 with dropout)
- Active learning: query points where posterior predictive uncertainty is highest
- Bandit / Thompson sampling: action drawn from posterior over reward params
- Bayesian optimisation (BO) for hyperparameter search uses GP posteriors
- Modern LLM uncertainty quantification uses MC dropout or deep ensembles as Bayesian approximations

⚙ How AI uses it

Conjugate update is one line; PyMC and NumPyro handle MCMC:

```
# Beta-Binomial (closed form)
alpha, beta = 2, 2          # prior pseudo-counts
alpha += heads;  beta += (n - heads)   # posterior
# Posterior predictive: P(next=H | D) = alpha/(alpha+beta)
# MCMC sketch (NumPyro)
def model(data):
    theta = numpyro.sample('theta', dist.Beta(2, 2))
    numpyro.sample('y', dist.Bernoulli(theta), obs=data)
mcmc = MCMC(NUTS(model), num_warmup=500, num_samples=2000)
```

🧮 Worked Example

Coin with prior Beta(2, 2). After 50 flips with 30 heads, give the posterior, its mean, and predictive probability for the next flip being heads.

Beta-Binomial: 50 flips, 30 heads

GIVEN	prior Beta(2, 2), data: 30 heads, 20 tails
POSTERIOR	Beta(2 + 30, 2 + 20) = Beta(32, 22)
MEAN	$\alpha / (\alpha + \beta) = 32 / 54 \approx 0.5926$
VARIANCE	$\alpha\beta / [(\alpha+\beta)^2(\alpha+\beta+1)] = 32 \cdot 22 / (54^2 \cdot 55) \approx 0.00440 \rightarrow SD \approx 0.0663$
95% CI	$\approx$ mean $\pm$ 2·SD $\approx$ [0.460, 0.725] (approx; exact via Beta quantiles)
PREDICT	P(next = H | data) = posterior mean = 0.5926. Not 30/50 = 0.60 (MLE) — the prior pulls slightly toward 0.5. This is calibrated forecasting: the posterior predictive accounts for both the data and the residual uncertainty.

---

18 Sampling Methods

When you can't compute, sample — Monte Carlo, importance, MCMC, rejection

🎲

Plain English: Most posterior integrals are impossible analytically. Solution: replace $E[f(X)]$ with $(1/n)\ \Sigma\ f(x_i)$ using samples. Direct sampling works for nice distributions (uniform, Normal, Beta). For nastier targets, use rejection sampling, importance sampling, or Markov-chain Monte Carlo — each trades off setup cost against sample quality.

```
Monte Carlo: E[f(X)] ≈ (1/n) Σ f(xᵢ),   xᵢ ~ p
Importance: E_p[f(X)] = E_q[f(X) p(X)/q(X)]
Rejection: accept x ~ g if u·M·g(x) ≤ p(x),   u ~ Uniform(0, 1)
```

AI Deep Dive — Sampling Methods

🎲 The Concept

Monte Carlo replaces expectations with sample averages: $E[f(X)] \approx (1/n) \Sigma f(x_i)$. The estimator is unbiased and the law of large numbers gives convergence; the central limit theorem gives error bars of order $1/\sqrt{n}$ — a remarkable feature, because this rate *doesn't depend on dimension*. Different problems need different sampling techniques. **Inverse-CDF sampling** works when F^{-1} is easy. **Rejection sampling** wraps a tight envelope $M \cdot g(x)$ around the target; acceptance rate is $1/M$. **Importance sampling** samples from a proposal q and reweights by $p(x)/q(x)$, avoiding rejection but introducing variance. **MCMC** (Metropolis-Hastings, Gibbs, HMC, NUTS) builds a Markov chain whose stationary distribution is the target — needs only an unnormalised target density, perfect for posteriors. **Quasi-Monte Carlo** uses low-discrepancy sequences instead of random samples for better deterministic error rates.

💡 ***Intuition:*** *Sampling turns a hard integral into many easy operations. The price you pay is variance, but the law of large numbers gives $1/\sqrt{n}$ convergence in any dimension — better than grid integration as soon as $d \geq 4$.*

? Why AI needs this

Sampling is everywhere: VAEs, diffusion, RL policies, LLM decoding, Bayesian inference, even gradient estimators.

- VAE/policy gradient: reparameterised samples for low-variance gradients
- Diffusion models: every generated image is the end of a long sampling chain
- LLM decoding: top-k, top-p, nucleus sampling are all sample-from-softmax variants
- RL exploration: Thompson sampling draws θ from the posterior to pick actions
- Bayesian neural nets / probabilistic programs: posterior MCMC via NumPyro, Stan

⚙ How AI uses it

NumPyro & PyTorch make modern sampling one-liners:

```
# Inverse CDF
u = torch.rand(1000); x = dist.icdf(u)
# Importance sampling estimator
w = p(samples) / q(samples)
estimate = (w * f(samples)).mean() / w.mean()
# MCMC (NumPyro NUTS)
mcmc = MCMC(NUTS(model), num_warmup=500, num_samples=2000)
mcmc.run(rng_key, data)
# LM nucleus sampling: top-p subset, renormalise, sample
```

🧮 Worked Example

Use rejection sampling to draw from Beta(2, 5) using a Uniform(0, 1) proposal. What's the acceptance rate?

Rejection sampling Beta(2, 5) with Uniform proposal

TARGET	$p(x) = \text{Beta}(2, 5)$ PDF, mode at $x = (2-1)/(2+5-2) = 0.2$
STEP 1	Peak height $= p(0.2) \approx 2.46$ (using $B(2,5) = 1/30$ normaliser)
STEP 2	Proposal $q(x) = \text{Uniform}(0, 1) \rightarrow q(x) = 1$. Need $M \geq \max(p/q) = 2.46$. Pick $M = 2.5$.
STEP 3	For each draw: $x \sim U(0,1)$; $u \sim U(0,1)$; accept if $u \cdot 2.5 \leq p(x)$
STEP 4	Acceptance rate $= 1/M = 0.40 \rightarrow \sim 40\%$ of proposals are accepted
RESULT	To get 1000 Beta(2, 5) samples, expect ~2500 proposals. Higher M = lower efficiency. In high dimensions, M explodes exponentially — that's why MCMC replaces rejection for nontrivial Bayesian problems.

19 Law of Large Numbers & Central Limit Theorem

Why averages stabilise — and why they look Gaussian

🌪

Plain English: Two miracles of probability. *LLN:* the sample mean of n i.i.d. RVs converges to the true mean as n grows. That's why training on more data works. *CLT:* the sample mean (suitably scaled) is approximately Normal — regardless of the shape of the underlying distribution. That's why Gaussians appear everywhere and why confidence intervals always look like ±1.96·SE.

```
LLN: (1/n) Σ Xᵢ → E[X]
CLT: √n · (X − μ) / σ → N(0, 1)
Standard error: SE(X) = σ / √n
```

AI Deep Dive — LLN & CLT

📚 The Concept

Both theorems concern $\bar{X}_n = (1/n) \Sigma X_i$ for i.i.d. samples X_i with mean μ and finite variance σ^2. **Law of Large Numbers** says $\bar{X}_n \to \mu$ — the sample mean converges to the true mean. There are two flavours: weak LLN (convergence in probability) and strong LLN (almost-sure convergence). LLN justifies estimating expectations by averages, which underpins all of empirical risk minimisation. **Central Limit Theorem** goes further: it characterises *how fast* $\bar{X}_n$ converges. The standardised sample mean $\sqrt{n}\,(\bar{X}_n - \mu) / \sigma$ converges in distribution to N(0, 1), regardless of the source distribution's shape. Two consequences: $SE = \sigma/\sqrt{n}$ shrinks at the universal "square-root rate," and approximate confidence intervals $\bar{X}_n \pm 1.96 \cdot SE$ work for almost any parent distribution given enough data. CLT extends to martingales (martingale CLT) and to dependent data (mixing CLT) — both used in modern theory of stochastic gradient descent.

> 💡 ***Intuition:*** *LLN says "averages stabilise." CLT says "averages stabilise in a Gaussian-shaped way." Together they explain why almost every well-behaved measurement in science looks Gaussian, and why the standard error always looks like $\sigma/\sqrt{n}$.*

? Why AI needs this

Mini-batch SGD, evaluation metrics, model averaging, and bootstrap CIs all rely on LLN/CLT.

- Mini-batch gradient $\approx E[\nabla L(X; \theta)]$ — unbiased by LLN; variance shrinks as batch grows (CLT)
- Test-set loss → true risk by LLN — that's why eval sets work
- Bootstrap confidence intervals: resample to get SE without parametric assumptions
- Deep ensemble averaging stabilises predictions via LLN
- SGD convergence theory uses martingale CLT under stochastic gradient noise

⚙ How AI uses it

Standard error and z-score are workhorse calculations:

```
x = test_losses                  # length n
mean = x.mean()
se = x.std(unbiased=True) / n**0.5    # SE(mean) by CLT
ci95 = (mean - 1.96*se, mean + 1.96*se)
# Mini-batch SGD gradient noise:
# Var(g_batch) = Var(g_single) / batch_size  (CLT)
```

🧮 Worked Example

Source: Uniform(0, 1). Take n = 30 samples and average. What's the approximate distribution of the mean?

CLT applied to U(0, 1) with n = 30

SOURCE	$X \sim \text{Uniform}(0, 1) \to \mu = 0.5, \sigma^2 = 1/12 \approx 0.0833$
STEP 1	Mean of sample mean: $E[\bar{X}] = \mu = 0.5$
STEP 2	Variance of sample mean: $\text{Var}(\bar{X}) = \sigma^2 / n = 0.0833 / 30 \approx 0.00278$
STEP 3	$SE = \sqrt{\text{Var}(\bar{X})} = \sqrt{0.00278} \approx 0.0527$
CLT	$\bar{X} \approx N(0.5, 0.00278)$, so 95% CI $\approx 0.5 \pm 1.96 \cdot 0.0527 = [0.397, 0.603]$
RESULT	Source is uniform but the sample mean of 30 draws is essentially Gaussian around 0.5 with width ±0.10. To halve the SE: quadruple n. To shrink it 10×: need 100× more data — the universal $1/\sqrt{n}$ curse and blessing of statistical learning.

20 Hypothesis Testing

Is the effect real, or could it just be random noise?

> ⚖

Plain English: Frame a null hypothesis H_0 ("nothing is happening") and an alternative H_1. Compute a test statistic from your data. Ask: assuming H_0 is true, how rare is a statistic at least this extreme? That probability is the *p-value*. Small p (typically < 0.05) → reject H_0 in favour of H_1. The math behind every A/B test, model comparison, and offline evaluation experiment.

```
z = (X − μ₀) / (σ / √n)
p-value (two-sided) = 2·(1 − Φ(|z|))
Reject H₀ at level α  ⇔  p < α
Type I error = α (false positive); Type II error = β (miss the effect); power = 1 − β
```

AI Deep Dive — Hypothesis Testing

📚 The Concept

Hypothesis testing is decision-making under uncertainty. Set up the null hypothesis H_0 (the conservative default — "the new model is no better than the old one") and alternative H_1. Compute a test statistic T from data; under H_0 it has a known distribution. The **p-value** is P(|T| ≥ |t_observed| | H_0) — the probability of seeing data this extreme or more if H_0 were true. Reject H_0 at level α when p < α; α is the false-positive rate (Type I error) you're willing to tolerate. The **power** 1 − β is the probability of correctly rejecting H_0 when H_1 is true; it grows with sample size and effect size, and shrinks with stricter α. Modern alternatives include confidence intervals (often more informative than p-values), Bayesian posteriors (give P(H_1 | data) directly), and the false-discovery rate (FDR / Benjamini-Hochberg) for many simultaneous tests.

💡 ***Intuition:*** *A p-value is "if there were really no effect, how surprising would these data be?" Small p = surprising under H_0 = evidence against H_0. It's NOT P(H_0 | data), and it's NOT effect size — only a measure of how compatible the data are with the null world.*

? Why AI needs this

Every model comparison, A/B test, and "is this regression coefficient real?" question is hypothesis testing.

- Online A/B testing: did variant B beat A? — z-test or sequential test
- Comparing model accuracies: McNemar's test for paired predictions
- Multiple comparisons (e.g. ablation studies): Bonferroni or FDR correction
- Permutation tests: resample labels to build the null distribution
- Sequential testing for early-stop in long experiments (mSPRT, always-valid p-values)

⚙ How AI uses it

z- and t-tests are one line; SciPy / statsmodels handle most cases:

```
from scipy import stats
# z-test (large n, known sigma)
z = (xbar - mu0) / (sigma / n**0.5)
p = 2 * (1 - stats.norm.cdf(abs(z)))   # two-sided
# Two-sample t-test (e.g., model A vs B)
t, p = stats.ttest_ind(losses_A, losses_B)
# Paired test for matched predictions
t, p = stats.ttest_rel(scores_A, scores_B)
```

🧮 Worked Example

You hypothesise the population mean is 5.0. Sample n = 100 with mean 5.2 and known σ = 1.0. Compute z, p, and decide at α = 0.05.

Two-sided z-test of μ = 5.0

GIVEN	H_0: μ = 5.0; H_1: μ ≠ 5.0; $\bar{X}$ = 5.2, n = 100, σ = 1.0, α = 0.05
STEP 1	SE = σ / √n = 1.0 / 10 = 0.1
STEP 2	z = (5.2 − 5.0) / 0.1 = 2.0
STEP 3	p = 2·(1 − Φ(2.0)) = 2·(1 − 0.9772) = 0.0455
STEP 4	Compare: p = 0.0455 < α = 0.05 → reject H_0
CI	95% CI = 5.2 ± 1.96·0.1 = [5.004, 5.396] — does not include 5.0 ✓
RESULT	Reject H_0 at the 5% level. But p = 0.045 vs 0.055 is barely-different evidence; report the CI alongside. With n = 1000, the same $\bar{X}$ = 5.2 gives z = 6.3 and p ≈ $3 \cdot 10^{-10}$ — a tiny effect can be screamingly significant given enough data, which is why effect size matters more than significance alone.

21 Cheat Sheet

Every formula from all 20 topics — one bookmark, complete reference

🎲 Probability Basics

Axioms
$P(A) \geq 0 \cdot P(\Omega)=1 \cdot P(\cup A_i) = \Sigma P(A_i)$ for disjoint
Foundation of all probability

Complement
$P(A^c) = 1 - P(A)$
Probability not-A

Union (inclusion-exclusion)
$P(A \cup B) = P(A) + P(B) - P(A \cap B)$
Avoid double-counting

Conditional
$P(A|B) = P(A \cap B) / P(B)$
Update belief given evidence

Chain rule
$P(A,B,C) = P(A) \cdot P(B|A) \cdot P(C|A,B)$
Decompose joints — autoregressive LMs

Independence
$P(A \cap B) = P(A) \cdot P(B) \Leftrightarrow P(A|B)=P(A)$
Naive Bayes assumption

Bayes' theorem
$P(H|D) = P(D|H) \cdot P(H) / P(D)$
Posterior ∝ likelihood · prior

Total probability
$P(D) = \Sigma_i P(D|H_i) \cdot P(H_i)$
Bayes denominator (marginalisation)

📊 Random Variables & Moments

PMF (discrete)

$p(x) = P(X=x), \Sigma p(x) = 1$

Probability of each outcome

PDF (continuous)

$p(x) \geq 0, \int p(x)dx = 1, P(a \leq X \leq b) = \int_a^b p(x)dx$

Density, not probability

CDF

$F(x) = P(X \leq x)$, monotone non-decreasing

Sampling, quantiles

Expectation

$E[X] = \Sigma\ x \cdot p(x)$ or $\int x \cdot p(x)\ dx$

Mean / centre of mass

Linearity of E

$E[aX + bY] = a \cdot E[X] + b \cdot E[Y]$

Always true (no independence needed)

Variance

$Var(X) = E[(X-\mu)^2] = E[X^2] - (E[X])^2$

Spread; SD = √Var

Var of sum

$Var(X+Y) = Var(X)+Var(Y)+2Cov(X,Y)$

Independent → Cov=0

Covariance / Correlation

$Cov = E[(X-\mu_X)(Y-\mu_Y)] \cdot \rho = Cov/(\sigma_X \sigma_Y) \in [-1,1]$

Linear association

🎲 Key Distributions

Bernoulli(p)
`P(X=1)=p · E=p · Var=p(1−p)`
Binary classification head

Binomial(n,p)
$P(X=k) = C(n,k)\ p^k\ (1-p)^{n-k} \cdot E=np \cdot Var=np(1-p)$
Sum of n Bernoullis

Categorical(π)
$P(X=k) = \pi_k,\ \Sigma\pi_k = 1$
Softmax outputs (next-token)

Softmax
$\pi_k = \exp(z_k/T) / \Sigma_j \exp(z_j/T)$
Logits → probabilities, T = temperature

Normal N(μ, σ²)
$p(x) = (1/\sqrt{(2\pi\sigma^2)}) \cdot \exp(-(x-\mu)^2/(2\sigma^2))$
VAE prior, reparam trick

Standard Normal Φ
`Φ(1)=0.8413, Φ(1.96)=0.975, Φ(2)=0.9772`
95% CI: ±1.96σ

Beta(α, β)
$p(x) \propto x^{(\alpha-1)}(1-x)^{(\beta-1)} \cdot E = \alpha/(\alpha+\beta)$
Conjugate prior for Bernoulli p

Dirichlet(α)
`Multivariate Beta on simplex`
Conjugate prior for Categorical

Poisson(λ)
$P(X=k) = \lambda^k\ e^{-k} / k! \cdot E=\lambda \cdot Var=\lambda$
Counts in fixed window

Exponential(λ)
$p(x) = \lambda e^{-\lambda x} \cdot E=1/\lambda \cdot$ `memoryless`
Gaps between Poisson events

🧮 Information Theory

Shannon entropy
$H(P) = -\Sigma\ p_i \log p_i \cdot$ `max at uniform`
Average info content / uncertainty

Cross-entropy
$H(P, Q) = -\Sigma\ p_i \log q_i \geq H(P)$
THE classification loss in DL

KL divergence
$D_KL(P\|Q) = \Sigma\ p_i \log(p_i/q_i) \geq 0$, `asymmetric`
VAEs, RLHF, diffusion training

Identity
$CE(P, Q) = H(P) + D_KL(P\|Q)$
Why minimising CE = matching P

Mutual information
$I(X;Y) = H(X) - H(X|Y) = D_KL(p(x,y)\|p(x)p(y))$
Feature selection, contrastive learning

JS divergence
$JS(P,Q) = ½\ KL(P\|M) + ½\ KL(Q\|M),\ M=½(P+Q)$
Symmetric, bounded; original GAN loss

🎯 Estimation

Likelihood
$L(\theta) = \prod_i p(x_i \mid \theta)$
Probability of data under θ

Log-likelihood
$\ell(\theta) = \Sigma_i \log p(x_i \mid \theta)$
Numerically stable, sums not products

MLE
$\theta_MLE = \text{argmax}_\theta\ \ell(\theta)$
DL training is MLE of network params

MAP
$\theta_MAP = \text{argmax}_\theta\ [\ell(\theta) + \log p(\theta)]$
MLE + regulariser = log prior

L2 regularisation
$-\lambda\|w\|^2$ = log of Gaussian prior on w
Weight decay = MAP with $N(0,\sigma^2)$ prior

L1 regularisation
$-\lambda\|w\|_1$ = log of Laplace prior on w
Sparsity-inducing

Bayesian update
$p(\theta|D) \propto p(D|\theta)\cdot p(\theta)$
Posterior = likelihood × prior

Conjugate pairs
Beta-Bernoulli · Dirichlet-Categorical · Normal-Normal
Closed-form posteriors

🎲 Sampling & Monte Carlo

Inverse CDF
$U \sim \text{Uniform}(0,1) \rightarrow X = F^{-1}(U)$
Universal sampling recipe

Box-Muller
$Z = \sqrt{(-2 \ln U_1)} \cdot \cos(2\pi U_2) \sim N(0,1)$
Sample Gaussians from uniforms

Rejection
Accept $x \sim q$ with prob $p(x)/(M\cdot q(x))$
Sample p when only q is easy

MCMC (Metropolis-Hastings)
Accept x' with prob $\min(1, p(x')/p(x))$
High-dim Bayesian inference

Reparameterisation
$z = \mu + \sigma\cdot\varepsilon,\ \varepsilon \sim N(0,1)$
VAEs — gradients flow through samples

Gumbel-softmax
$y = \text{softmax}((\log \pi + g) / \tau),\ g \sim \text{Gumbel}$
Differentiable categorical sampling

📐 Inference & Limit Theorems

LLN (weak)
$X_n \to^p \mu$ as $n \to \infty$
Sample mean converges to true mean

CLT
$\sqrt{n}\,(X - \mu) \to^d N(0, \sigma^2)$
Sample mean is asymptotically Gaussian

Standard error
$SE = \sigma / \sqrt{n}$ · or · $s / \sqrt{n}$ if σ unknown
Spread of the sample mean

95% CI for mean
$X \pm 1.96 \cdot SE$ (or t-quantile if small n)
Plausible range of true μ

z-test
$z = (X - \mu_0) / (\sigma/\sqrt{n})$
Compare sample mean to claim

Two-sided p-value
$p = 2\cdot(1 - \Phi(|z|))$
P(more extreme | H_0)

Type I / II
$\alpha = P(\text{reject } H_0 \mid H_0)$ · $\beta = P(\text{fail} \mid H_1)$ · power $= 1 - \beta$
FP rate / FN rate

Multiple testing
Bonferroni: α/m · FDR (BH): rank, compare to $(i/m)\cdot q$
m simultaneous tests